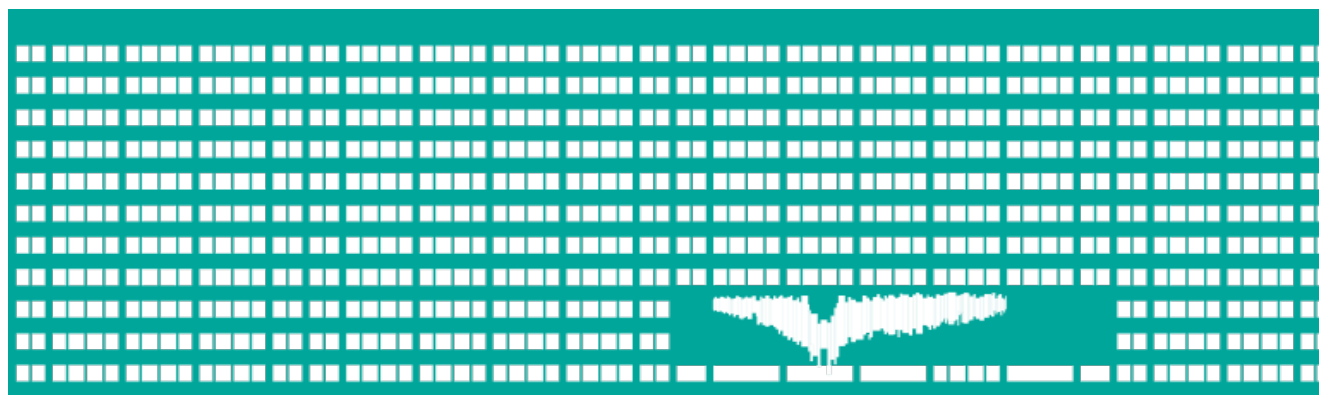


Introducing Mobile Platforms



TAMZ 1
Lecture 1

Contact Information

Name: Pavel Moravec

Office: EA-409

Address: Dept. of Computer Science, FEECS
VSB-TU Ostrava

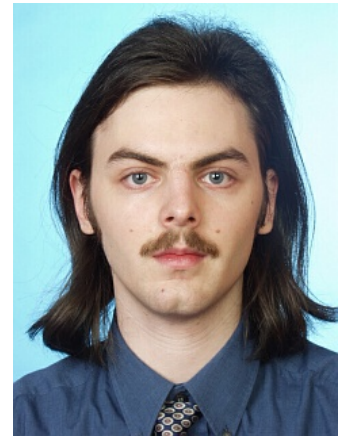
17. listopadu 15

708 33 Ostrava-Poruba

Phone: +420 59 732 5896

E-mail: pavel.moravec@vsb.cz

Web: <http://wiki.cs.vsb.cz/index.php/Moravec/cz>



Course Requirements

During the semester:

- **Project+Implementation** (**15** to 30 points)

Required to complete the course !

- implementation of chosen project for a mobile device + its defense.
- programming of certain assignments in labs, points assigned for each completed assignment (can replace some points of the project)

- **Presentation** (**7** to 15 points)

non-compulsory presentation of pre-selected topic from subject area in labs, 10-15 minutes.

- **Total sum** (**22** to 45 points)

After the end of the semester:

- **Final exam** (**20** to 55 points)

About TAMZ I Subject

What is the subject about?

- General specifics of mobile application development
- Basic development in HTML+ECMAScript (vanilla JavaScript) + frameworks
- Basics of operation with multimedia and 2D game development
- Possibility of developing web frontend to IoT/single-chip solution
- Showing how to create hybrid applications (or develop PWAs)
- Developing in your preferred IDE (or none) on your platform (W, L, MacOS)

What will not be covered in this subject?

- Native Android development (TAMZ II – this subject is not a prerequisite of)
- iOS development – we do not have required HW available for students
- Xamarin development – was considered but would be unfair to Java branch
- Advanced techniques, specific frameworks (react), desktop JS development 4

References

- ~~Onsen UI 2: Beautiful HTML5 Hybrid Mobile App Framework and Tools~~
<https://onsen.io/>
- Apache Cordova API Documentation <http://cordova.apache.org/docs/en/2.4.0/>
- W3 Schools: <http://w3schools.com/>
- P. Retting: Professional HTML5 Mobile Game Development, John Wiley & Sons (14 Sep 2012)
- J. M. Wargo: PhoneGap Essentials: Building Cross-platform Mobile Apps, Addison-Wesley Professional, 2012
- M. Pilgrim: Dive into HTML5, Czech Translation online at:
<https://knihy.nic.cz/files/edice/html5.pdf>
- ...

What is a Mobile Device?

A pocket-sized computing device, typically with:

- a small visual display screen for user output
- a miniature keyboard or touch screen for user input.

Examples:

- **Smartphones**, Tablets
- Handheld game consoles
- IoT devices, monitoring devices, “smart” devices
- Navigation devices
- Mobile phones
- Also: Cameras, Walkie-talkies, portable media players, old PDAs, ...
- ...

Limitations of Mobile Devices

- Small (often lower-resolution) display (on some types of devices even OLED/grayscale/BW)
- Limited input options (phone/software touch keyboard, set of a few, e.g. 1-3 hardware buttons or 3x4 keyboard with extra function buttons)
- Lower operating memory & internal storage space
(the latter solved by external memory cards on devices supporting them)
- Limited CPU power + absent or limited GPU capabilities on some devices
- Limited instruction set (e.g. only software-emulated floating point arithmetic on historic/limited platforms – ATmega328, no SSE, vector-processing instructions)
- Network connectivity issues
- Smaller library of system functions
- Power considerations
- ...

Mobile Operating Systems

- Vendor-specific firmware, vendor SDKs for custom builds
 - SDKs for Espressif ESP*, CMSIS SDKs
- Real-time operating systems – e.g. FreeRTOS
- **iOS** on Apple devices (phones, tablets, media players)
- **Android**
- Other Linux-based firmware:
 - *WRT – OpenWRT, DD-WRT for networking/communication devices
 - ~~Firefox OS~~ → KaiOS (dies a slow death, cheap feature phones)
 - ~~Ubuntu Mobile~~
 - ~~Tizen, Sailfish OS, Jolla,~~
- Windows – mainly on tablets (Surface)
 - ~~Windows Phone 8.x, Windows RT (8), Windows CE~~
- ~~BlackBerry OS, Samsung BADA, Symbian OS~~

Mobile Frameworks

- **HTML5-based frameworks or basic PWA**
 - AngularJS, Onsen UI, jQuery Mobile, Sencha Touch, vue.js,
 - Game frameworks: Phaser (v3), GDevelop, BabylonJS, Construct, Modd.io, HTML5 Quintus, ...
 - Cordova, Ionic (with Capacitor), React (with ReactNative), PhoneGAP, ...
 - Electron-based apps for desktop
- **Xamarin**
- **Game frameworks:** Unity 3D, Cocos 2D, Corona SDK, libgdx, ...
- QT
- CodeName One
- 3rd party frameworks (of varying quality) based on
 - Java, Python/MicroPython, LUA, ...
- Obsolete: ~~Flash Lite~~, ~~JavaME~~ — ~~Java 2 Micro Edition~~, JavaFX

Programming languages

- **HTML5** + Javascript/TypeScript/ECMAScript 8-14 + CSS3
- **Java** – special-purpose devices, Android^(*), libgdx, ...
- **Kotlin** – Android
- **Objective C, Swift** – iOS
- **C/C++** – native libraries, e.g. Android NDK, single-chip solutions, custom firmware development, QT, RTOSes
 - Processing language (Arduino)
- **C#** – Windows, **Xamarin**, **Unity3D**, ~~.NET CF~~
- **LUA** – some game frameworks, Smart TVs, OpenWRT
- **Python**
- **Exotic choices**: Shell script, PHP, ...
- ...

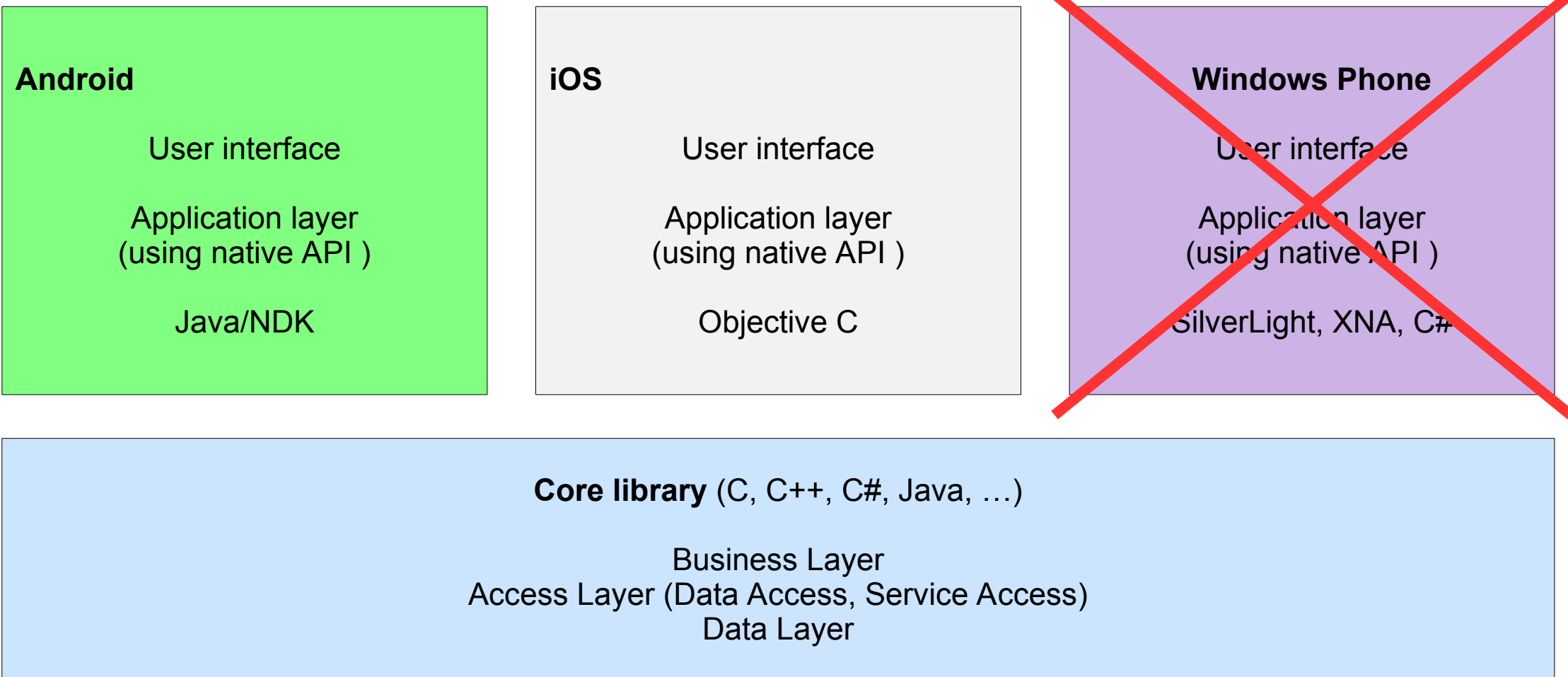
New platforms changed everything!

- Device fragmentation → Platform fragmentation
 - Platform fragmentation is getting “better” in recent years as we have 2 major p.
- Vendors try to keep customer by vendor lock-in
 - Applications sold by vendors directly through their Play Store, AppStore, etc. including some manufacturers like Samsung
- 2 dominant platforms on smartphones and tablets:
 - Android (~70%)
 - iOS (~29%)
 - Samsung (~0.3%)
 - KaiOS (~0.15%)
 - ~~Windows (Windows Phone 8/10, Windows Mobile), Blackberry (US) / Symbian (EU)~~
- Feature phones may still offer Java ME, but they often don't bother anymore (but Android emulator exists), or use KaiOS (HTML5-based API)

Why is platform fragmentation bad?

- Different environment
 - Different APIs
 - Different programming languages (Java, Objective C, Swift, Kotlin, C#, C/C++, LUA, Python, etc.)
 - Different SDKs, specific OS needed for SDK and development (iOS requires a computer with MacOS to build apps in XCode, some needed Windows)
- The code needs to be written several times
 - We may use “Native libraries” (usually in C/C++)
 - Platform-independent SDKs (e.g. CodeName One, JavaFX, QT)
 - HTML 5 for simple applications – Cordova, Ionic, React
 - Android/JavaME had basic common parts in Java, GUI/API calls had to be solved separately, not many developers transformed apps to Android APIs

Multi-platform Development 2010s



Present Multi-platform HTML5 Development

HTML5 Frontend (HTML5, CSS, Javascript/ECMAScript, ...) running in **WebView**

Presentation Layer
Platform/API Adaptation Layer

Android

Application layer
(using native API)

Java/Kotlin + NDK

iOS

Application layer
(using native API)

Objective C/Swift

Desktop

Application layer
(using native API)

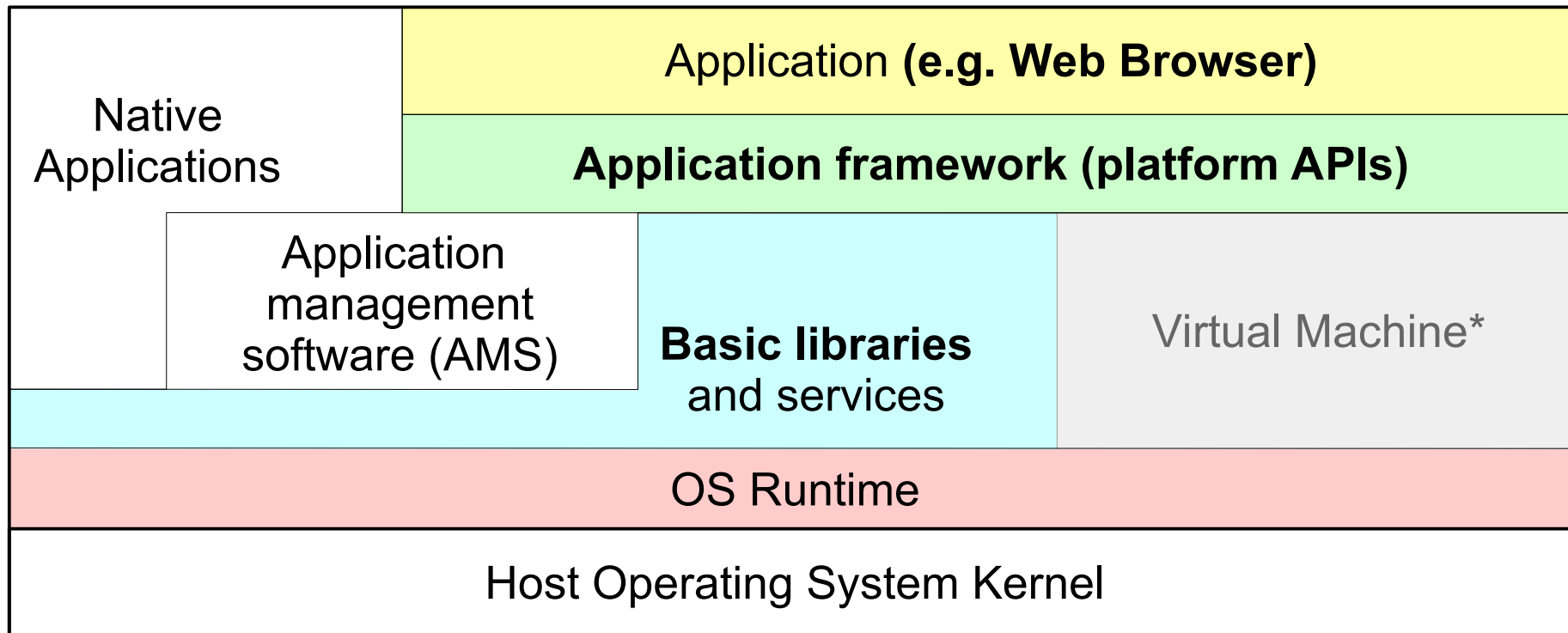
C#/Java/C/C++/...

Core library (C, C++, C#, Java, ...)

Business Layer
Access Layer (Data Access, Service Access)
Data Layer

Mobile Platforms Common Features

Generic Mobile OS Architecture



* Virtual machine is not used on some platforms, which run native code directly.

Usually, we use **platform APIs** for development, but on most platforms we can also write **native code** using built-in or even user-supplied **libraries**, or a HTML+JS+CSS application run in the web **browser** or embedded web view.

Handling Security

Any seriously considered mobile OS needs protection from **malicious code** that may access system information or resources.

Advanced security model provided by desktop OS may be too memory-consuming (although frameworks like SELinux, verified boot, hardware-based encryption, and full filesystem encryption have already been included in latest Android versions).

The classic security model, which is provided in most mobile devices, is the **sandbox model**, where the applications are run in a closed environment in which they can only access insensitive resources, the access outside of the sandbox must be granted by the user.

Typically, all sensitive APIs have to be listed in application manifest (in some cases the application using these APIs may have to be signed) and are part of a permission model. Also, some APIs require (repeated) runtime permissions.

Java on Mobile Platforms

Why has Java been so popular?

- Java was originally intended for embedded and mobile development.
- Many old-time mobile developers were used to program in JavaME, which was easy-to understand platform with well-defined standards (jcp.org)
 - Still may be the only choice for feature phones
- Android uses Java bytecode as a step in developing mobile applications.
- Java-base SDKs (Netbeans/Eclipse) are available for all 3 major platforms (Windows, MacOS, Linux)
- But you may develop HTML5 apps in them as well

Official Java Editions

In 1995 the first version of Java, now known as Java Platform Standard Edition (JavaSE), was released. Since that time its landscape has changed significantly.

Standard Edition (JavaSE)

Designed to run on desktop and workstation computers.

Enterprise Edition (JavaEE)

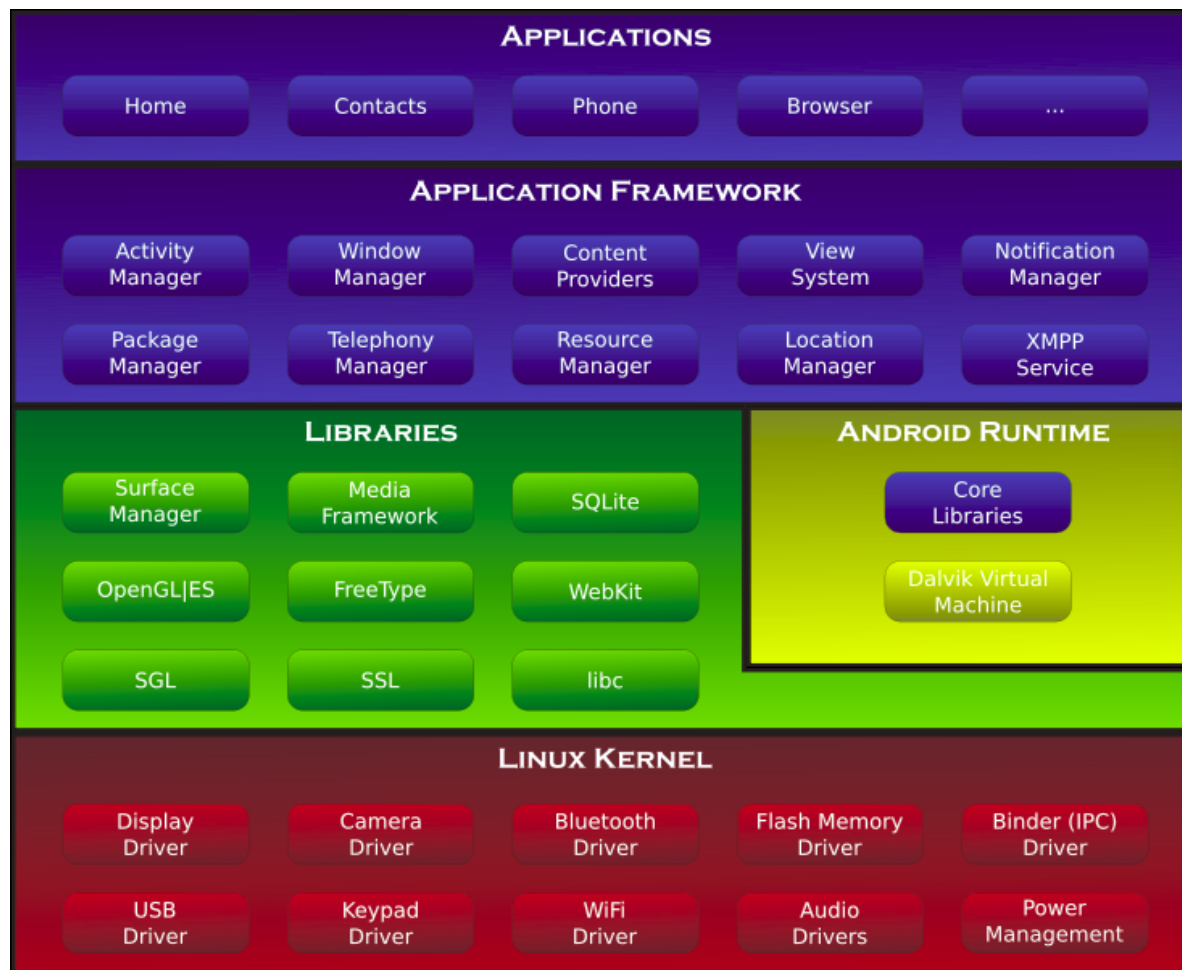
This edition is aimed at server-based applications. Contains built-in support for Servlets, JSP and XML.

Micro Edition (JavaME)

Designed for devices with **limited** memory, display and processing power.

Java Card – programming of Smart Cards (e.g. SIM, payment)

Historical Android OS Architecture



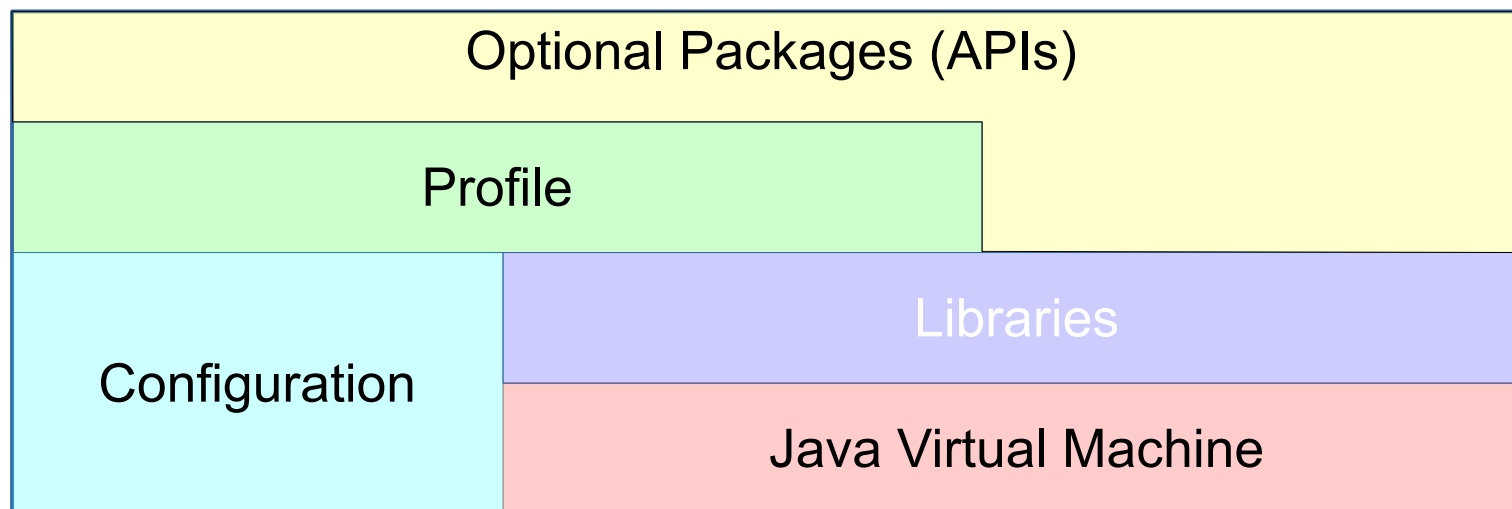
Example of a formerly widespread mobile Platform

—

Java ME
(for reference only)

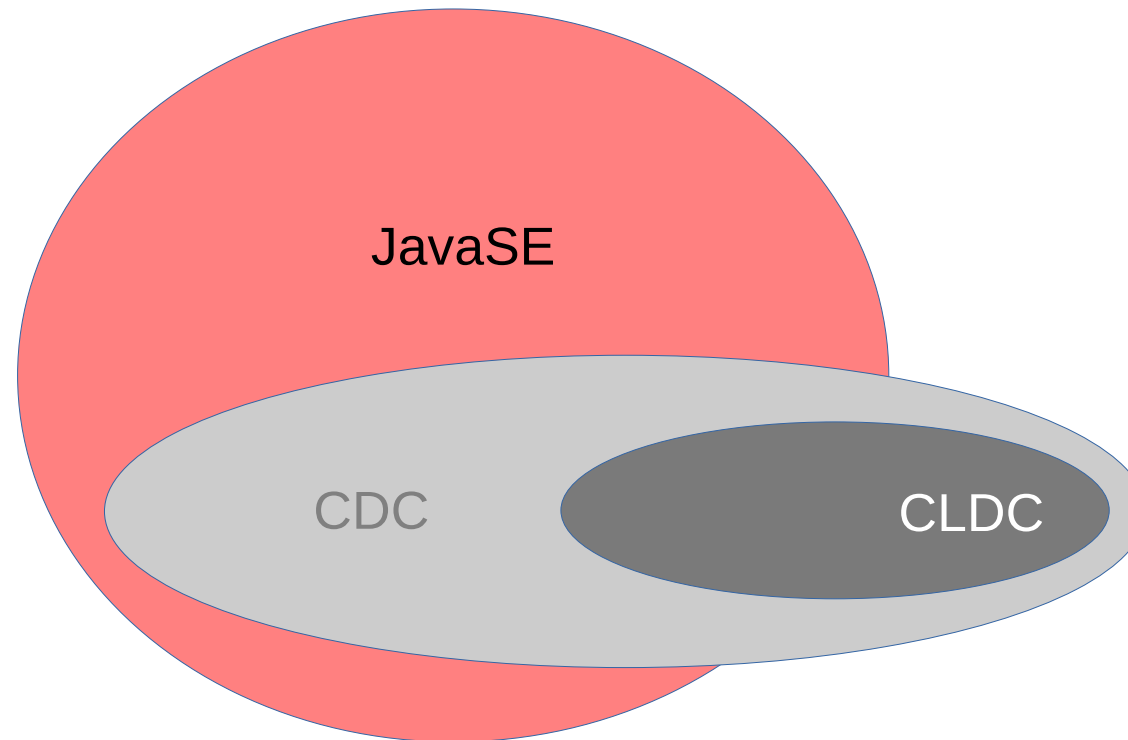
Configurations and Profiles

Capabilities of devices within the JavaME may vary greatly. A PDA may have a much larger display than a pager. One Java platform will most definitely not fit all. In order to accommodate a broad range of consumer electronics and embedded devices JavaME introduces two new concepts: **configurations** and **profiles**.



CDC versus CLDC

CDC is a subset of JavaSE, plus some extra classes
CLDC is a **subset** of CDC.



Connected Device Configuration

This configuration is designed for devices with more memory, faster processors and greater network bandwidth. It is appropriate, at least in the near term, for home automation and automotive navigation, entertainment and telemetry systems.

A programming model closer to JavaSE simplifies porting existing desktop clients for enterprise systems to mobile devices that support CDC.

CDC has never been used much outside of Multimedia Home Platform (MHP) and some handheld game consoles.

Connected Limited Device Configuration

This configuration is intended for devices with intermittent network connections, small processors and limited memory. Expected targets included two-way pagers, mobile phones and entry-level PDAs.

However, in practice, the functionality delivered by CLDC and the associated profiles and optional packages is very close to that of CDC. As a consequence it is used today on most high-end mobile phones, or smartphones, which are replacing PDAs in the marketplace.

Java Language Differences

There are three main areas that distinguish the Java Language for CLDC to those defined in the Java Language Specification.

Floating Point Math

The CLDC 1.0 implementation of the Java language **does not support** floats. CLDC 1.1 specifies such support. But MIDP 2.0 only requires CLDC 1.0.

Finalization

The CLDC does not support the `finalize()` method because an overhead and processor requirements are too steep.

Error Handling

The JVM supports a limited set of error handling exceptions due a significant demand on the system's device-dependent.

JavaME Java Virtual Machines

C virtual machine (CVM)

- classic Java virtual machine (not Dalvik!)
- used with CDC
- requires 32b microprocessor
- at least 2MB of RAM & 2.5 MB of ROM

Kilobyte virtual machine (KVM)

- used with CLDC
- 16b/32b (slow) microprocessor
- limited math, string, and I/O functions
- lacks some features of classic JVM
- only a small subset of JavaSE core libraries
- at least 32kB of RAM & 128kB of ROM

Profile

A profile supplements a configuration to provide capabilities for a specific family of devices or specific industry segment application need.

For example, a configuration provides a generic Java API for battery powered, limited display and processing capability systems, while a profile provides a supplemental Java API specific to cellular telephones, which is a member of this larger group of devices. A profile must supplement a configuration.

A profile will not work without the configuration and virtual machine providing the core classes/API and runtime environment.

Profiles

CDC profiles

- **Foundation profile** – basic CDC profile
- **Game Profile** – developing game applications
- **Personal Profile** – complex UI components FP
- **Personal Basis Profile** – simple UI components (single window)
- **RMI Profile** – Remote Method Invocation support classes

CLDC profiles

- **Mobile Information Device Profile** – local storage, UI, network
- **PDA Profile** – better displays, larger memory compared to **MIDP**
- **KJava** – Sun-specific API for Palm OS
- **Information Module Profile** – subset of MIDP 1.0 without its display and input classes

Mobile Information Device Profile

The Mobile Information Device Profile (MIDP) is a key element of the Java 2 Platform, Micro Edition (JavaME). When combined with the Connected Limited Device Configuration (CLDC), MIDP provides a standard Java runtime environment for today's most popular mobile information devices, such as cell phones and mainstream personal digital assistants. The profile can be characterized by the following features:

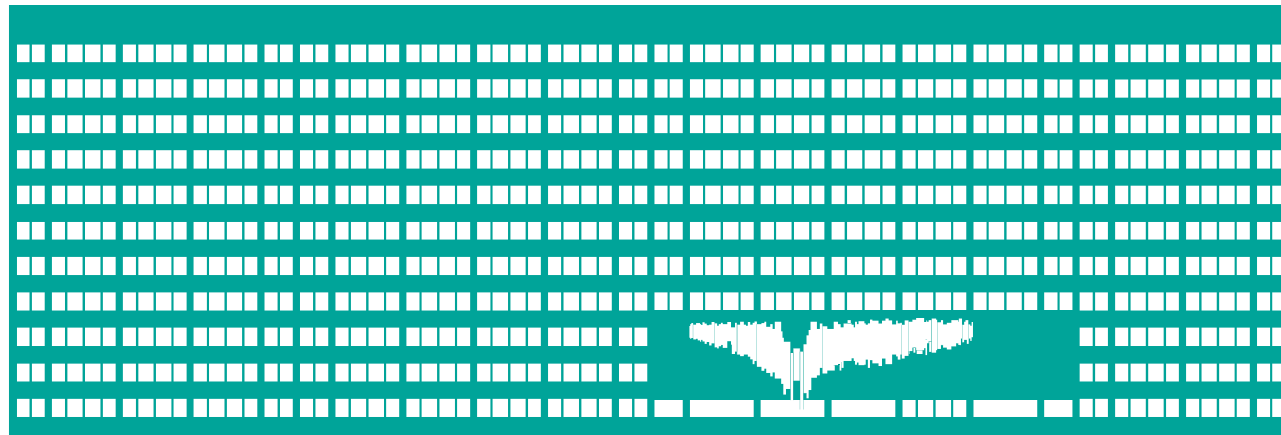
- rich user interface capabilities
- extensive connectivity
- multimedia and game functionality
- Over-the-air-provisioning
- end-to-end security

MIDP Optional Packages

Once again the basic library was insufficient for some applications (remember the JavaSE history), which resulted in the development of several new packages:

- **PDA Optional Package** – more PDA-specific APIs, e.g. PIM, files
- **Bluetooth API** – Bluetooth communication (SDK problem)
- **Wireless Messaging API** – SMS (MMS) capabilities
- **Location API** – location tracking
- **Presence & IM API** – JavaME version of JAIN, IM & presence support based on SIP
- **Mobile Media API** – audio/video controls, streaming media
- **JavaME Web Services API** – Web Services-related XML classes
- **Security and Trust API** – Interaction with security smart card
- **Mobile 3D Graphics API** – 3D graphics, mainly for games
- ...

Over-the-Air Programming (OTA) HTML+JS+CSS



TAMZ 1
Lecture 2

Over-the-Air (OTA) Programming

- The software is installed without connecting the device to a PC or uploading it to memory card
- Used for distribution of OS and software updates, encryption keys, configuration
- Possible both for mobile device and (U)SIM/WIM card
- Typically through mobile broadband or Wi-Fi network
 - In older platforms, OTA over Bluetooth was also used
 - SMS OTA **update** (esp. for SIM card) also possible
- Types:
 - over-the-air service provisioning (OTASP)
 - over-the-air provisioning (OTAP)
 - over-the-air parameter administration (OTAPA)
 - over-the air backup/restore

OTA on Recent Platforms

- Whole mobile OS/firmware update
 - Complete download (can be extremely large > 200MB)
 - Incremental update (often requires unchanged system)
 - Used by most vendors on Android, Windows Phone, ...
- Application updates and new installation
 - Large updates may be accessible only through fast data connections (WiFi, LTE, 4G)
 - Automatic application/applic. data update/installation
 - Externally executed installation (e.g. Play Store web)
 - Automatic malware removal (when discovered in the store/on device)
- SIM updates – core firmware, SIM toolkit, certificates
- Operator-based settings (e.g. APN for data/MMS)

OTA & HTML pages on server

- The web pages and resources are downloaded dynamically in browser
- Local storage of application data may be used
- Off-line applications (e.g. Google docs may be used without Internet connection on a PC to a degree)
- Verification of resource/page modification through HTTP protocol headers (If-Modified, Etag)
- If the application changes often, inconsistencies may be possible
 - Cached vs. newly downloaded data
 - Reload of the whole page may be necessary
 - Forced changes of application manifest to indicate changes.

HTML Basics Refresher

Cheat sheet e.g.: <http://goo.gl/2ZeJw>

HTML Document Structure

<!DOCTYPE html>

<html>

<head>

...

</head>

<body>

...

<!-- Invisible comment -->

</body>

</html>

Element Tags (case insensitive, you should use lowercase)

- **<tag>content</tag>**
- **<tag attribute="val">content</tag>**
- **<tag_without_content ... />**

Entities: **&entity;** or **&#number;**

- **&**; **&**, **>**; **>**, **<**; **<**, ** **; ...

Attributes:

- **attr="value"** or **attr='my "extra" val'**

Doctypes for older HTML versions

- **XHTML (2000):** <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
- **HTML 4.01 (1999):** <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">

Default HTML attributes (optional)

Attribute	Meaning
id	Element identifier (unique in the whole document)
class	CSS class(es) of the element
style	Inline definition (or override) of element style
title	Extra information displayed as tooltip (mouse over element on desktop)
lang	Language code of the element (e.g. en or cs)
tabindex	Ordering elements when user moves through the page (starts with 1)

Notes:

- In jQuery, JS selectors and CSS, id is referenced by *#id* and class is referenced by *.class*
- Inline styles should be avoided where class can be used.

More details can be found at:

http://www.w3schools.com/tags/ref_standardattributes.asp

HTML headings, breaks, paragraphs

- Several levels of headings (like chapters, sections, subsections)

<h1>Heading 1</h1>

<h2>Heading 2</h2>

<h3>Heading 3</h3>

<h4>Heading 4</h4>

<h5>Heading 5</h5>

<h6>Heading 6</h6>

- Line breaks: Line 1**
**Line 2

- Horizontal lines: **<hr/>**

- Paragraphs

<p>Paragraph text</p>

<p>Second paragraph</p>

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

Line 1

Line 2

Paragraph text

Second Paragraph

HTML text formatting (when needed)

`<b>Bold text</b>`, `<i>Italic text</i>`,
`<b><i>Bold italic</i></b>`

`<strong>Important text</strong>`
`<em>Emphasized text</em>`

`<small>smaller text</small>`

`<sub>`_{subscript}`</sub>` and `<sup>`^{superscript}`</sup>`

`<del>Delete</del>` or `<ins>insert</ins>` text and
`<mark>Mark it</mark>`, `<pre>Preformatted text</pre>`

`<dfn>Definition term</dfn>`

`<code>A piece of computer code</code>`, `<samp>its output</samp>`
`<kbd>Keyboard input</kbd>`, `<var>Variable</var>`

Note: Additional tags @ http://w3schools.com/html/html_formatting.asp

Bold text, *Italic text*, ***Bold italic***

Important text *Emphasized text*

smaller text subscript and superscript

Delete or insert text and **Mark it**,

Preformatted text

Definition term

A piece of computer code, its output. Keyboard input, *Variable*

Note: Added paragraphs between parts for better readability

HTML – links and images

Top

Links: [Go to the top](#)  [VSB](http://www.vsb.cz)

- Links:
 - Link target in the page: `<a id="anchor">Top</a>`
 - `<a href="#anchor">Go to the top</a>`
 - `<a href="http://www.vsb.cz" target="_blank">VSB</a>`
- Link attributes:
 - **href** – destination URL
 - **rel** – relationship between current and linked document
 - **target** – where to open the link (`_blank` | `_parent` | `_self` | `_top` | `name_of_frame`)
- Images (may be placed inside of links):
 - ``
 - ``
 - **usemap="#map"** for usage of local `<map>` and `<area>` tags (see http://w3schools.com/tags/tag_area.asp) and **ismap="true"** for sending click coordinates in image to the server

HTML – blocks and lists

- Basic blocks
 - **<div>** – separate block, typically starts on a new line
 - **** – inline block (like)
- Lists – can be of nested – more levels
 - **** – unordered list (e.g. bullets)
 - **** – ordered list (numbered)
 - **** – list item
 - **<dl>**, **<dt>** – keyword , **<dd>** - description
- Tables
 - Often replaced by divs nowadays
 - **<table>**, **<tbody>**
 - **<tr>** – table row
 - **<th>** – table header cell, **<td>** – table cell

```
<ul>
  <li>One</li>
  <li>Two</li>
</ul>

<ol>
  <li>One</li>
  <li>Two</li>
</ol>

<div>This is a div
and here is the
<span>span inside.
</span></div>
```

- One
- Two

1. One
2. Two

This is a div and here is the span inside.

HTML – forms (intro)

- **<form action="demo_form.asp" method="post">**
 - **enctype** may be **multipart/form-data** for forms with files, **action** specifies the target and **method** the HTTP methods (GET/POST)
- Inside the form are following basic GUI elements:
 - **<input type="text" name="x" id="i1" value="Hi"/>**
 - **type**: **text**, **password**, **hidden** input, **buttons** (including **submitting** and **resetting** the form), **radio** button or **checkbox**, **file**, **image** submission button
 - the **value** tag contains pre-set value of the input.
 - **checked** – for checkboxes and radio buttons
 - **disabled**, **readonly**, **maxlength** – maximum number of input in characters, **size** – visual size in characters
 - **<textarea rows="4" cols="3" ...>** – multiline input
 - **<fieldset>**, **<label>** for **<input>**, **<legend>** for **<fieldset>**, **<select>** – drop down list

HTML – form example

```
<form action="test.php" method="post">
```

```
<label>Name: <input name="name" type="text"/></label>
```

```
<label>Student: <select name="stype"><optgroup label="Czech">  
  <option value="P">Full-time</option><option value="K">Combined  
  </option></optgroup><option value="E">Erasmus</option>  
</select></label>
```

```
<label>I am paying attention <input  
  name="att" type="checkbox"/></label>
```

```
<fieldset>
```

```
<legend>Gender</legend>
```

```
<label><input type="radio" name="gender"  
  value="M" checked="yes"/>Male
```

```
</label>
```

```
<label><input type="radio" name="gender"  
  value="F"/>Female</label>
```

```
</fieldset>
```

```
<input type="submit" value="Submit"/>
```

```
</form>
```

Full-time
Czech
Full-time
Combined
Erasmus



Name:

Student:

Full-time

☐ I am paying attention

Gender

☒ Male

☐ Female

Submit

HTML – `<head>` contents

- `<meta />` – tag containing meta information (normally present in HTTP headers)
- `<title>` – document title as displayed by browser
- `<base href="..." />` – base URL prepended to all relative URLs on the page (current path by default)
- `<link rel="stylesheet" type="text/css" href="style.css" />`
 - External CSS style sheet. Local CSS can be specified by `<style>` tag
 - e.g. `<style type="text/css">p {color:blue;}</style>`
- Not limited to head section, but typically placed there:
 - `<script>` – code, evaluated in the browser
 - `<script type="text/javascript" src="ex1.js"></script>`
 - `<noscript>` – alternate content for users without scripting

HTML – special sequences

- Encoding of character codes
 - In HTML – through entities – `∑`, `∑`, `∑`
 - In URL – through encoding 8-bit characters as hexadecimal sequences `%20` – space (but we use `+` instead), `%C4%8D` – Unicode sequence of letter `č`
 - Charset of the whole document – `<meta>` in `<head>`
 - HTML<5: **`<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">`**
 - HTML5: **`<meta charset="UTF-8">`**
- Encoding of colors
 - **`#RRGGBB`** – color components are hexadecimal numbers between 00 (no light) and FF (full light), e.g. `#FFFFFF` – white, `#000000` – black.
 - 17 named colors: e.g. black, blue, gray, green, silver

JavaScript Basics

Cheat sheet e.g.: <http://goo.gl/s9sVy>

JavaScript

- Permits in-browser document modification (through Document Object Model – DOM) and client-side scripting
- Prototype-based (interpreted) programming language with dynamic typing
- Supports object-oriented, functional and imperative programming styles
- Formalized as ECMAScript
- Also for server side, currently e.g. the Node.js framework
- Platform-independent (browser-dependent)
- Syntax based partly on Java/C/C++
- Can be still disabled (NoScript) or may not be present (e.g. in really basic browsers)

JavaScript – basic constructs

- Operators known from C/C++/Java
 - Arithmetic: **+** **-** ***** **/** **%**, prefix and suffix **++** **-**
 - Assignment: **=** **+=** **-=** ***=** **/=** **%=**
 - Comparison: **==** **!=** **<** **>** **<=** **>=**, *strict* equality **===** **!==**
 - **"123" == 123**, but **"123" !== 123**
 - Logical operators: **&&** **||** **!** **^(XOR)** **<<** **>>** (shift left/right)
 - Assignments: **&=** **|=** **^=** **<<=** **>>=** **>>>=**
 - **+** for string concatenation (Java), **;** for statement separation
 - **,** for separation of arguments, **.** for sub-objects, attributes
 - Ternary operator **x?x_true:x_false**, code blocks: **{ }**
- Code constructs known from C/C++/Java
 - **if () { } [else { }]**, **switch () {case 'ABC': ... break; default:}**
 - **for(x;y;z) { }**, **for(x in object) { }**, **while() { }**, **do { } while ()**
 - Cycle control: **break**, **continue**
- Logical values: **true false**, Null pointer: **null**

JavaScript – comments, variables

- Comments:
 - `/* Whole block */`
 - `//Comment to the end of line`
- Variables
 - **`var x;`** //x is set to **undefined**
 - **`var x = 3, y=5, z=true;`** //Create vars and assign values
- Assignment to undeclared variable makes it global

JavaScript – arrays and objects

- Arrays (more info later)
 - **var cars=new Array(); cars[0]="Saab"; cars[1]="Volvo";**
 - Condensed: **var cars=new Array("Saab","Volvo");**
 - Literal: **var cars=["Saab","Volvo"];**
- Objects
 - **var person=new Object();**
 - **var person={firstname:"John", surname:"Doe"};**
 - Accessing fields: **person.firstname** or **person["firstname"]**
 - **var z = new String(), x=new Number();**
 - **person.firstname.length** returns 4

JavaScript – functions

- Functions and methods are defined the same way
- Simple recursive function (factorial)

```
function fac(n) {  
    if (n === 0) { return 1; }  
    return n * fac(n - 1);    }
```

- Anonymous functions

- written directly in code (**function**()) { /* do something */ }();
- \$(document).ready(**function**() {alert("Document ready");})

- Variable number of arguments in a function

```
var avg = function() {  
    var i, x = 0.0; //we could place var i in for, i.e. for(var i=0  
    ...    for (i=0; i < arguments.length;i++) { x +=  
    arguments[i]; }  
    return x / arguments.length;  
}
```

- Lambda functions are also available ...

JavaScript – other features (1)

- Object prototypes – “classes” instantiated with **new**

Example(...) { ... }

Example.prototype = { // Can be also used for inheritance

constructor: Example,
method1: function(arg) { ... },
method2: function() { ... }
}

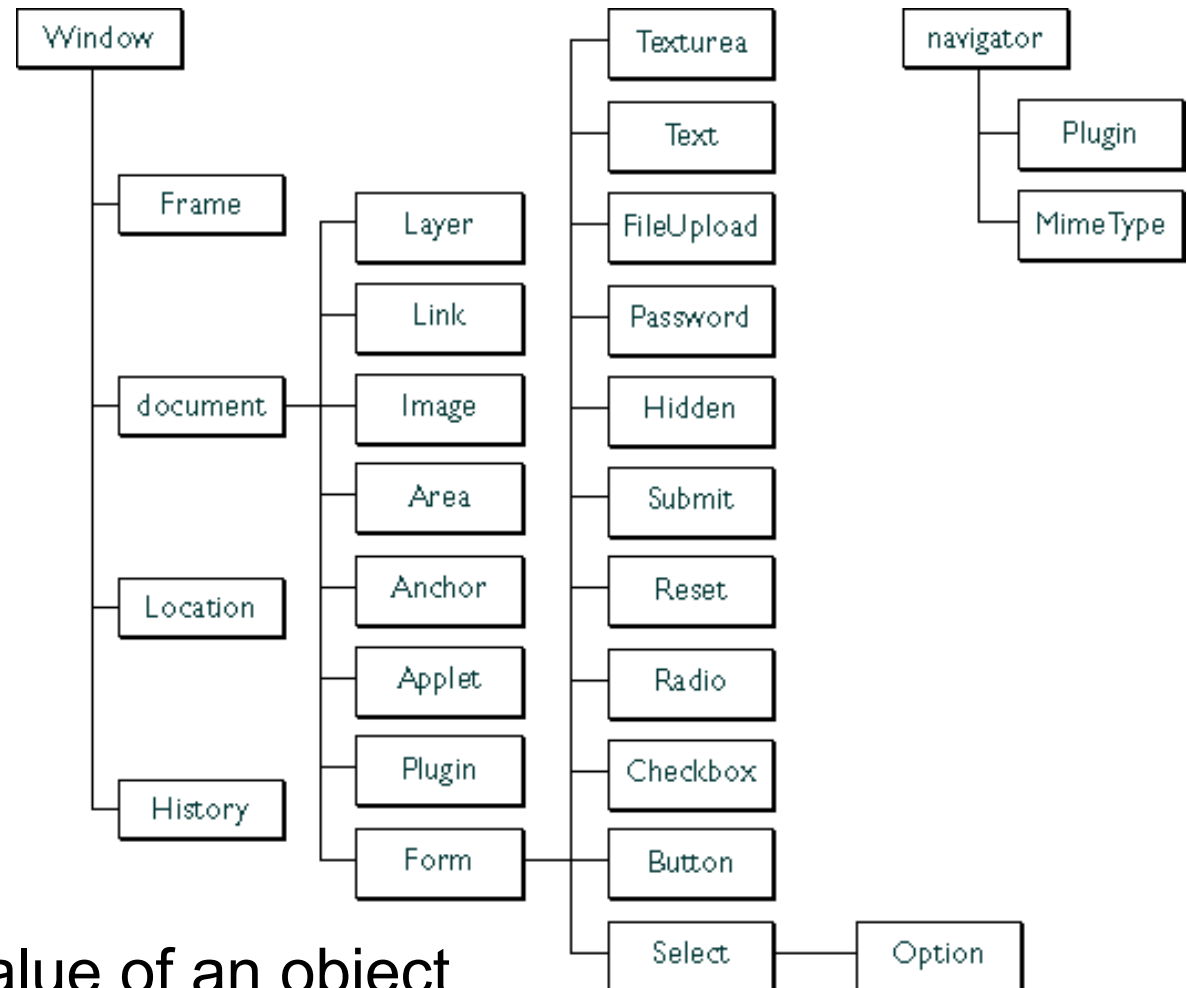
- Operator **typeof** – detecting type of a value
- Keyword **this** – the object on which a method is invoked

JavaScript – other features (2)

- Closures – making sure the value becomes fixed
 - `var createF = function(i) { return function()
{ alert(i); }; };
for (var i=0; i<3; i++) {
 $('<p>click
me</p>').appendTo('body').click(createF(i));
}`
- Error handling – **try/catch(error)/throw "message"**
- *Note: Detailed OOP tutorial e.g.
<http://phrogz.net/JS/classes/OOPinJS.html>*

JavaScript – basic object hierarchy

- DOM objects →
- Core Objects
 - Array
 - Boolean
 - Date
 - Math
 - Number
 - String
 - *Function*
 - *RegExp*
- .valueOf() – primitive value of an object
- toString() – string conversion



Note: More info about language objects:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Predefined_Core_Objects

JavaScript – Numbers

- All numbers are 64-bit Floating Point
- Octal (**0**12 == 10) & hexadecimal (**0x**C == 12) numbers
- Precision problems: $0.1 + 0.2 \rightarrow 0.3000000000000000000004$
- Number can be Object or Number, comparison problems
 - `123 !== new Number(123) // typeof(123) → Object`
- Special object properties
 - Overflow **Infinity** (also $x/0$, $x > 0$) **-Infinity** (also $x/0$, $x < 0$)
 - **NaN** (`123 / "ahoj"`, `0/0`)
 - **Number.MIN_VALUE**, **Number.MAX_VALUE**

JavaScript – Number methods

- Methods
 - **valueOf()** → primitive value of Number object
 - **toString([radix])**
 - e.g. `new Number(12).toString(2)` → 1100
 - **toPrecision([digits]),**
toExponential/toFixed([decimal_pl])
 - e.g. `(12.333).toFixed(1)` → 12.3, `.toFixed()` → 12
 - `(12.3).toPrecision(1)` → 1e+1, `.toExponential(1)` → 1.2e+1

JavaScript – Numbers – Math

- Calculations similar to `java.lang.Math`
- Special object properties
 - **E** – Euler's number e , **LN2** – natural logarithm of 2
 - **LN10** – natural logarithm of 10
 - **LOG2E**, **LOG10E** – base-2, base-10 logarithm of e
 - **PI**, **SQRT1_2**, **SQRT2** – π , $\sqrt{1/2}$, $\sqrt{2}$
- Methods
 - **abs(x)** – $|x|$, **min(x,y,z,...)**, **max(x,y,z,...)**
 - **asin(x)**, **acos(x)**, **atan(x)** – $\arcsin$ of x , where x is in radians
 - **atan2(y,x)** – returns the arctangent of the quotient of its args
 - **ceil(x)**, **floor(x)**, **round(x)** – rounds up/down/to nearest int
 - e.g. `ceil(1.22)` → 2, `floor(1.22)` → 1, `round(1.22)` → 1
 - **sin(x)**, **cos(x)**, **tan(x)** – goniometric functions (in radians)
 - **exp(x)** → e^x , **log(x)** → $\log_e(x)$, **pow(x,y)** → x^y , **sqrt(x)** → $\sqrt{x}$
 - **random()** – a random number between 0 and 1

JavaScript – Strings

- Simplest usage: `var xxx = "Hello";`
- Strings are enclosed by `" "` or `' '`, escaping of characters: `\\ \n \`
- Special object property: **length** – number of characters in string
- Standard methods
 - **charAt(i)/charCodeAt(i)** – character/unicode val at index *i*
 - **fromCharCode(code)** – converts unicode value to character
 - **concat(a,b,...)** – joins two or more strings
 - **indexOf(), lastIndexOf()** – position of the first/last match
 - **x.localeCompare(other)** – compares two strings in the current locale, returns -1 (other is after x), 0 (`x == other`), 1
 - **toLocaleLowerCase(), toLocaleUpperCase(), toLowerCase(), toUpperCase()** – letter case conversion
 - **split(separator[, limit])** – splits a string into an array of substrings on separator (char/regexp) at most limit times.
 - **trim()** – removes whitespaces from both ends of a string

JavaScript – String manipulation

- Pattern matching
 - **match()** – returns matches between reg. exp. & a string
 - **replace(x, y)** – returns a new string with all occurrences of x replaced by y
 - **search()** – like **match()**, returns the position of the match
- Substring manipulation
 - **substring(start[, idx2])** – extracts substring from a string, beginning at a specified start position, ending at idx2-1. Swaps start and idx2 when idx2 < start
 - **slice(start[, idx2])** – like substring, but does not swap arguments and accepts negative numbers (references from the end of the string), e.g. slice(-3).
 - **substr(start[, length])** – extracts substring from a string, like slice but the second argument is length, not position.

JavaScript – Arrays

- Associative arrays are possible (see objects earlier)
- Array may also work as a *queue* or a *stack*
- Special object property: **length** – number of array items
- Methods
 - **concat**(y, ...) – returns a join of two or more arrays
 - **indexOf**(elem), **lastIndexOf**(elem) – returns element's position in array (or -1 if the element is not found).
lastIndexOf – start the search from the end of array.
 - **join**() – joins all elements of an array into a string
 - **slice**() – selects a part of an array, like slice() for strings
 - **sort**() – Sorts the elements of the array
 - **reverse**() – reverses the order of the elements in an array
 - **splice**(index, remove_count[, add1, add2, ...]) – adds and removes elements from an array, e.g. splice(2, 1, "AAA") will remove the third element and replace it with "AAA"

JavaScript – Arrays – extended use

- Element manipulation methods – stack, queue
 - **pop()** – removes the last element of an array, and returns it
 - **push()** – adds a new element (or elements) to the end of an array, returns the new array length
- **shift()** – returns the first element of an array, removes it
- **unshift()** – like push, but for the beginning of the array

JavaScript – Dates (1)

- Date objects created by calling new Date()
 - new Date(); // current time & date
 - new Date(milliseconds); // since 1.1.1970
 - new Date(dateString); // Parse date from specified string
 - new Date(year, month, day, hours, minutes, seconds, ms);
- Obtaining date components
 - **getDate()** – day of the month (1-31), **getDay()** – DoW (0-6)
 - Sunday → 0, Monday → 1, ..., Saturday → 6
 - **getMonth()** – month (Beware! **0-11** NOT 1-12)
 - January → 0, December → 11
 - **getFullYear()**, **getHours()** – 0-23, **getMinutes()**, **getSeconds()**
 - **getMilliseconds()** – 0-999
 - **getUTC*()** – same methods as **get*()**, using UTC time instead local
 - **getTime()** – number of milliseconds since 1970-01-01 0:00
 - **valueOf()** will also return this value
 - **getTimezoneOffset()** – the time difference in minutes between UTC and local time

JavaScript – Dates (2)

- Setting date components (we can also use `d.setXXX(d.getXXX()-Y)`)
 - **setDate()** – day of month, **setFullYear(year[,month, day])**, **setMonth()**
 - **setHours(h[,m,s,ms])**, **setMinutes()**, **setSeconds()**, **setMilliseconds()**
 - **setUTC*()** – same methods as **set*()**, using UTC time instead local
 - **setTime(milliseconds)** – number of milliseconds since 1970-01-01 0:00
- Parsing dates
 - **Date.parse(date_str)** – returns the ms for date_str like `getTime()`
 - **Date.UTC(year, month, day[, h, m, s, ms])** – as above, numeric args
- Converting dates to strings
 - **toDateToString()/toTimeString()** – converts the date/time portion into a readable string
 - **toString()** – converts whole date into readable string
 - **toLocaleDateString()/toLocaleTimeString()/toLocaleString()** – the same as abovementioned methods, but using locale conventions
 - **toUTCString()** – like `toString`, but using universal time
 - **toJSON()** – returns JSON-formatted ISO date string
 - **toISOString()** – the same, not supported in IE < 8
- Comparison with standard operators (`==`, `<`, `>`, `<=`, `>=`, ...) also works

JavaScript – Usefull globals

- Pre-defined global properties (already mentioned)
 - **NaN**, **Infinity**
 - **undefined** – variable has not been assigned a value
- Pre-defined global functions
 - **decodeURI()**, **decodeURIComponent()** – decodes URI/its component
 - **encodeURI()**, **encodeURIComponent()** – encodes URI/its component
 - encodeURI: “http://a.b/c=Ahoj !” → “http://a.b/c=Ahoj%20!”
 - ...Component: → “http%3A%2F%2Fa.b%2Fc%3DAhoj%20!”
 - **eval()** – evaluates and executes a string like the script code
 - **isFinite()** – determines whether a value is a **finite, legal number**
 - **isNaN()** – determines whether a value is an **illegal number**
 - **Number()/String()** – converts object's value to a number/string
 - **parseFloat()** – parses a string and returns a floating point number
 - **parseInt()** – parses a string and returns an integer`

JavaScript – Events intro

- HTML DOM allows JavaScript to react to HTML events
 - JavaScript can be executed when an event occurs, e.g. when user clicks on some element or submits a form
 - Events can be assigned directly in html, e.g. `<input type="button" onclick="JavaScript code"/>` or in code accessing DOM through JS
- Some basic events
 - Global (assigned to document body, frame, ...)
 - **onload**/onunload – finished loading/leaving the page
 - onabort, onerror
 - **onresize**, onscroll
 - Form events
 - **onchange** – element content changed by user
 - **onfocus/onblur** – element obtains/loses focus
 - **onsubmit/onreset** – form is submitted/reset
 - onselect – text is selected, e.g. in textarea
 - Mouse events – **onclick/ondblclick**/onmouseover/onmouseout/...
 - Keyboard events – onkeydown/onkeypress/onkeyup

CSS Basics

Cheat sheet e.g.: <http://goo.gl/BS5CQ>

Cascading Style Sheets

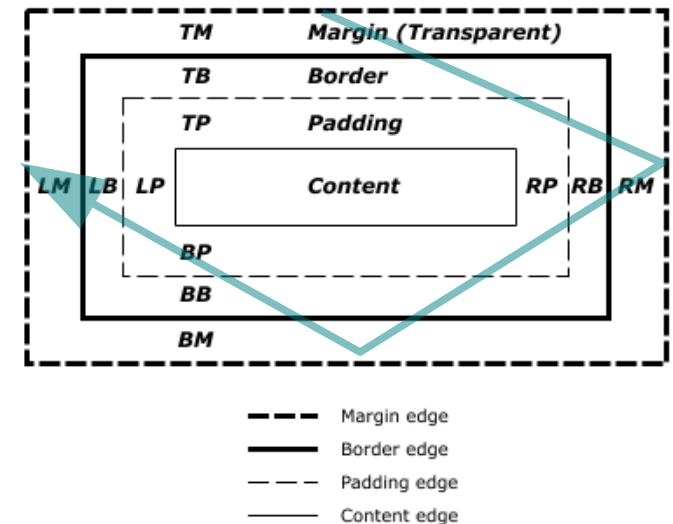
- Describes look and formatting of documents
 - Web pages, HTML application interfaces, but also elsewhere – XML, SVG, XUL, other applications (e.g. J2ME Polish styling)
- Separation of document content and presentation
 - fonts (family, size, shape), colors, layouts, alignment, ...
 - we can create the page layout without the use of HTML tables
- Conditional rendering, responsive layout
 - different rendering for display and for printer
 - **adaptation to screen size or device type**
- Changing the page presentation without adapting HTML
 - User-defined CSS may be also possible, or CSS may be skipped, e.g. because of accessibility
- Priorities (cascade) if element matches more than one rule
 - User defined→Inline→Media Type→Importance→Selector specificity→Rule order→Parent inheritance→CSS property definition in HTML document→Browser default (lowest)

CSS Basics

- CSS rule:

```
selector {  
    property: value;  
    property2: value2;  
    /* Longer comment */  
} //end of declaration block
```
- Selector
 - element **name**, **#id**, **.class**
 - attributes **el[attr]**, **el[attr=val]**
 - pseudo-selectors **:xyz**,
e.g. **:hover**, **:link**, **:before**
 - path to given element in DOM
- Properties cover large area of functions
 - Multi-value attributes: start with top and go clockwise (**t r b l**)
 - HTML color codes

```
#page1 .info > div {  
    border-width: 1px;  
    margin: 10px 2px 10px 2px;  
    border-style: solid;  
    border-radius: 15px;  
}  
  
#myheader { font-weight: bold; }  
a:hover {  
    background: #2DB200;  
    color: #FFFFFFF;  
}
```



Placing CSS in HTML document

- Directly at HTML tag using **style** attribute, e.g.

```
<h4 style="color:blue"> Example 1. </h4>
```

- Internal CSS inside **<style>** tag, e.g.

```
<style type="text/css">  
  ul { margin: 0px !important; }  
  #people-list { width:100%; overflow: hidden; }  
</style>
```

- External CSS stylesheet

```
<link href="path/file.css" rel="stylesheet"/>
```

- path/file.css contains the stylesheet definition

Note: We will not discuss CSS in great details, the jQuery Mobile uses Theme Roller for user theme definition. you may also look at CSS W3school

Basic CSS Responsive Layout

Used to adapt screen division for different devices

//Basic layout for standard screens

```
#phoneBook > .ui-block-a { width: 33%; }
```

```
#phoneBook > .ui-block-b { padding-left: 3%; width: 66%; }
```

@media all and (max-width: 35em) { //Smallest screens

```
#phoneBook > .ui-block-a, #phoneBook > .ui-block-b {  
    width: 100%; float: none;
```

```
}
```

```
}
```

@media all and (max-width: 55em) { //Bigger screens, must be down here

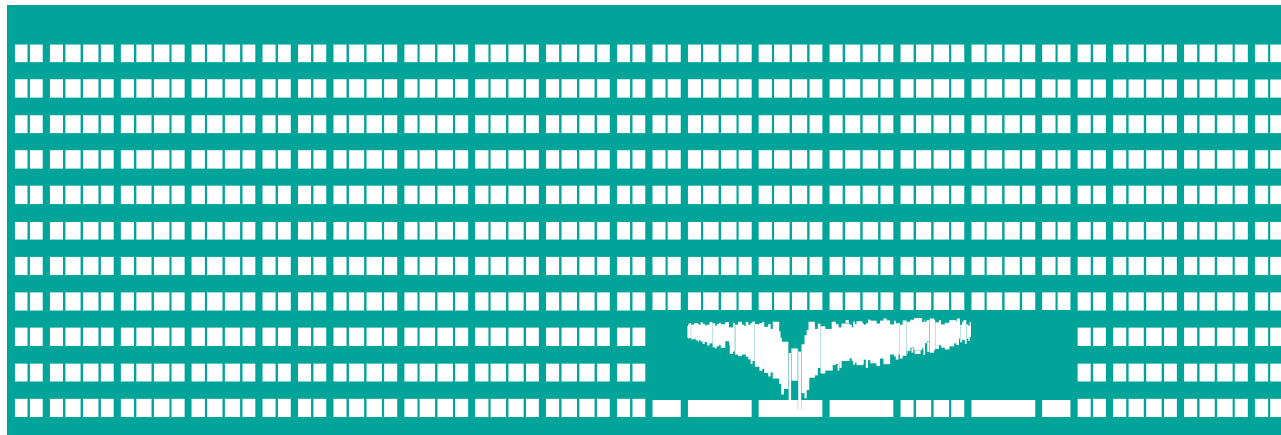
```
#phoneBookResult .ui-btn-inline {  
    white-space: normal !important; display: block;
```

```
}
```

```
}
```


JavaScript DOM/BOM

HTML5 basics – part 1



TAMZ 1
Lecture 3

JavaScript

Browser Object Model
Document Object Model

Browser O. M. – Window object (1)

Window – browser window, default, references other objs

- Obj.: document, frames, history, location, navigator, screen
- Attributes providing/setting browser window parameters
 - **closed** (closed window?), **length** (number of (i-)frames),
 - **innerHeight/innerWidth** (window content sizes)
 - **outerHeight/outerWidth** (window dimension incl. toolbars and scrollbars),
 - **screenX/screenY** (relative window position to screen), **pageXOffset/ pageYOffset** (pixel position scrolled from TL corner)
 - **name** (window name)
 - **opener** (window, which opened this one), **parent** (parent window), **self** (current window), **top** (topmost window)
- Extra error checking – write on first line of the JS file this:
"use strict";

Browser O.M. – Window object (2)

Window methods, directly accessible in code

- **alert**(msg) – alert message with OK button
- **confirm**() – confirmation message with OK+Cancel buttons
- **prompt**(text, default_val) – prompts for user input with a dialog (OK/Cancel), returns string with input or null (cancel)
- **open**()/**focus**()/**blur**()/**close**() - Open/Focus/Unfocus/Close current window, may be blocked (e.g.close) or **unavailable**
- **btoa**()/**atob**() - Encodes/Decodes a base-64 string
- **moveBy**(dx,dy)/**moveTo**(x,y)/**resizeBy**(dw,dh)/**resizeTo**(w,h)
 - change window dimensions
- **scrollBy**(dx,dy)/**scrollTo**(x,y) - scrolls content of the window
- **stop**() – stops loading of the the window
- **print**() – prints the content of this window
- **setTimeout**(fn, ms)/**clearTimeout**(id) – delayed execution
- **setInterval**(function, ms)/**clearInterval**(id) – repeated timer

Browser Object Model – Objects

Several objects accessing browser info/current location

- navigator – information about the web browser
 - appName (e.g. Mozilla), appName (e.g. Netscape), appVersion (e.g. 5.0 (additional info)), product (engine), platform (e.g. Linux x86_64), userAgent (full UA string)
 - cookieEnabled (check cookie support), onLine (is online?)
 - language (default language of the browser, e.g. **cs**)
 - javaEnabled() – check Java support in browser
- history – access browser history
 - Methods: **back()**, **forward()**, go(**+/-index**), Attribute: length
- location – get/update document's URL
 - Methods: **assign(new_URL)**, **reload()**, **replace(repl_URL)**
 - Attributes: protocol, host, port, pathname, **hostname** (server:port), origin (http://**hostname**) hash (**#xyz**), search (**?xyz**), href (protocol://host:port/pathname**hashsearch**)
- screen – screen view parameters
 - availHeight/availWidth – screen sizes (without taskbar),
 - width/height – total screen sizes, colorDepth (BPP)

The document Object

Provides access to individual parts of HTML document

- Properties for accessing individual parts
 - `document.anchors`, `.links`, `.images`, `.forms`
 - `document.cookie`, `.implementation` (object), `.applets`
- Global document properties
 - `baseURI`, `documentURI`, `URL`, `domain`, `inputEncoding`
 - `title`, `referrer` (the document which lead here), `lastModified`
 - `doctype` (object with document parameters)
 - `readyState` (uninitialized/loading/interactive/complete)
 - `document.documentElement` (the whole `<html>`), `.body`
- Methods for HTML document manipulation
 - `getElementsByName("x")` – all form elements with name `x`
 - `open(MIME, repl_hist)/close()` – output HTML stream
 - `write()`, `writeln()` - write HTML or JS code to the document
 - `importNode(n, desc)` – import node `n` from other document
 - `createDocumentFragment()` – empty fragment for DOM
 - `createElement()/createTextNode()/createComment()/createAttribute()`

DOM – Elements, whole Document

- Accessing document elements

- `var x = document.getElementById("id1");` //<x id="id1"/>
 - returns single DOMElement with given ID (if available)
- `var y=x.getElementsByTagName("p");`
 - returns an array of elements with given tag name (e.g. <p>)
 - for example, replace first item in first unordered list

```
var list=document.getElementsByTagName("ul")[0];  
list.getElementsByTagName("li")[0].innerHTML="Replaced  
I1";
```

- Document manipulation:

- `normalize()` – deletes empty text nodes, joins adjacent nodes

Note: The DOM slides are mainly for reference, we will use mainly jQuery for DOM manipulation

DOM – Element (1)

- Attributes of Element object
 - R/W access to standard set of attributes and content:
 - accessKey, className, dir, id, lang, tabIndex, style, title
 - nodeValue – value of form elements, innerHTML – inside of the element, textContent – text without formatting
 - Arrays of objects: attributes, childNodes
 - Navigation: firstChild, lastChild, nextSibling, previousSibling
 - parentNode, ownerDocument (root element)
 - namespaceURI, tagName/nodeName, nodeType (1 - element)
 - Dimensions:
 - offsetHeight/offsetWidth – element size,
 - clientHeight/ clientWidth – viewable, offsetLeft, offsetTop, offsetParent
 - scrollHeight/scrollWidth – entire size, scrollTop/scrollLeft - distance of element from top/left edge of the view

DOM – Element (2)

- Methods of Element object
 - Feature/parameters verification: `isDefaultNamespace()`
 - `hasAttribute(name)/hasAttributes()/hasChildNodes()`
 - `isEqualNode()/isSameNode()/isSupported()`
 - Attribute manipulation
 - `getAttribute(name)` – just text, `getAttributeNode()` – node
 - `setAttribute()`, `setAttributeNode()`, `setIdAttributeNode()`
 - `removeAttribute()/removeAttributeNode()`
 - User data key/element: `getUserData()/setUserData()`
 - Node manipulation:
 - `appendChild()` – adds a new lastChild, `cloneNode(deep_c)`
 - `insertBefore(new, existing)` – insert new node after existing
 - `n1.compareDocumentPosition(n2)` – position n2 from n1
 - `toString()`
 - Child node list – attribute length, method `item(x)`

DOM – Attribute

- Attributes
 - `isId` – is this attribute an ID?, `specified` – is present?
 - `name` – name of the attribute, `value` – value of the attribute
- Named node map (`element.attributes`)
 - `length` – number of attributes
 - `item(index)` – returns item with given *index*
 - `getNamedItem(name)` – returns item for given *name*
 - `removeNamedItem(name)` – removes attribute *name*
 - `setNamedItem(node)` – sets the specified attribute

```
var btn=document.getElementsByTagName("h3")[0];  
var type=document.createAttribute("class");  
type.nodeValue="myclass";  
btn.attributes.setNamedItem(type);
```

HTML5

(as a mobile platform)

Basic functions of a mobile platform

- Provide user interface
 - High-level GUI
 - Output using pre-defined well-known UI elements
 - Input of basic input types (text, passwords, selections, but also numbers, dates, ...)
 - Low-level GUI
 - Output using graphics primitives
 - Direct input from a user – touches, keypresses, mouse, ...
- Provide storage for user data
 - Configuration, persistent data
 - Data shared between different applications/screens
 - Caching of commonly used application components
- Provide means of communication
 - HTTP, sockets, local connectivity, ...

Are the features present in HTML5?

- Provide user interface
 - High-level GUI
 - Yes, we have already done that for HTML forms
 - HTML5 UI extensions (platform compatibility issues)
 - Custom UI elements (e.g. the Date pickers)
 - Low-level GUI
 - HTML5 Canvas
 - Events in HTML documents
- Provide storage for user data
 - Web DOM Storage for configuration, user data, ...
 - Different pages in a document can share data directly
 - Web Application Cache for offline use
- Provide means of communication
 - HTTP/HTTPS, AJAX, Web Sockets (some browsers), ...

HTML5

Web Storage & Offline use

Examples e.g.: http://www.w3schools.com/html/html5_webstorage.asp
http://www.w3schools.com/html/html5_app_cache.asp
<http://diveintohtml5.info/storage.html>

Specifications: <http://www.w3.org/TR/webstorage/>
<http://www.w3.org/TR/offline-webapps/>

Web Storage

- Abused nowadays for usertracking, no easy way to display stored data, but main idea was to persist data inside a session and between sessions
 - `localStorage` – data without expiration date
 - `sessionStorage` – data for current browser session (lost after browser restart)
- Everything stored as string key-value pairs in an object (must be converted to strings and back for non-string values – esp. objects and arrays)

```
// Really basic use for objects, using JSON
if (typeof(Storage)!=="undefined" && localStorage != null) {...}

localStorage.conversions = JSON.stringify(conversions);
conversions = JSON.parse(localStorage.conversions);
```

Storage interface

Storage interface

- Attributes
 - length – number of elements in storage (readonly)
- Methods
 - key(*idx*) – returns string key for *idx*
 - getItem(*key*) – returns string value (*data*) for *key*
 - setItem(*key*, *data*) – stores *data* for *key* in the storage
 - removeItem(*key*) – deletes *data* for *key* from the storage
 - clear() – clears the whole storage (removes everything)
- Storage event in DOM
 - fired on each setItem(), removeItem() and clear()
 - *storage* event (key, oldvalue, newvalue, url, storage area)
- Storage attributes in HTML document
 - localStorage, sessionStorage
- QUOTA_EXCEEDED_ERR – storing to already full storage

Web Storage examples

- Persistent counter with local storage

```
<div>Page display count in this browser: <span id="cnt">?</span></div>
<script>
  if (!localStorage.pageShows) { localStorage.pageShows = 0; }
  localStorage.pageShows = parseInt(localStorage.pageShows) + 1;
  document.getElementById('cnt').textContent = localStorage.pageShows;
</script>
```

- Removing an item from local storage

```
localStorage.removeItem('pageShows');
```

- Removing all items session storage

```
sessionStorage.clear();
```

- Accessing all items in session storage

```
for (var i = 0; i < sessionStorage.length; i++) {
  var key=sessionStorage.key(i);
  document.writeln("K: " +key+ " V: "+sessionStorage.getItem(key));
}
```

Web Storage closing remarks

- There are no methods to filter/sort/iterate the storage objects directly, but we can write our own
 - use `.length` and `.key(id)` in a *for* cycle like in the example
 - write a filter function used within the cycle
 - e.g. `myfilter(key) → true/false` (include the item?)
 - create an output array with the filtered result
 - provide a sort function which will be used to sort the array
 - e.g. `mysort(a, b) → returns: -1 (a<b), 0 (a==b), 1 (a>b)`
- Same origin policy is applied
 - Scripts having same protocol, host and port share storage
 - We can use the data from other pages on server
 - The ~5 megabyte limit is shared by these scripts!
 - Security considerations, race conditions
 - we should use prefixing, e.g. **app.record**, encrypt data
- Two basic approaches for storing all application data
 - Store everything in single key × each item has its own key

Application cache – removal in pr.

“Offline” or pre-cached pages defined in application manifest

- Inside HTML

```
<html manifest="demo.appcache">
```

- Cache manifest file MIME type (e.g. demo.appcache)

```
Content-Type: text/cache-manifest
```

- Contains CACHE MANIFEST, NETWORK:, FALLBACK: and newer SETTINGS: section

- By default SETTINGS is fast (use cached data when on-line)

- Comments in manifest

- start with # (not first line!)
- may be used to change the manifest & force reload of pages, unless they load with an error

- Cache size limit (~5MB)

```
CACHE MANIFEST
# File ver. 1.2.1
/theme.css
/logo.gif
/main.js

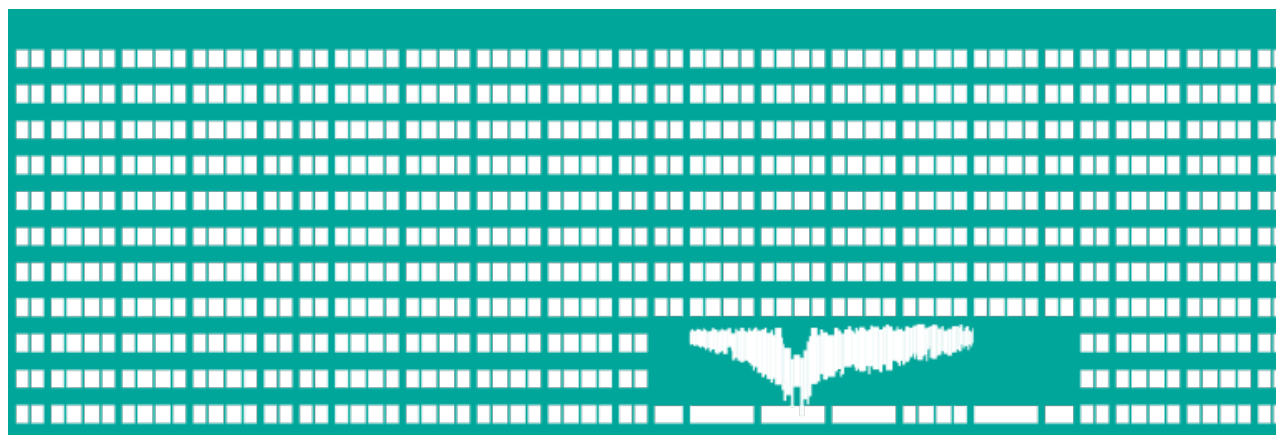
NETWORK:
* #File names or *

FALLBACK:
/html/ /offline.html

SETTINGS:
prefer-online
```

VŠB TECHNICKÁ
UNIVERZITA
OSTRAVA

VSB TECHNICAL
UNIVERSITY
OF OSTRAVA



www.vsb.cz

TAMZ I

(Design of Applications for Mobile Devices I)

**Lecture 4 (2 parts) – Selectors, ES6..8, jQuery
HTML5 basics – part 2 – UI, Onsen UI basics**

Pavel Moravec

HTML5 Selectors

- JavaScript: `document.querySelectorAll(selector)`
 - Uses CSS selector syntax, Examples:
 - * – all , a, .myclass, #id1, [attribute]
 - [attribute="val"], [attribute1="val"][attribute2] – both
 - div, span – <div> or
 - div > span – with parent <div>
 - div + span – as the first sibling after <div>
 - div ~ span – all elements sibling of <div>
 - div span – with predecessor <div>
 - Attribute matches: [x **o**= y], **o**: ^ - starts with y, \$ - ends with y,
 - * - contains y as a substring, ~ - contains word y with spaces around, ! - does not have the attribute or y as value
 - Pseudoselectors :*inputType*, :*nav*, :checked, :focus, :empty, :enabled, :disabled, :first, :last, :even, :odd, :visible, :hidden, :target, :parent, :contains(text), :has(element), :not(sel), ...

HTML5

UI extensions

Examples e.g.: http://www.w3schools.com/html/html5_form_input_types.asp
http://www.w3schools.com/html/html5_form_elements.asp

Resources: <http://html5please.com/>
<http://caniuse.com/>

Form improvements

- New elements:

- `<datalist>` element with pre-defined options for an input and user can enter his own as well
- `<keygen>` element with for public/private key pair generation
- `<output>` element for presenting results

- New form element attributes

- `autocomplete`, `novalidate`

```
<input list="browsers">  
<datalist id="browsers">  
  <option value="MSIE">  
  ...  
  <option value="Opera">  
</datalist>
```

```
<form action="kg.asp" method="get">  
  UID: <input type="text" name="uid">  
  Encryption level: <keygen name="sec">  
  <input type="submit">  
</form>  
  
<output name="z" for="x y"></output>
```


Input type improvements

New input types:

- color
- date, ~~datetime (UTC)~~, time
- datetime-local (no timezone)
- month, week
- email
- number, range
- search
- tel, url

New input attributes:

- autocomplete, autofocus
- form, formtarget
- formaction, formenctype, formmethod, formnovalidate
- height & width (only for image)
- list (datalist ID), placeholder
- min, max & step (where applicable), pattern (regexp)
- multiple (email, file), required

Syntactically “wrong” example: `<input type="number" name="ex" step="2" min="1" max="10" value="3" form="form1" pattern="[0-9]+">`

Graceful degradation

- Fallback in case that the browser does not support requested feature
 - All new HTML input types degrade to **text**
 - Some elements provide fallback content
 - Unknown tags (without fallback)/attributes are ignored
- Example – <datalist> element workaround

```
<label for="source">How did you learn about VSB-TUO?</label>  
<datalist id="sources"><select name="source">  
  <option>please choose...</option>  
  <option value="advert">Advertisement</option>  
  <option value="internet">Internet</option>  
  <option>Other</option>  
</select>If other, please specify:  
</datalist>  
<input id="source" name="source" list="sources">
```

ECMAScript 2015+ Extensions – ES6..ES8 (Android 5.0+, iOS 10.3+)

Cheat sheets e.g.: <http://www.developer-cheatsheets.com/es6>
<https://devhints.io/es6>

W3Schools: https://www.w3schools.com/js/js_es6.asp

New keywords

- **let** instead of **var** – variable valid only inside of a block (**var** could have been used after the block ended as well)
- **const** – it is now possible to declare constant value, which must be assigned when declared and cannot be changed
- **for** (variable **of** iterable) – iteration over the iterable (or string), an example will be provided later
- Support for classes (finally) – but note that class unlike an prototype is not an object, but a template (for convenience)
 - **class** MyClass **extends** OtherClass {
 constructor(myAttr) {
 this.attr = myAttr;
 }
 static factoryMethod(param) { ... }
}
- For loading/exporting code we can use **import/export**
 - **import** Class **from** 'name', **export default** function mf() {...}

New operators and numeric values

- It is possible to assign default values to function arguments
function hello(message = "Hello world!")
- Binary and octal numbers: 0b11010, 0o1737
- Multiline strings and templated strings can be used:
 - var ml=`This is a
multiline string`
 - const name="John Doe", hello=`Hello \${name}`
- Exponent operator exists (instead of Math.pow(x,y))
var two = 4 ** 0.5; //result is 2 (square root of 4)
- Arrow function operator (very pleasant shorthand)
instead of function f(x,y) {...} you may write (x,y) => {...}
 - It can also contain implicit return: let fn = n => n * 2
- Spread/rest operator ...
 - function print(x, ...extra_args), also print('a', ...[1,2,3])
 - let a = [...b, ...c, '123'], o2 = { ...o1, name : 'Obj2' }

New code constructs

- It is possible to destructure arrays and objects (like Python)
 - `let o = {a: 15, b: "ABC"};`
`let {a, b} = o` // variable a will be 15, b string "ABC"
 - Deconstruction also works on nested values
`let o2 = {a: 10, b: { h: "Hello", w: "world" } };`
`let { a, b: {h, w} } = o2` //h will contain "Hello", w "world"
 - `let [x, y=2, z=3] = [10, 9]` //will result in to separate vars:
x set to 10, y to 9 and z to 3
 - `function f({ left: x, top: y }) {console.log(`x: ${x}, y: ${y}`)}`
`f({left: 10, top: 55})` // i.e. we can destructure f. args
- Object with attributes set to values of same variables can be recorded without key-value pairs, e.g. `o3 = {a, b}` //→ `o`
- Object has extra operations: `assign(to, src1, src2 [, src3])`
 - `Object.entries(o)` – returns an array of `[key, val]` pairs
- Object now can have a method written directly:
 - `let o4 = { a: 'attribute', b() {console.log(this.a)}};`

New objects and attributes

- Number has following extra attributes and methods:
 - EPSILON, MIN_SAFE_INTEGER, MAX_SAFE_INTEGER
 - Number.isInteger(), Number.isSafeInteger()
- String has extra operations:
 - includes, startsWith – true/false result
 - repeat – copy string specified number of times
- isFinite and isNaN methods are official in ES6+
- Array has find and findIndex methods
- Extra Symbol primitive type has been defined for unique symbols with internal hidden value for given argument
 - let id = Symbol('id'); // Can be compared, but not printed
- It is possible to declare getters and setters of objects:
 - const Obj = { // Use with Obj.enabled
 - get enabled () { return this.status === 'e' },
 - set enabled (value) { this.status = value ? 'e' : 'd' }

Working asynchronously, iterating

- Some function return a Promise object (promise), code is executed asynchronously. If we want to synchronously react on such code, we have to use:
 - Call following functions on the returned promise object:
 - `.then((result) => { ... })` // Executed after completion
 - `.catch((error) => { ... })` // Error inside of async code
 - `.finally(() => { ... })` // Only ES8+, both success&error
 - Use `await/async` (ES 2018, ie. ES8)
 - `async` function `f () { ... }` // Returns a promise
 - `let v = await promise;` // Gets a result from promise
- Generators can be used to provide multiple values with `yield` instead of `return`
 - `function* idMaker () { let id=0; while (true) {yield id++ } }`
 - `let generator = idMaker()`
 - `console.log(gen.next().value)` // Loop: for (let val of gen)

ES9 (2018) – ES13 (2022) mention

- Limited support for iOS for some features on iOS before 16.4 (mostly for iOS 12.2+ for ES9, 14.5+ for ES10, 15.4 for ES11-12) but some, such as static class initialization blocks, are missing even in iOS 16.4)
- Generally supported from ES9...ES11 for iOS 12+ – iPhone 5s+:
 - support of rest (...) operator in arrays and objects
 - asynchronous generators (async function*) with await yield, and iterators
 - .description for unaccessible Symbols
 - .flat(level) for making flatter arrays to given level (removing extra [[]])
 - .trimStart() for Strings – like trim, but only in the beginning
 - possibility to dynamically call import as function and create a Promise
 - globalThis object referencing to “this” in global context
 - catch without variable to store exception (catch all)
- Generally supported in ES11..ES13 for iOS15+ – iPhone 6s-7:
 - replaceAll(what, with) on Strings replacing all occurrences of substring
 - &&=, ||=, any/allSettled for an array of Promise(s), ?. (undefined if non-existent field), ?? instead of using || for just null/undefined (not false/0)
 - Weak references, private and static class fields, matchAll() for Strings → [...]
 - Separators in large numbers (_,), e.g. 100_000_000, BigInt for huge integers

JavaScript (DOM) Events

(without jQuery)

Resources e.g.: https://www.w3schools.com/js/js_htmlDOM_eventlistener.asp
https://www.w3schools.com/js/js_htmlDOM_events.asp
<https://developer.mozilla.org/en-US/docs/Web/Events>
MDN reference

Event registration and processing

The JavaScript is event-driven, a lot of code is executed as a reaction to some external event (user interaction, received data, etc.)

- We have several categories of events, many of them associated with Document Object Model (DOM)
 - Forms and inputs: `focus/blur` – `enter/leave` the entry, `change` – entry changed, `input` – character was entered in input, `select` – text selected in textarea, `submit` – form submitted
 - Mouse and touch screen `click`
 - Keyboard: `keydown`, `keypress`, `keyup` and Mouse: `dblclick`, `mousedown`, `mouseup`, `mouseover`, `mouseout`, `mouseenter`, `mouseleave`, `hover`, `mousemove`
- They can be registered directly in HTML, e.g. `<button onclick=...>`
- They can be assigned by modifying the DOM element and calling them from JS code (function `click_fn(ev) {...}`; `my_func = ev => ...`)
 - `myElement.onclick=my_func`; `myElement.click()`
- Any event type can be registered/deregistered on DOM Element by
 - `myElement.addEventListener("click", click_fn);`
 - `myElement.removeEventListener("click", click_fn);`

Event object

The actual event object passed to the event handler function has following attributes/methods (and much more, not listed here)

- Attributes:

- **target** – who initiated the event, **currentTarget** – where the event "bubbled" if **bubbles** is true or went down, **relatedTarget** – other DOM element related to the event (e.g. some mouse events)
- **type** – which of the events was used (e.g. **click**), **timeStamp**
- **isTrusted** – is it from user?, **which** – key (unicode key code) or button relevant to the event (e.g. 1 for left mouse button)
- **pageX**, **pageY** – mouse event position relative to TL corner of the document, **offsetX/Y**, **clientX/Y**

- Methods:

- **preventDefault()** – prevent triggering of default action associated with the event (e.g. form submission)
 - **defaultPrevented** – has preventDefault been used?
- **stopPropagation()** – parents/children will not receive the event
 - **stopImmediatePropagation()** – also skips rest of the listeners
 - **bubbles** – does it continue to parent nodes?

jQuery basics (not really necessary anymore, convenience)

Cheat sheet e.g.: <http://oscarotero.com/jquery/>

Selector reference: http://www.w3schools.com/jquery/jquery_ref_selectors.asp

Why are we discussing jQuery?

- Lightweight library to simplify writing code in JavaScript and unify some concepts which require more complex use in standard JavaScript
- Works in all current major browsers
- Features of jQuery
 - Event handling
 - HTML/DOM manipulation
 - CSS manipulation
 - AJAX
 - Effects and animations
 - Delayed execution
 - Base for the jQuery Mobile framework
 - ...

Basic Syntax of jQuery

- `$(selector).action(arguments)`
 - `$` – indicates the jQuery framework use
 - `(selector)` – a query indicating one or more HTML elements
 - Can encapsulate standard objects to jQuery, examples:
`$(document)`, `$(this)`, `$(window)`
 - Can use standard CSS Selectors, we did mention in the beginning of this lecture, including attribute matches and pseudoselectors
- `action` – what should be done with the element(s)

Actions for Accessing HTML DOM

- Following actions can be used to get/set parts of the DOM:
 - Element content:
 - `html([cont])` – get 1st element's inner HTML or set it to `cont`
 - `text([cont])` – get combined text content or set it to `cont`
 - `attr(name, [cont])` – get 1st attribute value or set it to `cont`
 - Properties and data and form element values:
 - `val([cont])` – get form item value(s) or set it (them) to `cont`
 - `prop(name, [cont])` – get the first property or set it to `cont`
 - `data(name[, cont])` – get element's data-`name` attribute or set its `cont`. Checking for data presence: `hasData(name)`
 - Instead of `cont`, a `function(index, currentcont)` can be used.
 - Use `removeXXX(name)` with `Data`, `Val` and `Prop` to erase it.
- Following actions can be used to get info from DOM:
 - `index([e2])` – index of the 1st match in its siblings (*or to e2*)
 - `get([idx])` – get the DOM element from selector (*@ index idx*)
 - `each(function(idx, element))` – run function for each element
 - `toArray()` – get array of DOM elements for the selector

Actions for Accessing CSS/position

- Following actions can be used to access UI information
 - Dimensions (similar to JavaScript DOM):
 - `height()/width()` – element dimensions
 - `innerHeight()/innerWidth()` – element dimensions + padding
 - `outerHeight()/outerWidth()` – as above + margin + border
 - Offset against TL window corner or other parts of GUI
 - `offset([coord])` – get or set the coordinates of elements relatively to the document
 - `offsetParent()` – offset of the closest positioned **ancestor**
 - `position()` – coordinates of the 1st element rel. to its parent
 - `scrollLeft()/scrollTop()` – scrollbar position for 1st element
- Properties and data of CSS
 - `css(name[,cont])` – get CSS style property for the 1st element/set it to `cont`.
 - Work with CSS classes: `addClass(cl)`, `removeClass(cl)`, `hasClass(cl)`, `toggleClass(cl)` – disable ↔ enable class
 - Instead of `cont`, a `function(index,currentcont)` can be used.

Actions for working with Elements

- Cloning/*copying* element(s) (deep copy by default)
 - `clone([withDataAndEvents ] [, deepWithDataAndEvents ])`
- *Inserting* content
 - *Around element*: `wrap()` – around each, `wrapAll()` – around all elements (together), `wrapInner()` – around content
 - To the *end* of the element(s): `append(cont)` – append `content`, `sel1.appendTo(sel2)` – append `sel1` to `sel2`
 - To the *beginning* of the element(s): `prepend()/prependTo()`
 - Directly outside the element: `after()/before()` – content, `insertAfter()/insertBefore()` – supplied elements (see `appendTo()`)
- *Removing elements* from DOM: `detach()` – remove from DOM but keep for future use, `empty()` – remove child nodes, `remove()` – fully remove, `unwrap()` – remove parent element (opposite of `wrap`)
- *Replacing*: `replaceWith(cont)` – replace all elements with `cont`, `new.replaceAll(old)` – replace all matches of `old` with `new`
- *Displaying* the content: `show()`, `hide()`, `toggle()`

Working with document structure

- Following *filters* are used to make the selector more precise. We may use them directly in selector (*:nav* earlier)
 - `eq(index)` → element on given index, `slice(start[,end ])` → subset with index range, `is(sel2)` → true/false, `not(sel2)` → remove those matching `sel2`, `has(descendant)` → filtered set, `filter(sel2)` → filtered set, `map(function(index, Element))` → pass each through function, `first()`, `last()`
- *Traversal*: `each(function(index, Element))`, `add(elements)`, `end()` – terminate previous traversal, `addBack()` – add previous set to current set, `contents()` – return children of element(s)
- *Tree traversal*: `closest()` – first ancestor matching for each element, `find()` – first descendant matching, `children()`, `siblings()`
 - `parent()`, `parents()` – all element's ancestors, `parentsUntil(match)`
 - `next()` – first following sibling, `nextAll()` – all following siblings, `nextUntil()` – following siblings before condition is met
 - `prev()`, `prevAll()`, `prevUntil()` – like `next*`, but for preceding siblings

Event processing

- You can register a handler or trigger following events
 - Browser Events: `resize()/scroll()` – GUI events
 - Document Loading: `ready()` – execute code once document is fully loaded, `holdReady()` – delay `ready()` execution for now, `load()` – element fully loaded, `unload()` – page unloaded, left it
 - we will use different events in jQM, reflecting its lifecycle
- General event handler (de)registration
 - `bind()/unbind()` – assign or remove event(s) at given element(s)
 - `on(events [,selector][, data ], handler(eventObject))` – register event handler executing each time the event occurs
 - e.g. `$( "button" ).on( "click", notify );`
 - `one(events [,selector][, data ], handler(eventObject))` – execute the handler only once and de-register it after the use.
 - e.g. `$( "body" ).one( "click", "#foo", bar);`
 - `off(events [,selector][,handler(eventObject)])` – deregister handler
 - `trigger(event [,extraParameters ])` – trigger the event manually
 - `triggerHandler()` – like `trigger()`, but directly on the matched 1st element, default action is not performed.

Event shortcuts

Several shortcuts exist for registering/triggering most common events (instead of `on/trigger("xyz", ...)`, we use `xyz()`)

- Forms
 - `focus()/blur()` – enter/leave the entry, `focusin()/focusout()` – works for parent elements as well
 - `change()` – entry changed
 - `select()` – text selected
 - `submit()` – form submitted
- Keyboard: `keydown()`, `keypress()`, `keyup()`
- Mouse
 - `click()`, `dblclick()`
 - `mousedown()`, `mouseup()`
 - `mouseover()`, `mouseout()`
 - `mouseenter()`, `mouseleave()`, `hover(hndl_ent, hndl_leave)`
 - `mousemove()`

Event object

The actual event object has following attributes/methods

- Attributes:

- **target** – who initiated the event, **currentTarget** – where the event "bubbled", **relatedTarget** – other DOM element related to the event (e.g. some mouse events)
- **type** (e.g. click), **timeStamp**, **data**, **namespace**
 - `$( "p" ).trigger( "test.something" );` → something
- **which** – key or button relevant to the event
- **result** – last returned result of event handler for the event
- **pageX**, **pageY** – mouse event position relative to TL corner of the document

- Methods:

- **preventDefault()** – prevent triggering of default action associated with the event (e.g. form submission)
 - **isDefaultPrevented()** – has preventDefault been used?
- **stopPropagation()** – ancestors will not receive the event,
 - **stopImmediatePropagation()** – also skips rest of handlers
 - **isPropagationStopped()/isImmediatePropagationStopped()**

jQuery core functions

Shorthand used: array *a*, object *o*, collection (*a* or *o*)

- `$.each(collection, function(index, val))` – process all items
- `$.extend([deep,] target_o, o1 [, ... , on])` – merge objects
- `$.merge(x,y)` – merges two arrays into returned value
- `$.inArray(val, a [, fromIdx])` – element index in *a*, or -1
- `$.grep(a, funct(elem, index) [, invert_r])` – filters *a* with funct
- `$.unique(a)` – sorts arr. of DOM elements, drops duplicates
- `$.now()` – returns current time (number)
- `$.parseHTML(data [,context] [,keepScripts])` – takes string with data, converts to DOM nodes in given context
- `$.parseJSON()` – converts JSON string to JS object
- `$.parseXML()` - converts string to XML document
- `$.proxy(f, context)` – returns new function for *f* with fixed ctx
- JS equivalents: `$.trim(string)`, `$.type(o)` – extended typeof
- `$.isArray()`, `$.isEmptyObject()`, `$.isFunction()`, `$.isNumeric()`,
`$.isPlainObject()` – created: `new/{} , $.isWindow(), $.isXMLDoc()`

jQuery closing remarks

- Actions on selectors may be chained, e.g.
 - e.g. `$("#menu li").css("border","2px solid silver").filter(':odd').css("background-color","yellow").end()`
- It is a good idea to distinguish jQuery variables from standard JavaScript ones, typically prefixing them with `$`
 - e.g. `var $red = $(".redItems");`
- We have skipped the Callbacks, Deferred execution and AJAX parts of jQuery for now, they will be discussed in a separate lecture.

Onsen UI

Examples e.g.: <https://onsen.io/v2/api/js/> <https://onsen.io/v2/api/css.html>

Onsen UI – GUI framework for many libraries

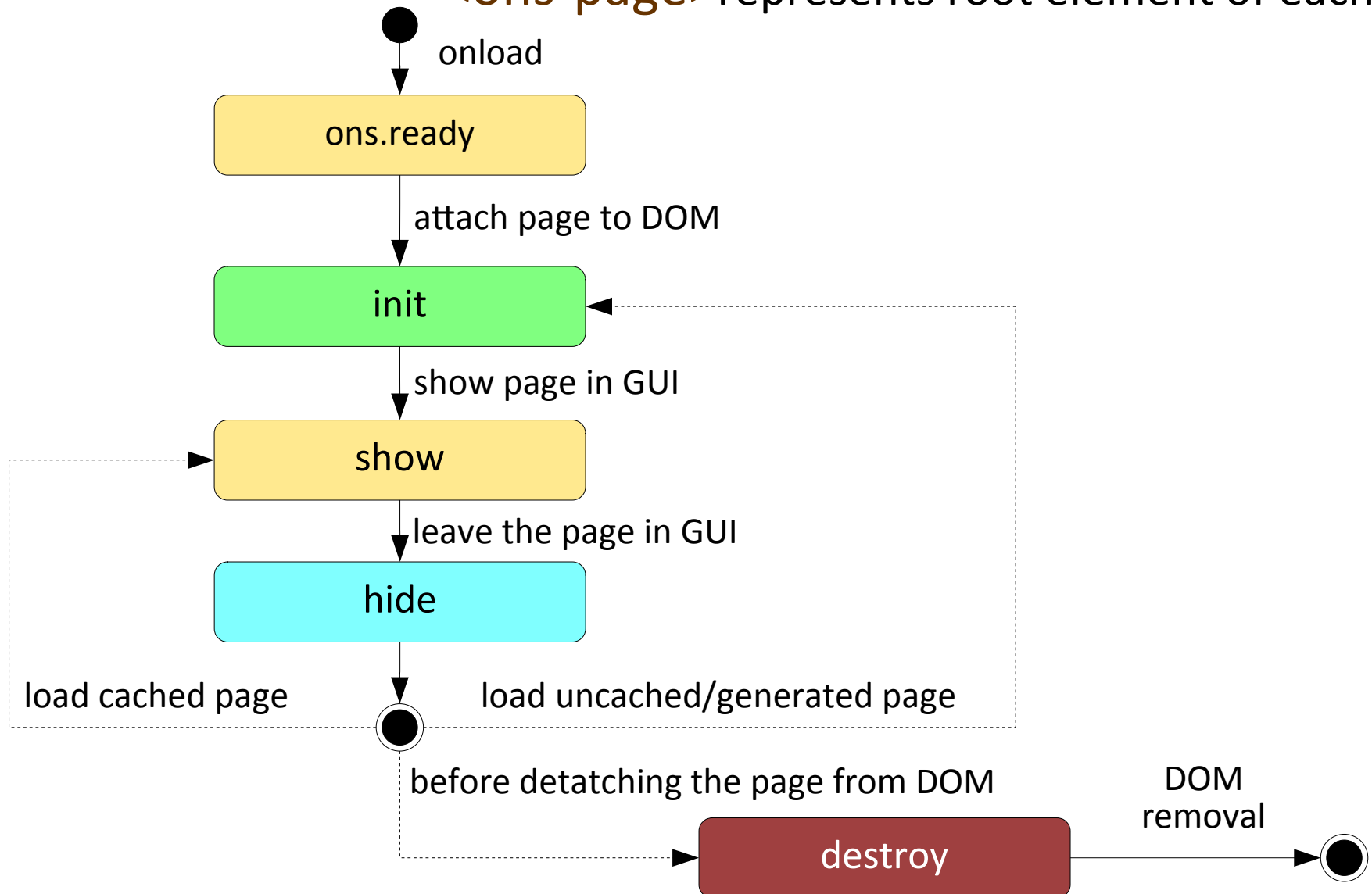
- Works (to a degree) on almost every platform with web browser (including desktop PCs)
- Meshes well with existing frameworks, and provides their integration and extra features for them
 - Angular 1 & 2+
 - React
 - Vue.js
- But it is possible to use it in
 - Pure JavaScript
 - With JQuery
 - With just CSS styles older approach, not adding any custom elements into HTML

OnsenUI properties

- Web apps with look & feel as close to native as possible
 - Possibility to build hybrid applications
 - iOS support
 - Android Material design support
 - Platform is automatically detected, conditional sections for given platform or orientation may be included
- Markup driven
- Minimal code required to get up & running
- Animations optimized for mobile environment to be smooth
- Easy to learn according to its authors
- Multiple pages in a single HTML file are possible

Onsen UI page life cycle

`<ons-page>` represents root element of each page



Gesture detector for mobile events

Besides regular events, a set of specific events is created, using on-gesture-detector

- Drag gestures
 - drag
 - dragleft, dragright, dragup, dragdown
- Hold gestures: hold, release
- Swipe gestures
 - swipe
 - swipeleft, swiperight, swipeup, swipedown
- Tap gestures: tap, doubletap
- Pinch gestures
 - pinch
 - pinchin, pinchout
- Other: touch, transform, rotate

GUI remarks

- Regular GUI Form elements are available through their Onsen UI skinned variants
 - Text Inputs, Search boxes, Sliders (& Switches), Control Groups (Checkboxes, Radiobuttons), Selects (i.e. popups), Collapsibles (Menus), Buttons
 - Grouping of form elements where suitable is possible
 - Submitting
- Besides dialogs, we can also display toasts, etc.
- You can't pass parameters between internal pages (but you can use plugins which implement this feature or share data with Storage, global variables, ...)
- All elements can be themed through pre-defined themes, CSS3 (manually or generated through a ThemeRoller)

Onsen UI – extra GUI

- Besides the basic HTML form elements, already covered in the second lecture OnsenUI offers several other features:
 - Carousels
 - Conditionals
 - Dialogs (with action sheets), popovers and toasts
 - Control elements: Pull hooks, segments, speed dials
 - Grids (with columns and rows), for replacing tables
 - Lists (dynamically loaded if necessary)
 - Menus (using splitter)
 - Pages and their navigation, including toolbars and tabbars
 - Progress indication, styled notification count ([notification](#) class)
 - Page templating
 - Visual effects
 - Icon sets integration
 - Collapsed content and grouping into cards

Page transitions

- Animation on transition controlled by **animation** attribute on `ons-navigator` when changing page, opening dialog box, ...
- Transition animations
 - fade
 - pop
 - slide
 - none
- “swipe to pop” on iOS
 - `swipeable` attribute (force value for Android as well)
- Pushing pages

```
var myNavigator = document.getElementById('myNavigator');
myNavigator.pushPage('page2.html', { data: { title: 'New Page', // ...
} /*, other options */ });
// Data passed accessible through myNavigator.topPage.data
```
- **ons.ready**(*h*) handler *h* to be executed after loading library

Onsen UI default object methods

- After extracting HTML element for ons object, following methods may be used (e.g. `document.querySelector('ons')`)
 - `isReady` – true if Onsen UI is initialized; `ready(callback)`
 - `isWebView` – true if running inside Cordova
 - `setDefaultDeviceBackButtonListener(listener)` – handle HW back
 - `enable/disableDeviceBackButtonHandler` - in `ons.ready`.
 - `enable/disableAutoStatusBarFill` – status bar fill feature on iOS7+
 - `enable/disableAnimations/AutoStyling` - debugging
 - `disableIconAutoPrefix` – disable adding fa- prefix automatically to ons-icon classes, for custom icon packs.
 - `preload(templatePaths)` – preloading templates for faster GUI
 - `createElement/Popover/Dialog/AlertDialog(x, options)` – create new item from a template.
 - `openActionSheet(options)` – instant Action Sheet for user choice
 - `resolveLoadingPlaceholder(page)`
 - `getScriptPage` – last created page access from the current script scope in children of ons-page element.

Onsen UI platform object methods

- After extracting HTML element for platform object, following methods may be used (e.g. `document.querySelector('ons.platform')`)
 - `select(platform)` – chooses rendering platform, e.g. for testing
 - `isWebView` – is running in Cordova?
 - `isiPhone/isiPhoneX/isiPad` – iPhone/iPhone X*/iPad
 - `isAndroidPhone/isAndroidTablet`
 - `isIOS/isAndroid/isWP/isBlackBerry([forceActualPlatform])`
 - `isOpera/isFirefox/isSafari/isChrome/isIE/isEdge([forceAP])` – browser
 - For iOS, special checks also exist:
 - `isIOS7above/isIOSSafari`
 - `IsUIWebView` – necessary in iOS 7 and bellow
 - `IsWKWebView` – should be used in iOS 8 and above

Toolbars, Tabbars

- `<ons-toolbar>` represents a navigational component typically split into left, center and right part, placed on top of the page
 - We can place `<ons-toolbar-button>` inside of toolbar for navigation or actions on page

`<ons-toolbar>`

`<div class="right"><ons-toolbar-button>`

`<ons-icon icon="ion-navicon, material:md-menu"></ons-icon>`

`</ons-toolbar-button></div>`

`</ons-toolbar>`

- Tabbars, represented by `<ons-tabbar>` on the other hand allow to show a sub-page referenced by `<ons-tab>` directly.
 - May be placed **automatically**, on **top** or on the **bottom**

`<ons-tabbar swipeable position="auto">`

`<ons-tab page="tab1.html" label="Ex 1" icon="ion-home" badge="99+" active>`

`<ons-tab page="tab2.html" icon="md-settings" active-icon="md-face">`

`</ons-tabbar>`

Grids and grouping

- Grids use `ons-row` and `ons-col`

```
<ons-row>
```

```
<ons-col width="50px"><ons-icon icon="fa-xyz"></ons-icon></ons-col>
```

```
<ons-col>Text column</ons-col>
```

```
</ons-row>
```

- By default the columns of the grid have the same width
- `width` and `vertical-align` (top/bottom/center) attributes can be used for columns (and width for rows as well)
- Groups can be done in CSS using `segment`, `segment__item` and inside of it `segment__input`/`segment__button` classes

```
<div class="segment" style="width:200px;margin:7px 50px;">
```

```
<div class="segment__item">
```

```
<input type="radio" class="segment__input" name="sa" checked>
```

```
<div class="segment__button">One</div>
```

```
</div><div class="segment__item"> ...
```

```
</div>
```

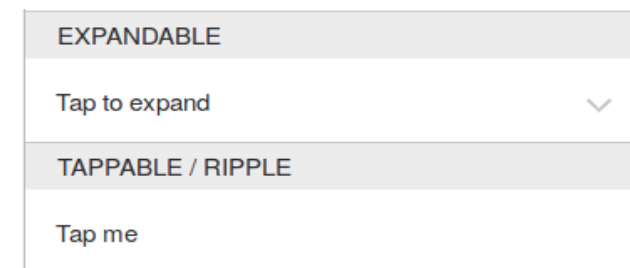
```
</div>
```

Menus, Navigation, Carousels

- Main/side menu – `<ons-splitter>` containing `<ons-splitter-side>` and `<ons-splitter-content>`
 - Opens by `menu.open()` closes by `menu.close()`, where menu is the object representing HTML splitter side element
 - `content.load(page).then(menu.close.bind(menu))` ;
/*After the page is loaded, menu object is bound to its close method to be passed as an object in argument and used – an example of a Promise */
- Navigation is done by `<ons-back-button>` or `<ons-navigator>`
 - ons-navigator was mentioned earlier, it handles page stacking
- Carousel is used to save space and display several items on the same position, switching between them by swipes or in code
 - `<ons-carousel>` is used for the carousel placement, for individual content parts, we use `<ons-carousel-item>`, each with an `index`
 - Carousel can be made to fit the whole page between toolbar and tabbar with `fullscreen` property, `direction` can be set to `horizontal` (default) or `vertical` and can be made `swipeable`
 - Navigation may be done by `carousel.prev()` and `carousel.next()`

Lists

- Use classic `<ul>` and `<ol>` tags with `class="list"`
- Read-only ones contain classic `<li>` items with `class="list-item"`
- Headers with `list-header`
- Expandable content with `list-item--expandable` + `expanded`
 - `list-item__expand-chevron` shows the expansion indicator
 - `list-item__expandable-content` is the content for expansion
- Tappable list items have class `list-item--tappable`
 - Split through `list-item__left`, `list-item__center`, `list-item__right`
 - No border around the list view: class `list--noborder`
- Advanced use with `<ons-list>`, `<ons-list-item>`, `<ons-list-header>`
 - `<ons-list-title>` – Name of the list
 - `<ons-lazy-repeat>` – “Infinite list” generated on the fly
 - `createItemContent`, `countItems`
 - `calculateItemHeight`, `destroyItem`



Conditionals, Pull Hooks, Segments, Speed dials

- Conditional parts of HTML – `<ons-if>`
 - `platform` attribute: `android`, `ios`, `windows`, `other`
 - `<ons-if platform="ios other">`
 - `orientation` attribute `"portrait"` or `"landscape"`
 - Pull to refresh – `<ons-pull-hook>`
 - `<ons-pull-hook id="pull-hook">Code to show</ons-pull-hook>`
 - `changestate` event – state of pull hook in `state` event attribute
 - `initial` → `preaction` → `action`
 - Segment with button bar auto changing `<ons-segment>`
 - `postchange` event – item index in `index` attribute
`segmentItem` contains the object reference
 - Speed dial – `<ons-speed-dial>` with `<ons-speed-dial-item>`
 - Floating action button – `<ons-fab>`
- ```
<ons-speed-dial position="bottom right" direction="up">
 <ons-fab><ons-icon icon="md-share"></ons-icon></ons-fab>
 <ons-speed-dial-item><ons-fab>...</ons-fab></ons-speed-dial-item>
</ons-speed-dial>
```

# Form items, Progress bars

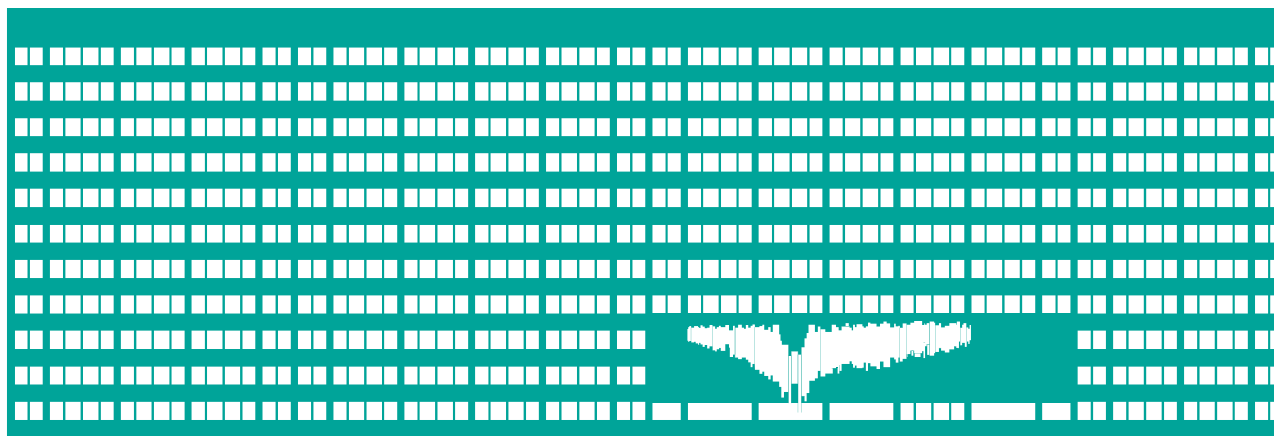
- Forms contain default form elements, mapped to Onsen UI
  - `<ons-button>` – default button, or with some modifiers
  - `<ons-checkbox>` – checkbox item, typically inside a list item
  - `<ons-radio>` – radio button, typically inside a list item
  - `<ons-input>` – representing alternative to default input
    - `<ons-search-input>` for search forms (contains search icon)
  - `<ons-range>` – range slider alternative to “range” input type
  - `<ons-select>` – replacing `<select>` element, option elements inside are the same
  - `<ons-switch>` – flipswitch instead of checkbox element (with optional checked and disabled properties)
  - `<ons-card>` is used to visually enclose content of a page/form
- Progress may be indicated by
  - `<ons-progress-circular>` with value, secondary-value and indeterminate (rotating with unknown progress), or
  - `<ons-progress-bar>` with the same attributes



# Dialogs, Popups, Toasts and Action sheets

- Dialogs are shown above active page
  - either default ons.notification objects mapping the default JS dialogs (plus an extra `ons.notification.toast`) or
  - dialogs created from a template by calling `.show()/.hide()` on their object retrieved by `ons.createDialog(template, { append: true })` – append set to true attaches them to DOM
    - `<ons-dialog>` is a default dialog box with any content
    - `<ons-modal>` preventing a user from hiding it
    - `<ons-alert-dialog>` with a title, content and button bar
- Popup is done by `<ons-popover>` with the `.show()/.hide()` methods `ons.createPopover(template)` and displays a popup text above the target
- Toast message is displayed with `<ons-toast>` element or as above, with `.show()/.hide()/.toggle()`
- Other content such as selection can be displayed in a (`cancellable`) `ons-action-sheet`

# Network communication



**TAMZ 1**  
**Lecture 5**

# Uniform Resource Identifier (URI)

The basic form of Uniform Resource Identifier as follows:

`<scheme>:<address+path> [ ?<query> ] [ #<fragment> ]`

- **scheme** – identifies how the connection is made:
  - socket, http, file, datagram, tel, sms, comm, ...
- **address** – identifies what to connect to and a **path** to res.
- **other** – identify other information required by the scheme

**Templates of some URIs are shown in the following list:**

- `http://user:pass@server/path/file?opt1=val1&opt2=val2`
- `mailto:mor03@vsb.cz?subject=TAMZ1&body=test`
- `sms:+15105550101,+15105550102?body=hello%20CSI`
- `tel:+15105550101`
- `javascript:alert(my_message);`

*These URI can be used to pass data to external applications!*

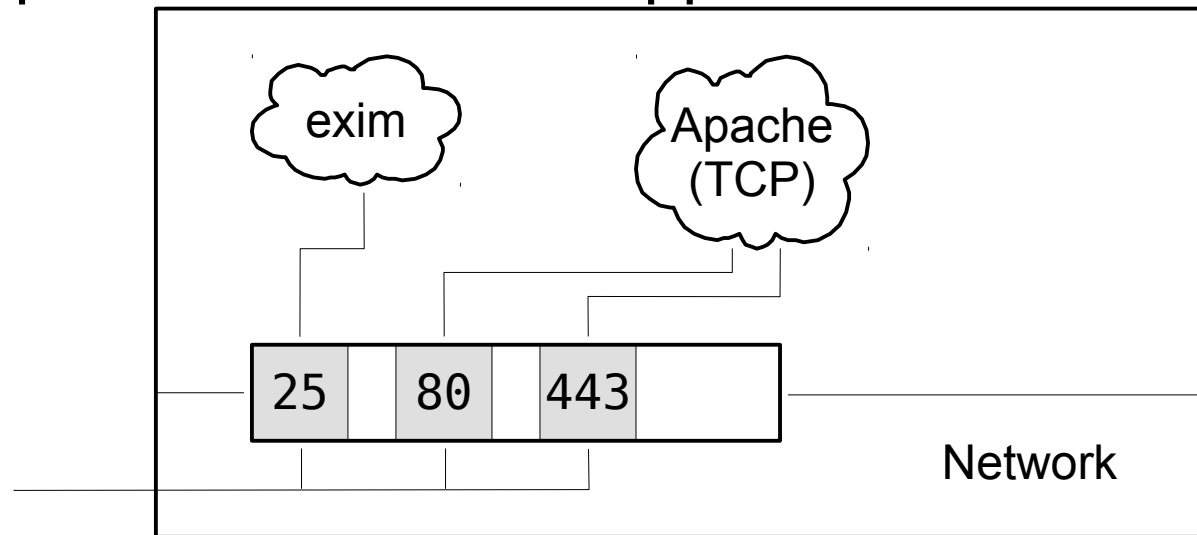
# Identifying the remote machine

- Machines are uniquely identified within the Internet by:
  - Domain Name which represents machine address by a human-readable form, for instance **www2.cs.vsb.cz**
  - The dotted quad form of IP address which is four numbers 0-255 separated by dots (IPv4), such as 158.196.109.35
    - or 8 hexadecimal numbers (0-ffff) separated by : (leading 0s dropped and :: can remove the longest block of :0[:0]), e.g. 2001:718:1001:207::35
- Transport layer protocols used in IP networks:
  - **TCP** (Transmission Control Protocol) – secure (but not encrypted) logical channel, we establish connection before the communication and acknowledge all received data sent by the peer. In the end, the connection must be closed.
  - **UDP** (User Datagram Protocol) – no connection, we send data in standalone packets. A packet may get duplicated, lost, come in later than the next packet and in some cases may be even damaged.

# Identifying an Application

An IP address identifies a unique endpoint, but several domain names may resolve to the same machine.

Each machine also contains **ports**, and when a client or a server is set up, a port for the remote application must be chosen.



The port is not a physical location in a machine, but a software abstraction. Typically, each service is associated with a unique port number on a given server machine. We need to use two ports – one on source (usually a random one) and one on destination machine. Default: **HTTP: TCP/80, HTTPS: TCP/443**

# HTTP protocol

- HTTP Request has following syntax:

<METHOD> <URL/PATH> HTTP/1.<version>

<Client header<sub>1</sub>>

...

<Client header<sub>n</sub>>

<Empty line>

<Client data, if any>

- HTTP Response has following syntax:

HTTP/1.<version> <3-digit error code> <Message>

<Server header<sub>1</sub>>

...

<Server header<sub>m</sub>>

<Empty line>

<Data from server, if any>

# Basic HTTP methods and codes

## HTTP methods:

- GET
  - Asks for the resource specified by URL
- OPTIONS
  - Asks for the support of some features (methods)
- POST
  - Sends data (web form) to the server
- PUT
  - Stores the document to a file on the server
- DELETE
  - Deletes the resource

## Error codes:

- 1xx – Informational
- 2xx – Successful
  - 200 – OK
- 3xx – Redirection
  - 302 Found
  - 304 Not Modified
- 4xx – Client Error
  - 401 Unauthorized
  - 403 Forbidden
  - 404 Not Found
  - ...
- 5xx – Server Error
  - 500 Internal Server Error
  - 502 Bad Gateway, ...

# HTTP protocol – GET method

**GET** /~mor03/TAMZ/TAMZ.php?login=mor03 **HTTP/1.1**

**Host:** homel.vsb.cz

Connection: keep-alive

**Accept:** \*/\*

**Origin:** *http://localhost*

User-Agent: Mozilla/5.0 (X11; Linux x86\_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/31.0.1650.63 Safari/537.36 SUSE/31.0.1650.63

**Referer:** http://localhost/~mor03/TAMZ1\_ex/public\_html/comm.html

Accept-Encoding: gzip,deflate,sdch

Accept-Language: cs,en;q=0.8,sk;q=0.6

**HTTP/1.1 200 OK**

Date: Sun, 09 Mar 2014 20:12:35 GMT

Server: Apache

**Access-Control-Allow-Origin:** \*

Content-Length: 32

Connection: close

**Content-Type:** text/plain

8842e9d8172c3875a8b5ed8721949632



# HTTP protocol – POST method

**POST** /~mor03/TAMZ/TAMZ.php **HTTP/1.1**

**Host:** homel.vsb.cz

Content-Length: 12

**Accept:** \*/\*

**Origin:** http://localhost

**API-Token:** eHh4MDAxOnh4eA==

User-Agent: Mozilla/5.0 (X11; Linux x86\_64) SUSE/31.0.1650.63

Content-Type: application/x-www-form-urlencoded; charset=UTF-8

**Referer:** http://localhost/~mor03/TAMZ1\_ex/public\_html/http\_test.html

Accept-Language: cs,en;q=0.8,sk;q=0.6

login=xxx001

**HTTP/1.1 401 Unauthorized**

Date: Mon, 10 Mar 2014 09:10:24 GMT

Server: Apache

**Access-Control-Allow-Headers: API-Token**

**Access-Control-Allow-Origin: \***

Content-Length: 43

Connection: close

Content-Type: text/plain

Sorry, the supplied secret code is invalid

# Referencing external resources in HTML/JS

# How to load external web resource?

- In HTML code – done from the beginning
  - Tags: link, a, img, script, form, ...
  - Attributes: href, src, action (in such tags)
  - External pages: frame, iframe, new window
- Inside Java applet, Silverlight, Flash, ...
- For some years we can load data with JavaScript – AJAX
  - XMLHttpRequest object
  - Asynchronously with jQuery
    - Shorthand methods \$.get, \$.post for basic use
    - Low-level interface \$.ajax
- Following data formats are typically supported
  - **HTML** (text/html), plain **text** (text/plain)
  - **XML** – any document in eXtensible Markup Language
  - **JSON, JSONP, JavaScript**

# What is not supported in browsers?

- Socket-based “raw” communication, i.e.
  - We can't send ICMP messages
  - We can't build a raw TCP stream and send data bidirectionally in a stream
  - We can't send UDP datagrams
- Accessing **any** other site's resources from our page through AJAX in JavaScript (e.g. we are in a sandbox)
  - By default, we are limited to same-origin policy
    - Cross-Origin Resource Sharing (CORS) is a workaround
  - We may be able to manipulate DOM into loading JSON object manually (JSONP)
- Using chunked HTTP/1.1 transfers in AJAX
  - Because of its asynchronous nature, we wait until the data is returned as the result.

# Cookies in JS (obsoleted by EU?)

Cookies can be set only in the **same domain** (same origin)!

- Through `document.cookie`
  - By default a string containing all cookies separated by ;
    - `the_cookie=val; cookie2=value; xyz=abc;`
  - Set cookie: `document.cookie="the_cookie=val; expires=2014-12-31T23:59Z; path=/"`;
  - Remove: `document.cookie="the_cookie=; expires=Thu, 01 Jan 1970 00:00:00 GMT; path=/"`;
- Using jQuery cookie plugin
  - Available at <https://github.com/carhartl/jquery-cookie>
    - Usage: `<script src="/path/to/jquery.cookie.js"></script>`
  - Set: `$.cookie('the_cookie', 'the_val' [, { expires: 7, path: '/' }]);`
  - Get: `$.cookie('the_cookie');` *//undefined* if not present
    - `$.cookie()` – all available cookies
  - Remove: `$.removeCookie('the_cookie');`

# JavaScript Object Notation

Man site: <http://json.org>

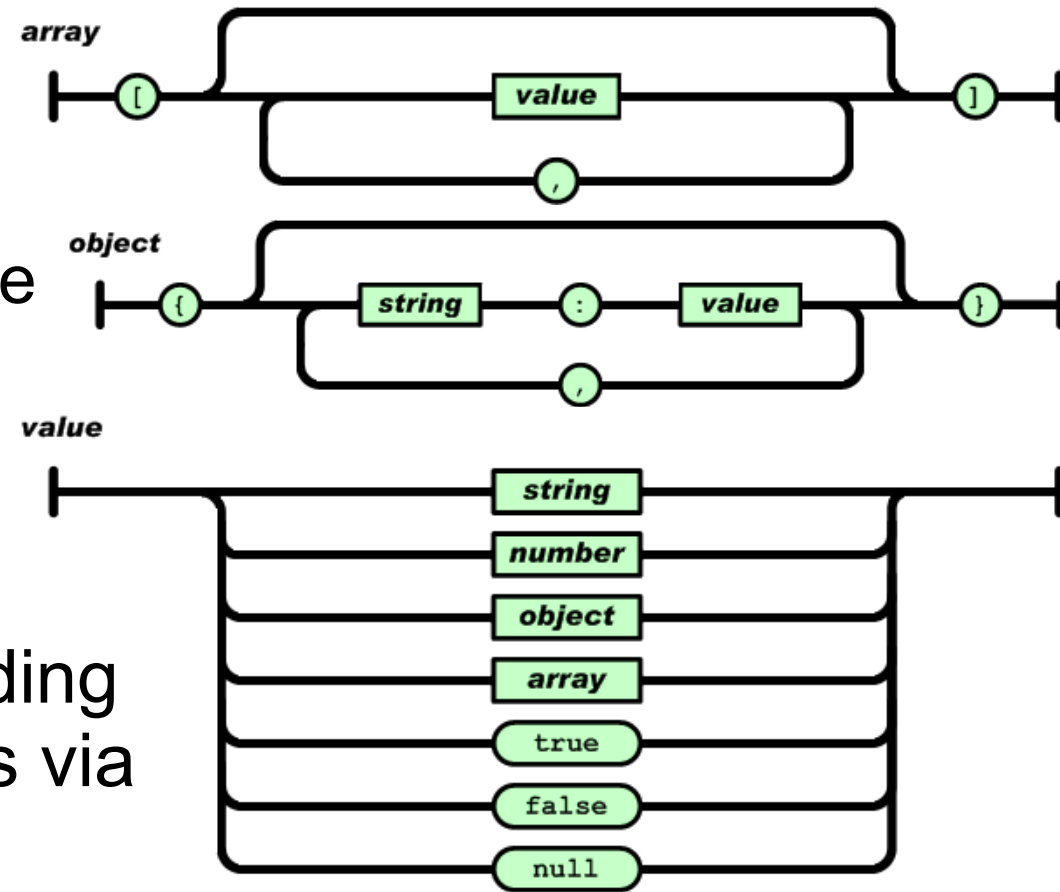
See e.g.: <http://w3schools.com/json/>

# JavaScript Object Notation (JSON)

- Similar to XML
  - In plain text, human-readable (to a degree)
  - Can be parsed directly in JavaScript (at least by eval) and transported using AJAX
  - May contain a hierarchy of arrays, objects and basic types
- Differences against XML
  - Worse orientation in structure of JSON data
  - No end tags (just parentheses), faster (de)serialization
  - No reserved words (but has special values true, false, null)
  - Uses C/Java syntax for writing strings (\ as escape character) and numbers. & < > are normal characters
- Faster than XML – no XML DOM construction necessary, just an immediate parsing of JSON data
- Requires less memory for storage/processing than XML

# JSON Syntax

- Object is stored as a string using JavaScript syntax
  - eval can be used, insecure  
`eval('(' + myJSONString + ')');`
- Javascript-enabled browsers following ECMAScript standard (2009) offer a way of encoding and decoding these objects via **JSON** object
  - JSON.parse**(myString[, rev]);
    - reviver function as 2<sup>nd</sup> arg.
  - JSON.stringify**(myObject[, rep]);
    - replacer function as 2<sup>nd</sup> arg.



## Example:

```
[{ "firstName": "Pavel" , "surname":
 "Moravec", "inRel" : false } ,
 { "firstName": "Michal" , "surname":
 "Krumnikl", "inRel": true } , null]
```



# AJAX: Asynchronous JavaScript and XML

See e.g.: <http://www.w3schools.com/ajax/>  
jQuery: <http://api.jquery.com/category/ajax/>

# Asynchronous JS and XML (AJAX)

- Updates existing page without reloading it by exchanging data with the web server, uses existing technologies
- Browser and platform independent
- Uses HTML, JS (DOM), CSS, data format (XML, JSON), XMLHttpRequest for sending the request
  - `var req = new XMLHttpRequest();` //modern browsers, IE7+
  - `req.open(method,url,async);` //Method: "GET"/"POST"
    - `async`: true/false – asynchronous request
  - Async, before open: `req.onreadystatechange=function() { if (xmlhttp.readyState==4 && xmlhttp.status==200) {...} }`
  - `req.setRequestHeader(header,value);` //User headers
  - `req.send([pdata])` //For POST, pdata is a string of user data
  - Access data: `req.responseText` or `req.responseXML`
- AJAX via POST – prevents caching, may submit more data, more robust for user input with strange characters

# AJAX Remarks

- Asynchronous requests should be used every time
  - We may do other things whilst waiting until AJAX call is completed (e.g. execute the rest of the script)
  - The application will be more responsive to the user
  - Once the data is ready, we may process it
- When dealing with `onreadystatechange` event, we have:
  - `readyState` – in which stage of AJAX call are we?
    - 0 – request not initialized (open has not been called yet)
    - 1 – server connection opened (send has not been called)
    - 2 – request received (send has been called, headers and status has been received, but not the data)
    - 3 – loading response (only partial data available)
    - 4 – done, request finished, complete response is ready
  - `status` – HTTP status code (200 – OK, 404 – not found, ...)
- We can call `req.abort()` before `readyState` reaches 4

# Problem with JavaScript AJAX

- Same-origin approach is applied when accessing remote resource via HTTP from JavaScript. If the origin does not match, the browser drops the result
  - Either the same web, or the access for different origin must be explicitly permitted in headers via Cross-origin resource sharing (supported in recent browsers)
    - JSONP may also be a solution (loading script into DOM instead by Javascript), but may be a security hole
    - Apache Cordova: `<access origin="*/URL" />` in [config.xml](#)
- Cookies may be set/retrieved only within the visited domain, not for a remote domain (e.g. alice.uk vs. bob.cz)
- Custom headers may be blocked by the server
  - AJAX checks with OPTIONS when sending POST first
- Images, pages, ... from external site may be loaded through standard HTML href and src, without AJAX by constructing the elements directly

# XmlHttpRequest advanced features

Working for extended AJAX functionality (iOS 8+, A. 4.4.4+)

- Support for binary, Blob and form data submission
  - `send(ArrayBufferView data); send(Blob data);`
  - `void send(Document data); void send(FormData data);`
- Changing the response data type to access received data in different manner through `req.response`
  - `req.responseType` may be set to
    - "text" (default) – v1 approach (also if type is set to "")
    - "arraybuffer" – binary data, "blob" – Blob instance ("file")
    - "document" – DOM document, "json" – JSON data
- Support for `timeout` (event), upload progress information
  - register `progress` event on `req.upload`
- **Cross-origin resource Sharing (CORS)**
- Both CORS and XMLHttpRequest2 work in all major mobile platforms' default browsers (some CORS bugs in IE10)

# Cross-origin resource sharing (CORS)

- When using AJAX, client sends *Origin request* header,
  - e.g. **Origin: http://www.vsb.cz**
  - When needed, *pre-flight* OPTIONS request is also done, with **Access-Control-Request-Method** & **Access-Control-Request-Headers** request headers to verify CORS access
- CORS defines set of *response* headers which allow AJAX from remote origin which the external server has to send:
  - **Access-Control-Allow-Origin** – for cross-origin access
    - e.g. **Access-Control-Allow-Origin: http://www.vsb.cz**
    - or for any origin: **Access-Control-Allow-Origin: \***
  - **Access-Control-Allow-Credentials: true** to permit cookies
  - **Access-Control-Allow-Headers** may list our custom headers which we want to send to the server, **Access-Control-Allow-Methods** lists the permitted HTTP methods
  - **Access-Control-Expose-Headers** – list AJAX-visible serv. h.
  - **Access-Control-Max-Age** – time to cache the pre-flight req.

# HTML Fetch API

- Makes the requests much more user-friendly. Since iOS 10.1 and on Android 5+ we may use Fetch API (polyfill)
- Basic use: `window.fetch(URL)`, e.g.
  - `fetch('https://homel.vsb.cz/~mor03/TAMZ/ex.json')`  
    `.then(r => { console.log(r.status); return r.json(); })`  
    `.then(data => console.log(data)); //Log returned JSON`
  - Response object mainly: **status**, **statusText**, **type** (basic, cors, opaque), **url**, **redirected** (true/false), **ok** (true/false), **headers**, and optionally **body/bodyUsed**
- You may pass a second argument with settings object:
  - `fetch(myurl, {"mode" : "cors", method : "post", headers : { "X-MyHdr" : "Myval", "X-hdr2" : "V2" }, body: "x=1" })`
  - Attributes: **mode** (see type), HTTP **method**, **headers** object (sent those HTTP headers), **body**, **credentials** (include – will send cookies), **cache** (no-cache), **redirect** (follow/manual/error), **referrer** – when to include it

# JSON with Padding – JSONP

- Old workaround for accessing JSON-like data. Uses `<script>` tag, added to DOM and loaded instead of AJAX
  - `<script type="application/javascript" src="http://example.com/data?callback=apply"></script>`
- Would not work by itself – JSON object does not provide function call or assignment, we must include padding:
  - the function name (which must be supported in server-side scripts) is either part of static data or is specified as an argument in the URL, typically `callback=function_name` or `jsonp=function_name`
    - e.g. `http://ip.jsontest.com/?callback=showMyIP`
    - or `http://time.jsontest.com/?callback=INetDate`
- **Beware – this approach may potentially lead to Cross-site request forgery attack** (we don't use Origin:)
  - a remote site may contain page, which will access potentially sensitive data. **Don't use JSONP where there is some other way** (you may also be running some code!)



# AJAX in jQuery

- Low-level API
  - `$.ajax()` – AJAX call, `$.ajaxPrefilter(handler(options, originalOptions, jqXHR))` – prefilter before `.ajax()` call, `.ajaxTransport()` – creates transport object for data transmitted
- Shorthand methods
  - `$.get(url [,data][,success(data, textStatus, jqXHR)] [, dataType])`
  - `$.post()` - like `$.get`, HTTP POST used, `$.getScript()` – load JavaScript code, `$.getJSON()` – get JSON/JSONP data
- Global event handlers – fired only when global is true
  - `$.ajaxSetup(global)` – set the global AJAX events state
  - `$(x).ajaxXXX(handler(event, XMLHttpRequest, ajaxOpts))`
  - **XXX: Complete, Error, Send, Start, Stop, Success**
- Helper methods
  - `$.param(x)` – serialized representation of obj. or array
  - Forms: `$(form).serialize()` or `$(form).serializeArray()`

# Basic form of jQuery.ajax request

We need this type of request when extended features (e.g. HTTP headers) are required:

```
var jqxhr = $.ajax({
 url: MY_URL,
 type: "POST", // Also: "GET", ...
 data: {attribute: 'text1'},
 headers: { 'Header1': 'Header value 1' },
 success: function(data) {
 alert('I was successful and received data: ' + data);
 }
})
.fail(function(err) {
 alert("Something went wrong: " + err.responseText);
});
```

# Shorthand jQuery.get,.post example

```
var jqxhr = $.get(MY_URL, {attribute: 'text1'}, function(dta) {
 // We can also use $.post or other shorthand method
 alert("success: " + dta);
 })
 .done(function() {
 alert("done...");
 })
 .fail(function() {
 alert("error");
 })
 .always(function() {
 alert("finished");
 });
```

# HTML5-based communication techniques

See e.g.: <http://caniuse.com/eventsource> <http://caniuse.com/websockets>

Examples: <http://www.websocket.org/>

[http://w3schools.com/html/html5\\_serversentevents.asp](http://w3schools.com/html/html5_serversentevents.asp)

# Web Sockets

- Web socket builds a bi-directional connection, similar to standard TCP socket, over HTTP(s).
  - Must be supported by both server & client software to work
- Main Problems: (Transparent) proxies must also support WS
  - No support on basic server installations by default, you will need framework for WS like Socket.IO (for Node.js server)
  - Not supported in Android browser up to Android 4.3
    - But a plugin for WS support exists for Apache Cordova
- URL starts with **ws://** (basic stream) or **wss://** (SSL/TLS)
  - `var ws = new WebSocket('ws://somesite.com/updates');`
  - `ws.send(string);` // Sends message to server
  - `ws.close();` // Closes the socket, can't be reopened
  - Three events which react to server-side socket information
    - `ws.onopen = function(e) { ... };` // Socket opened
    - `ws.onmessage = function(e) { alert( "MSG " + e.data); };`  
// Message received, accessible through `e.data`
    - `ws.onclose = function(e) { ... };` // Socket closed

# Server-Sent Events (SSE)

- A server-sent event – web page gets updates from server automatically (maybe CORS in future due same origin?)
  - No special support library needed on server (unlike WS)
  - Server must send data as **Content-Type: text/event-stream**
    - We should also disable caching: **Cache-Control: no-cache**
- Does not work everywhere, but polyfills are available:
  - eventsourcing.js (e.g. Yaffle), jQuery.eventsource – jQ plugin
- Use is really simple
  - Support check: `if (typeof(EventSource) !== "undefined") {...}`
  - `var src = new EventSource(URL);`
  - `src.onmessage = function(e) {... e.data ...};`
  - Other events: `src.onopen`, `src.onerror`
  - `src.close();` // Closes server connection
  - `src.readyState` – is: 0 – connecting, 1 – open, 2 – closed
- Server sends line-terminated message data, empty line terminates the event. Event data prefixed by **"data: "**
  - We can also include **"id: 123"** and event type **"event: ev1"**

# SSE examples and remarks

- If the browser drops connection for any reason, SSE will start polling (note that the onopen event will be fired each time reconnection happens)
- Multi-line messages are possible, the server will send
  - data: First line**
  - data: Second line**
- Events may have ID, that you can compare if connection drops, retry time (**retry:** ) and custom event type
  - custom ev.: `src.addEventListener("offer", function(e) {...});`
  - id: 2
    - event: stock\_change
    - data: +3**
  - id: 3
    - event: offer
    - data: Will buy 33 pieces for \$15.5**

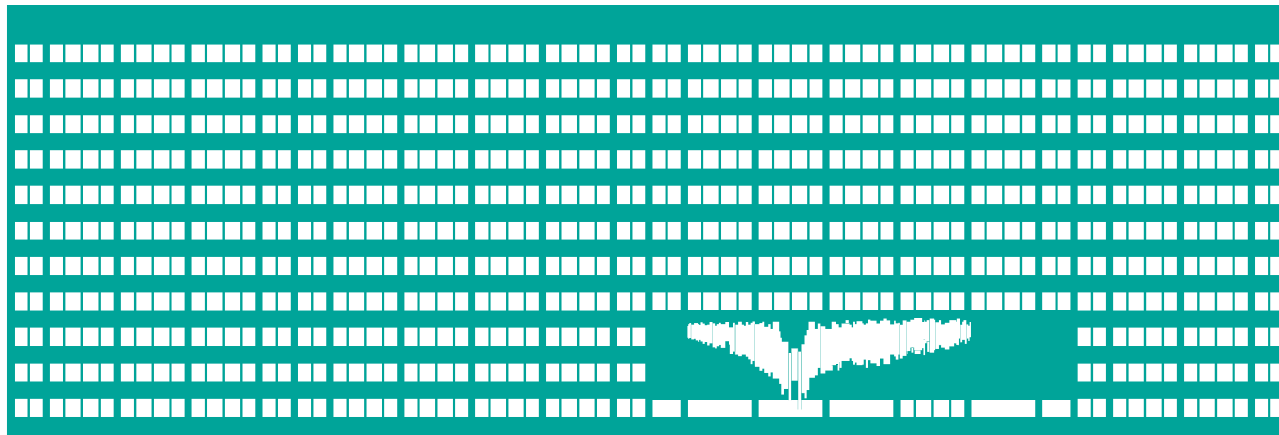
# PHP SSE code to send server time

```
<?php
header('Content-Type: text/event-stream');
header('Cache-Control: no-cache');
date_default_timezone_set('Europe/Prague');
$id=date('u') % 10000;
while(true) {
 $time = date('c');
 echo "retry: 1000\n"; // retry connection after 1s (default ~3s)
 echo "id: " . ($id++) . "\n";
 //echo ": Just a Comment\n";
 echo "event: time_change\n";//Comment out this line if using onmessage
 echo "data: $time\n\n";
 ob_flush();
 flush();
 sleep(1);
}
?>
```

Used in following example: <http://homel.vsb.cz/~mor03/TAMZ/sse/showTime.html>



# Low-level GUI 2D Games



TAMZ 1  
Lecture 6

# Low-level GUI

- Every mobile platform needs a way to draw graphical primitives in (at least) 2D directly to the display/some display area and process low-level input events.
- Before HTML5, one had to use Flash to draw in web browsers, but HTML5 brings several features to do so
- 3 basic ways how to draw graphical objects on mobile platforms:
  - 2D raster graphics (result is a bitmap) – **Canvas**
  - 2D vector graphics – (Inline) **SVG**
  - 3D vector graphics rendered to Canvas – OpenGL
- We need a way how to capture (at least) keyboard and pointing device events – key pressed, released, repeated (?), mouse/touch events

# Low-level GUI in HTML5

- 2D raster graphics – HTML5 Canvas
  - Supported on all current mobile platforms
  - Resolution-dependent result
  - On-the-fly drawing via JavaScript
  - events registered on predecessor; only basic text rendering
  - Well-suited for games (even graphic-intensive)
  - Several game frameworks are available
- 2D vector graphics – HTML5 inline SVG
  - Works on current mobile platforms (Android 3.0+, iOS 5.0+)
  - Resolution-independent, XML-based, has SVG DOM
    - DOM → slow rendering for complex shapes
  - Suitable for large rendering areas, unsuitable for games
  - Supports events
- 3D canvas graphics – WebGL
  - Limited support in current default mobile browsers
  - Uses OpenGL/OpenGL ES to render content into canvas

# Basic low-level events in JS

- JavaScript offers 3 basic keyboard events we may process on the whole document when accessing low-level keypresses
  - **keypress** – keyboard button pressed (or repeated based on OS support, but it may not work e.g. for arrow keys)
  - **keyup** – keyboard button released, polling possible
  - **keydown** – keyboard button pressed, polling possible
- Mouse events (already discussed)
  - **mousedown** – pushed
  - **mousemove** – movement to new coordinates at any time (with or without pressing), event indicates state of buttons
  - **mouseup** – released button
  - **click/tap** – clicked somewhere in GUI.
  - **mouseleave** – cancelled, leaving touch area

# Low-level touchscreen events

- On iOS and Android, we have a set of 4 touch events
  - **touchstart** – we started to touch the screen
  - **touchmove** – we moved the touch point over the screen
  - **touchend** or mouseup events
  - **touchcancel** – touch event cancelled by browser, e.g. because we are leaving touch area or touching more points than the screen supports.
- Touch events provide following event properties:
  - **Touchlists:** **touches**, **targetTouches**, **changedTouches**
    - They have **length** property and **item**(index) method to provide **Touch** object with touch details:
      - Each touch provides **pageX/Y**, **screenX/Y**, **clientX/Y**, **target** and **identifier** (identification number for touch point through all following events) attributes
  - The **ctrlKey/altKey/metaKey/shiftKey** booleans provide status of these modifiers (esp. in desktop browsers)
- ~~On WP in IE10+ we have ms-**pointer**\* events instead~~

# Polling of input events

- Instead of reacting to individual keypresses and (v)clicks, we may set/unset flags based on key state in given time period (so we may combine e.g. UP and LEFT to create diagonal movement).
- We will need a periodically-executed function and flags to record key states → setTimeout() or setInterval() and appropriate time (e.g. 30 to 50 ms)
  - Makes sense on devices with hardware keyboard (at least direction keys) or gamepad, the rest will probably not be able to deliver events fast enough.
- Example of one way of event capture for individual keyboard and vmouse events (+arrow key polling) may be found at [http://home1.vsb.cz/~mor03/TAMZ/low\\_level.html](http://home1.vsb.cz/~mor03/TAMZ/low_level.html)
  - We could easily poll for mouse button presses as well.

# HTML5 2D Canvas

See e.g.: [http://www.w3schools.com/html/html5\\_canvas.asp](http://www.w3schools.com/html/html5_canvas.asp)  
<http://www.html5canvastutorials.com>

Specification: <http://www.w3.org/html/wg/drafts/2dcontext/master/>

# Basic 2D Canvas Drawing Features

- Drawing primitives:
  - rectangles, arcs, lines (+width, joins & caps line styles)
  - Bezier and quadratic curves
  - fill/stroke object (i.e. fill/draw), clear rectangle
- Paths with the possibility of closing & filling do the most of the work to provide complex shapes
- Clipping support
- Scaling, rotating, translating and other transformations
- Support for images, pixel operations
- Support for web fonts
- Support for color, multicolor & gradient fills, shadows
- It is possible to save and restore drawing state of context
- Hit regions for user interaction have been defined in specification, but no support exist yet.



# Canvas element in DOM

- HTML: `<canvas id="myCID" width="200" height="100">`
  - Fallback content can be provided inside of canvas element
  - Two different Canvas “sizes”
    - Canvas drawing area set by **width** and **height**
      - Default: 300px wide and 150 px high
    - Logical Canvas dimension: CSS width/height
      - Default: drawing area dimensions
- JavaScript access to existing Canvas Element
  - `var c1 = document.getElementById("myCID");`
  - `var c2 = $("#myCID").get(0);`
- Once we have the Canvas element in DOM, we need to access its drawing context (2D, 3D or webgl)
  - `var context1 = c1.getContext("2d");`
  - `var context2 = c2.getContext("3d");` // 3D – won't work
- We can create Canvas dynamically without showing it
  - `var c3 = document.createElement('canvas');`

# Canvas context drawing state

Contains information needed for drawing graphics primitives:

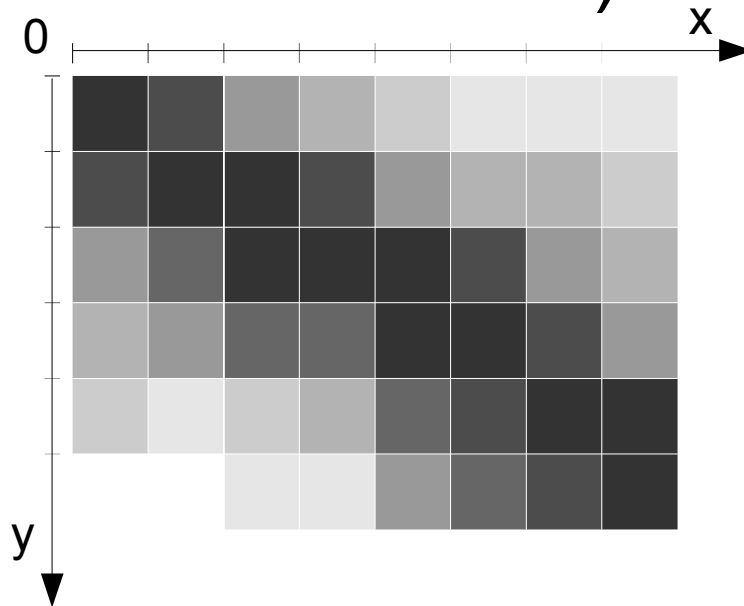
- Transformation matrix (legacy API, SVGMatrix in newer b.)
- Stroke and fill style (strokeStyle, fillStyle) – color, gradient (linear or radial) or pattern
- Global Alpha channel setting (globalAlpha) – 0.0 (tr.) → 1.0
- Line settings (lineWidth, lineCap, lineJoin, miterLimit)
- Shadow setting (shadowOffsetX/Y, shadowBlur, shadowColor)
- Text setting (font, textAlign, textBaseline)
- Clipping region (of an image or when drawing)
- Global composite operation (globalCompositeOperation)

Saving and restoring drawing context state (all of above):

- context.save() – pushes context state to the stack
- context.restore() – pops context state from the stack

# Coordinate system

- The coordinate system works the same way as the coordinates in the whole HTML document, relative to Canvas element  $(0, 0) \rightarrow$  top-left corner pixel center.
- The GUI output may use anti-aliasing or multiple pixels on high-resolution screens (when CSS and canvas dimensions do not match).



# Color Model

- The 24-bit color model consistent with CSS colors has eight bits for each of **red**, **green**, and **blue**, 8 bits in 32-bit integer are unused. For example **#ff8000** is **orange**. However, when we specify the missing 8 bits, we are requesting RGBA representation with alpha channel **#RrGgBbAa** (Aa=00 – transparent, Aa=ff – opaque)
  - We can use other notations: #RGBA #RGB, e.g. **#f80**, **rgb(255,128, 0)**, **rgba(rd, gd, bd, ad)**, **hsl**, **hsla**, **gray**, ...
  - Default background color is in most cases “transparent black, i.e. #00000000 or **rgba(0, 0, 0, 0)**).
  - We can adjust existing color or its component, e.g. **rgba(+20, -10, 50%, \*5)**, **whiteness(20%)**, **hue(+5)**, ...
  - *Note: Setting alpha channel component in colors does not work in many browsers, but `context.globalAlpha` does.*
- Not all devices support full 24 bits of color and map the color requested by the application → color available
  - As a result, we may consider using “Web-safe colors”

# Line styles

- We can specify basic line settings one would expect in context object
  - context.lineWidth – line width (finite, greater than 0)
  - context.lineCap – line cap style (butt, round, square)  

  - context.lineJoin – line join style (bevel, round, miter)  

  - context.miterLimit – maximum length to cut the arrow point in miter line joins (distance between inner and outer corner of line meet), otherwise bevel line join is used
  - context.setLineDash(segments)/.getLineDash()
    - Dashed line definition: [5] → dash & space: 5px, [1, 2] → 1px dot & 2px space, [3, 5, 1, 5], ...  

    - context.lineDashOffset – dash pattern offset (same units)

# Anchor Points and Text Alignment

- Anchor points are used to minimize the amount of required computations when placing objects (typically, **top/center/bottom/baseline** and **left/center/right**)
- For text, we also define **textAlign** and **textBaseline**, which can be used to calculate the anchor point
  - context.**textAlign**: start/end (depends on **direction**, which can be inherited, ltr or rtl), left, right, center
  - context.**textBaseline**: top, **hanging**, middle, **alphabetic**, **ideographic**, bottom

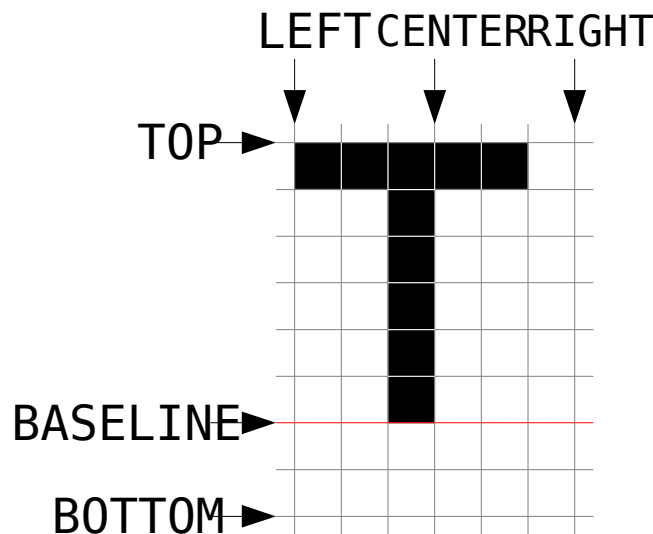


Image source: <http://www.w3.org/TR/2dcontext/>

# Text Fonts in Canvas

The font is set with `context.font`, standard font definitions (know from CSS) may be used:

- We should use only vector fonts, scaling would make the result look ugly.
- Typically, we set family, size, and style.
  - Many different font families (e.g. Helvetica or Verdana)
    - Web fonts may be loaded (only on some mobile platforms)
  - Generic families: `serif`, `sans-serif`, *cursive*, *fantasy*, `monospace`
    - For generic families, it is up to the device to select a font that most closely matches the requested attributes (and maybe the font will not fit correctly).
  - `text-style`: `normal`, *italic*, *oblique* bold fonts and horizontal stretch: controlled by `font-weight` (400 **600**), `font-stretch`
  - `font-size`: size specifier (150%, 15px, ...), adjustment or absolute:  
`xx-small`, `x-small`, `small`, `medium`, `large`, `x-large`, **`xx-large`**
  - Example: `context.font='italic 600 12px Droid Sans, sans-serif';`

# Coordinate transformations (2D)

- Following methods modify the transformation matrix, they are considered a legacy API, but can be still used:
  - Scaling: `context.scale(scale_x, scale_y)` – sets scale factors
  - Translation: `context.translate(dx, dy)` – adds dx and dy to current translation (negative values are possible)
  - Rotation: `context.rotate(angle)` – rotates clockwise by angle (in radians)
  - The transformations must be performed in reverse order.
  - Matrix manipulation: `context.transform(a, b, c, d, dx, dy)`
    - Multiplies current transformation matrix by 3x3 matrix based on a-f coefficients
  - `context.setTransform(a, b, c, d, dx, dy)` – sets the transformation matrix to supplied coefficients
- In some browsers you may use `context.currentTransform` to get/set transformation SVGMatrix object and use its set of methods to modify the matrix.

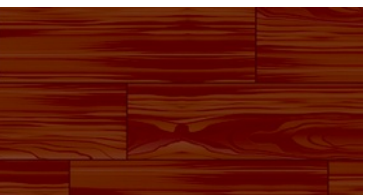
a	b	dx
c	d	dy
0	0	1



# Fill and stroke styles

Attributes `context.fillStyle`, `context.strokeStyle` may be set to:

- String containing color, e.g. `#7f4f10`, `rgba(127,79,16,127)`
  - By default, both attributes are set to `#000000` (black)
- *CanvasGradient* object (gradient):
  - `var grad1 = context.createLinearGradient(x0, y0, x1, y1);`
    - Linear gradient along the line `[x0,y0] → [x1, y1]` 
  - `grad2 = context.createRadialGradient(x0, y0, r0, x1, y1, r1);`
    - Radial gradient starting with circle `[x0, y0]` with radius `r0` and ending with circle `[x1, y1]` with radius `r1` 
  - `grad1.addColorStop(offset, color);` 
    - Sets gradient color on given offset (between 0.0 and 1.0)
- *CanvasPattern* object (image pattern)
  - `var pattern = context.createPattern(image, repetition);`
    - **Image**: `HTMLImageElement`, `HTMLCanvasElement`, or `HTMLVideoElement`.
    - **Repetition**: `repeat` (both x & y), `repeat-x` (horizontal only), `repeat-y` (vertical only), and `no-repeat` (neither)



# Casting shadows

- Each drawn object may cast a shadow controlled by following settings:
  - context.shadowColor - color of the shadow to cast.
  - context.shadowOffsetX - horizontal shadow offset
  - context.shadowOffsetY - vertical shadow offset
  - context.shadowBlur - level of blur applied to shadows. It is NOT the number of pixels.
- This part of 2D Canvas API may be subject of change in the future.



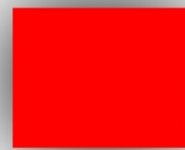
Blur Level 1



Blur Level 10



Blur Level 20



Blur Level 50



Blur Level 100

# Placing images on Canvas

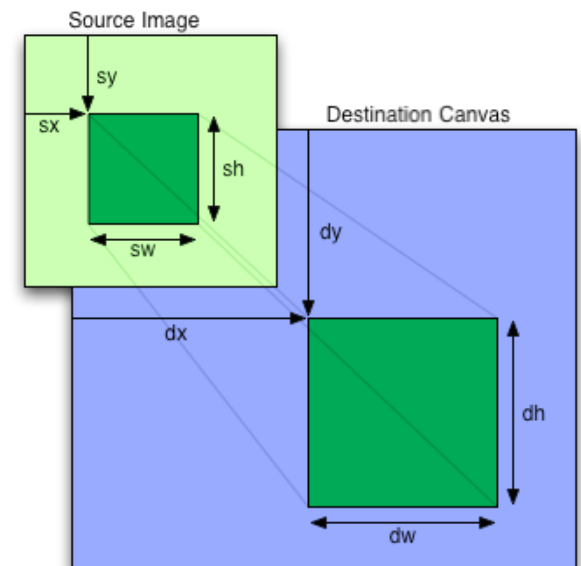
Draws image (or another canvas, video) on Canvas

- `context.drawImage(image, dx, dy)`
- `context.drawImage(image, dx, dy, dw, dh)`
- `context.drawImage(image, sx, sy, sw, sh, dx, dy, dw, dh)`

The arguments are interpreted as follows:

- `dx, dy` – image position on canvas
- `dw, dh` – destination size on canvas
- `sx, sy` – position in source image
- `sw, sh` – size of cut from original image

Can be used for double buffering and larger drawn area with a smaller viewport



# Compositing two images

- Set by context.`globalCompositeOperation`
- Composites image areas based on the operation
- Permitted modes for **A** (source) and **B** (destination):
  - **source-atop**, **source-in**, **source-out**, **source-over** (default)
  - **destination-atop**, **destination-in**, **destination-out**, **destination-over**
  - **lighter** –  $A+B$
  - **copy** – A (B is ignored).  
Display the source image instead of the destination.
  - **xor** – Exclusive OR of A and B areas
  - *vendor-operation*



source-atop



source-in



source-out



source-over



destination-atop



destination-in



destination-out



destination-over



lighter



darker



xor



copy

# Placing text on Canvas

The text can be drawn filled or as an outline

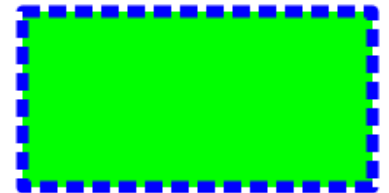
- `context.fillText(text, x, y [, maxWidth]);` – filled text
- `context.strokeText(text, x, y [, maxWidth]);` – outline text
  - `maxWidth` – if the text exceeds `maxWidth`, it will be scaled

Sometimes we need to measure the potential text width:

- `var metrics = context.measureText(text);`
  - returns `TextMetrics` object with for given text and current font set in the context
  - The actual width is then available through `metrics.width`
- Note that we need to supply actual text for proportional fonts, the metrics is a  $(\text{\#of\_characters} * \text{character\_width})$  only for `monospaced` fonts.

# Rectangles in Canvas

- The rectangle is the only graphic primitive, which can be filled and cleared directly without building a path.
- Three basic methods to draw rectangles directly:
  - `context.clearRect(x, y, w, h)` – clears all pixels on the canvas in the rectangle to transparent black (#0000).
  - `context.fillRect(x, y, w, h)` – paints filled rectangle with given fill size immediately.
  - `context.strokeRect(x, y, w, h)` – draws a box of rectangle outline with given rectangle using current stroke style. Line styles set in context are being used.
- When combining with other objects in a path, we use `context.rect(x, y, w, h)` instead.



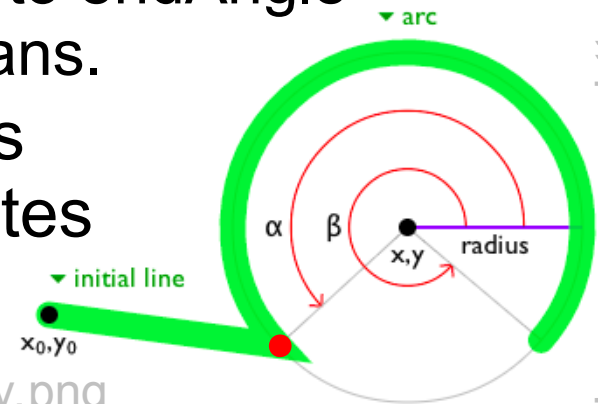
# Working with paths

More complex graphical operations besides simple text and rectangle drawing are done by creating a path and filling/stroking it:

- context.**beginPath**() - creates/resets (empties) a path
- context.**fill**() – fills area defined by path's subpaths
- context.**stroke**() – strokes path's subpaths
  - Both fill and stroke use corresponding styles in context
- context.**clip**() – constrains a path by clipping region (e.g. bounding box, see general clipping later)
- context.**isPointInPath**(x, y) – is point [x, y] on current path or inside of the area defined by the path?
- context.**drawFocusIfNeeded**(element) – indicates fallback element location, if focused, draws focus outline around current path (clipping applies).

# Building a path – basic shapes

- All drawing methods start with current point  $[x_0, y_0]$ , the last point of the path, as the reference point
  - `context.lineTo(x, y)` – draws a straight line from current position to  $[x, y]$ . Can be used to draw polygons & triangles
  - `context.moveTo(x, y)` – moves to  $[x, y]$  without drawing and creates a new subpath.
  - `context.rect(x, y, w, h)` – rectangle as a part of the path
  - `context.arc(x, y, r, startAngle, endAngle[, anticlockwise])`
    - draws a circle or arc with origin  $[x, y]$  and given radius  $r$ , connected to the start of the arc from previous point by a straight line (can be avoided by `context newPath()`).
    - The default drawing is from `startAngle` to `endAngle` clockwise, angles are specified in radians.
  - `context.closePath()` – makes the previous subpath closed (i.e. to be filled) and creates a new subpath starting at  $[x_0, y_0]$ .





# Building path – extended shapes

All three methods provide a non-linear interconnection between point  $[x_0, y_0]$  and  $[x, y]$

- `context.arcTo(x1, y1, x2, y2, radius)` – creates line and shortest arc from actual point  $[x_0, y_0]$  with given radius using  $[x_1, y_1]$  as a reference point for tangents with  $[x_0, y_0]$  and  $[x_2, y_2]$ . The arc will end in unspecified point  $[x, y]$ .
  - Note: implementation in some browsers is buggy!
  - May be used e.g. for rounded rectangles
- `context.quadraticCurveTo(cpx, cpy, x, y)` - draws a quadratic Bézier curve to  $[x, y]$  with control point  $[cpx, cpy]$ .
- `context.bezierCurveTo(cp1x, cp1y, cp2x, cp2y, x, y)` - draws a cubic Bézier curve to  $[x, y]$  with control points  $[cp1x, cp1y]$  and  $[cp2x, cp2y]$ .

# String widths – automatic line break

It is possible to determine width of string typed using given font.

```
function wrapText(context, text, x, y, maxWidth, lineHeight) {
```

```
 var words = text.split(' ');
```

```
 var line = "";
```

```
 for(var n = 0; n < words.length; n++) {
```

```
 var testLine = line + words[n] + ' ';
```

```
 var metrics = context.measureText(testLine);
```

```
 var testWidth = metrics.width;
```

```
 if (testWidth > maxWidth && n > 0) {
```

```
 context.fillText(line, x, y);
```

```
 line = words[n] + ' ';
```

```
 y += lineHeight;
```

```
 } else {
```

```
 line = testLine;
```

```
 }
```

```
 }
```

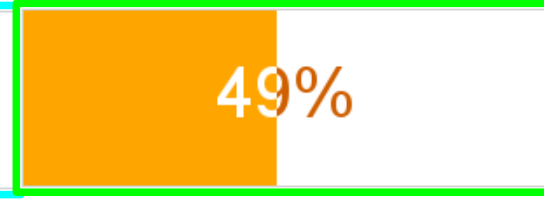
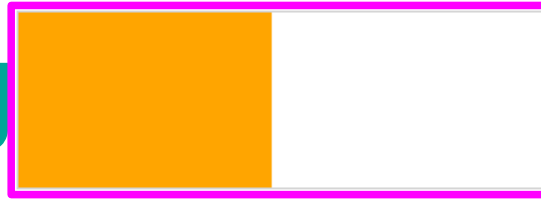
```
 context.fillText(line, x, y);
```

```
}
```

All the world 's a stage, and all the men and women merely players. They have their exits and their entrances; And one man in his time plays many parts.

Source: <http://html5canvastutorials.com/tutorials/html5-canvas-wrap-text-tutorial/>

# Clipping



- The **clip** is the set of pixels in the canvas context object that may be modified by graphics rendering operations.
  - The only pixels modified by graphics operations are those that lie **within** the clip. Pixels outside the clip are not modified by any graphics operations.
- `context.clip([path])` – clips content according to specified or current path
  - `.resetClip()` is not present everywhere → use `.save()` and `.restore()` instead.

```
ctx.textAlign='center';
ctx.textBaseline='middle';
ctx.font='40px sans-serif';
ctx.save();
 ctx.beginPath();
 ctx.rect(leftL, topL, wL, hL);
 ctx.clip();
 ctx.fillStyle=colorL_bck;
 ctx.fillRect(0,0,width,height);
 ctx.fillStyle=colorL;
 ctx.fillText(label, text_x, text_y);
ctx.restore();
ctx.save();
 ctx.beginPath();
 ctx.rect(leftR, topR, wR, hR);
 ctx.clip();
 ctx.fillStyle=color_R;
 ctx.fillText(label, text_x, text_y);
ctx.restore();
```

# RAW access to image data

Raw data access is done through ImageData structure, which allows us to modify individual pixels. Same origin policy limitations apply. CSS dimensions are being used.

- `imagedata=context.createImageData(sw, sh)` – create empty image with given dimensions, filled with trans. black
- `imagedata=context.createImageData(imagedata)` – clones dimensions from existing buffer, fills with transparent black
- `imagedata=context.getImageData(sx, sy, sw, sh)` – returns pixel-based representation of canvas data from sx, sy with dimensions sw, sh.
- `context.putImageData(imagedata, dx, dy [, dirtyX, dirtyY, dirtyWidth, dirtyHeight ])` – puts (scaled) image data to specified canvas context
- Dimensions: `imagedata.width`, `imagedata.height`
- Actual data: `imagedata.data` – one-dimensional array with RGBA order and values 0-255 for each (CSS) pixel component.

# HTML5 Canvas Example

```
var c=document.getElementById("myCanvas");
var ctx=c.getContext("2d");

var grd=ctx.createRadialGradient(75,50,5,90,60,100);
grd.addColorStop(0,"yellow");
grd.addColorStop(1,"#7f7f7f");

ctx.strokeStyle="red";
ctx.strokeRect(0,0,250,100);

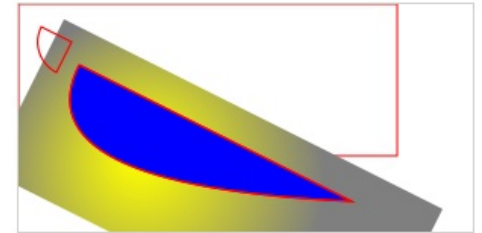
ctx.transform(1,0.5,-0.5,1,30,10);

ctx.fillStyle=grd; ctx.fillRect(0,0,250,100);

ctx.fillStyle="blue"; ctx.beginPath(); ctx.moveTo(20,20);
ctx.quadraticCurveTo(20,100,200,20);ctx.lineTo(20,20);

ctx.fill(); ctx.stroke();

ctx.moveTo(10,10);
ctx.arc(10,10,20,Math.PI/2, Math.PI);
ctx.closePath(); ctx.stroke();
```



# HTML5 SVG support

See e.g.: [http://www.w3schools.com/html/html5\\_svg.asp](http://www.w3schools.com/html/html5_svg.asp)  
[http://www.w3schools.com/svg/svg\\_examples.asp](http://www.w3schools.com/svg/svg_examples.asp)  
Specification: <http://www.w3.org/TR/SVG/>

# SVG support

- SVG in HTML5
  - SVG is XML based – we can use DOM, on the other hand, Canvas is a pixel buffer (once drawn, we can't change some of the primitives)
  - Shape in SVG is represented as an object – if something changes, we re-render it when changed
  - SVG is resolution-independent vector graphics
  - Event handlers are supported
  - We can place inline SVG into HTML document
- Due to limited use of SVG in this subject, we will not discuss the SVG in detail

```
<svg xmlns="http://www.w3.org/2000/svg" version="1.1" height="190">
 <polygon points="100,0 0,100 100,100 0,0"
 style="fill:yellow;stroke:red;stroke-width:5;fill-rule:evenodd;">
</svg>
```



# WebGL support

(iOS 8+, IE Mobile 11+, FF,  
partially Chrome on Android 40+)

See e.g.: <https://developer.mozilla.org/en-US/docs/Web/WebGL>  
<https://www.khronos.org/webgl>

Examples: <http://www.awwwards.com/22-experimental-webgl-demo-examples.html>  
[http://www.html5rocks.com/en/tutorials/webgl/webgl\\_fundamentals/](http://www.html5rocks.com/en/tutorials/webgl/webgl_fundamentals/)

Specification: <https://www.khronos.org/registry/webgl/specs/1.0/>



# WebGL support

- Limited support on mobile platforms, not of much use
  - `var gl = canvas.getContext("webgl");`
  - `gl.viewport(0,0,gl.drawingBufferWidth,gl.drawingBufferHeight);`
- Based closely on OpenGL ES 2.0 API and its methods
  - context, stencils, vertices, matrices, scissors
  - textures, blends, text output
  - framebuffer and renderbuffer, pixel manipulation
  - shaders and programs, uniforms and attributes

```
var canvas = document.getElementById("glcanvas");
var gl = null;
try {
 gl = canvas.getContext("webgl") || canvas.getContext("experimental-webgl");
} catch(e) {}
if (gl) {
 gl.clearColor(0.0, 0.0, 0.0, 1.0);
 gl.enable(gl.DEPTH_TEST);
 gl.depthFunc(gl.LEQUAL);
 gl.clear(gl.COLOR_BUFFER_BIT|gl.DEPTH_BUFFER_BIT);
}
```

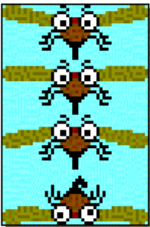
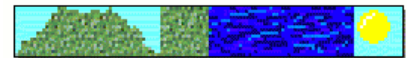
# HTML5 JavaScript Game Frameworks

See e.g.: <http://www.remcodraijer.nl/quintus/tutorial.html>

Frameworks: <http://html5quintus.com/> <http://kineticjs.com/> <https://phaser.io/>

# Basic game framework features

- Use existing technologies (HTML, JS, Canvas, SVG, ...) but provide easier way how to write games
- We could write things which game framework provide ourselves, but it would take some time
- Typically the frameworks use well-known approaches to provide basic entities to programmers.
- Basic generalizations used in game frameworks:
  - Layers – each layer represent a different part of game (map, player, enemies, items, ...)
    - TiledLayer – typically used to define maps with rectangular (sometimes hexagonal) grid. May be used both for vertical (e.g. Angry Birds) and horizontal (e.g. Civilization, PacMan, ...) games.
    - Sprite – animated object which represents a player, creature, ...
  - Scene/Stage – binds layers together to provide level or some other logical part of the game.



# Example game visual representation

The actual game subsystem may consist of:

- **InputSystem** – button and mouse input subsystem. It contains drawing methods & controls to set possible inputs
- **Scene** – typically a basic scene class, containing mainly the scene function and options passed to stage
- **Stage/Viewport** – base game object class for stage, manages sets of sprites, manipulates **viewport**.
- **Sprite** – basic sprite game object class, rendering either an asset or a frame from a sprite sheet. Width & height may be generated automatically from the supplied image assets, or set manually. Typical subtypes are:
  - **MovingSprite** – a sprite that adding basic Newtonian physics on each step
  - **TiledLayer** – **tilled layer sprite used as game map**
- **SpriteSheet** – class defining sprite parameters
- **Tweens** and **animation** sheet modules may also be used.
- Controls are used for sprite behavior, e.g. **stepControls** or **platformerControls**
- Sound effects may be included in games



- HTML5 game engine (2D games)
- JavaScript-friendly syntax (jquery-like)
- Runs on desktop and HTML5-compatible mobile devices
- Released under MIT License
- Supports Tiled layer/map/scene editor with TMX output:  
<http://www.mapeditor.org/> or combination of images and JSON data to define the sprite animations, sheets, etc.
- Available at: <http://html5quintus.com/>
- Basic functions:
  - collision detection
  - sprite animations
  - tile layers
  - sound
  - input handling

For API documentation, see: <http://html5quintus.com/api/>

For gaming tutorial, see e.g.: <http://www.remcodraijer.nl/quintus/tutorial.html>



# Phaser game framework

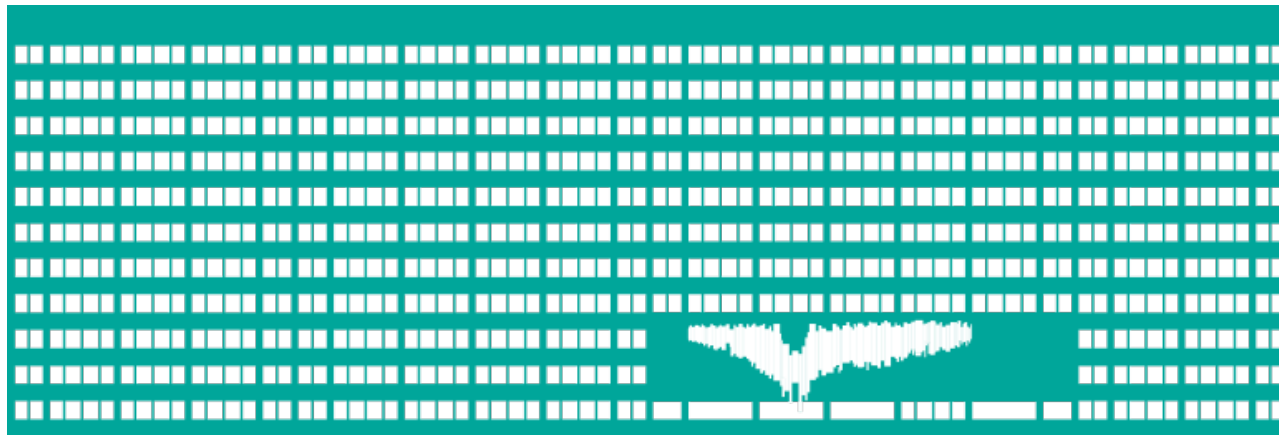
- Phaser supports following features:
  - 2D Canvas and WebGL
  - Preloading resources (images, sounds, metadata)
  - Physics for arcade, simple Axis-Aligned Bounding Box and impact physics
  - Animated Sprites and Tiled maps (with animated tiles in v3)
    - Groups of sprites are available
    - Sprite Animations by sprite sheets and some texture fmts
  - Particle system for explosions, jets, rain, ...
  - Multiple Camera positions, with the possibility to follow spr.
  - Multiple inputs support (mouse, keyboard, touch screen)
  - Sounds (including new Web Audio API – mainly desktop)
  - Scaling the game for screen dimensions & full-screen
  - Plug-in support through complex plug-in system

# Kinetic.js – games and graphics

Another framework for advanced image manipulations: you may add event listeners, draw shapes & images, move, scale and rotate them, use layers, detect image hits, drag&drop, ...

- Canvas wrappers: **Arc, Canvas, Circle, Context, Image, Line, Path, Rect, Text, Transform**
- Custom graphical elements: **Ellipse, Label, Ring, TextPath, RegularPolygon, Star, Tag, Wedge**
- Base classes: **Collection, Container, Node, Shape, Util**
- Item manipulation: **Filters, Easings** (transitions)
- Animations: **Animation, Tween**
- Gaming API (offers collision detections):
  - **Stage** – defines the whole scene (used with other parts, too)
  - **Group** – grouping elements and working with whole group
  - **Layer** – separate GUI items with different z-index and visibility setting
  - **Sprite** – animated 2D sprite

# Timers HTML5 APIs



TAMZ 1  
Lecture 7



# Working with timers

See e.g.: [http://www.w3schools.com/jsref/met\\_win\\_setinterval.asp](http://www.w3schools.com/jsref/met_win_setinterval.asp)  
[http://www.w3schools.com/jsref/met\\_win\\_settimeout.asp](http://www.w3schools.com/jsref/met_win_settimeout.asp)  
<http://javascript.info/tutorial/settimeout-setinterval>

# Why would we use timers?

- Basic JavaScript offers no background threads but we want to act, not just react to external events
- We need to do some actions repeatedly
  - Change time on clock
  - Check for new messages periodically
  - Animate some image (but we may use animated GIF instead)
  - ...
- We need to sample/do some task with reasonable time granularity



# Scheduling tasks with timers

There are basic two approaches to task scheduling:

- Scheduling **one-time tasks** – executes task after specified delay once
  - Set a timeout before retrying, error message, ...
- Scheduling repeating tasks – executes task at regular intervals. There are two variations for scheduling repeating tasks.
  - **Fixed-delay** – each execution of a task is based solely on how long it was since the previous task finished.
    - e.g. checking for new E-mails 5 minutes after the last mailbox access has been finished
  - **Fixed-rate** – each execution of a task is based on the time the first task call has started and the requested delay between tasks.
    - e.g. checking which keys are currently pressed

# JavaScript timers

- **One-time** task

- `var id = window.setTimeout(function, timeout_ms);`
  - e.g. `setTimeout(onetime, 8000);` // redraw after 8 s
- Cancelling the timeout: `window.clearTimeout(id);`

- **Fixed-rate** repeated task

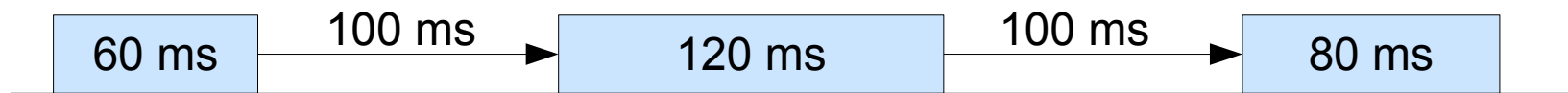
- `var id = window.setInterval(fixed_rate, interval_ms);`
  - e.g. `setInterval(repaint, 2000);` //function repaint every 2 s
- Stopping the timer: `window.clearInterval(id);`

- But what about **fixed-delay** task? It is possible:

- We can plan next one-time task on the end of one-time task function call (and stop it with `window.clearTimeout(id);`)
  - `var id = window.setTimeout(fixed_delay, first_ms);`  
    `function fixed_delay() {`  
        `...`  
        `id = window.setTimeout(fixed_delay, next_ms);`  
    `}`

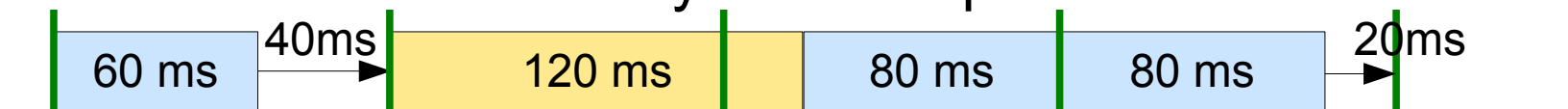
# Specific situations and use

- We can use object's method, i.e. `setTimeout(obj.me, 80);`
- Minimal granularity in JavaScript is between 1ms & 15ms
  - We don't know the exact time, but: 9ms ~ 10 ms ~ 11ms
- What happens if the function call takes longer than the delay we have allocated between two calls
  - Fixed-delay – no change, the interval is fixed regardless on how long the function call will take

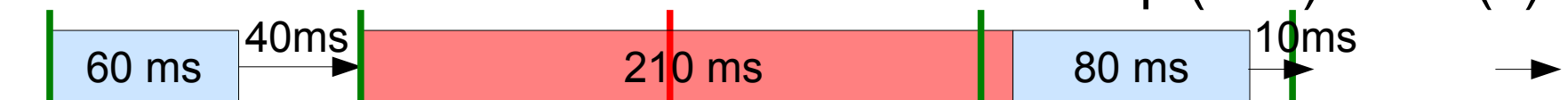


- Fixed-rate – the situation is more complicated. Only one event is queued, so two things may happen:

- The call finishes before the next interval is over → start the call immediately after the previous one finishes



- Call finishes after next interval → we skip (one) event(s)



# Do you need high-precision time?

- Some applications need high-precision timers
  - For very short time intervals
  - For performance measurement of individual parts of your application
- But these timers can be (ab)used for attacks such as Spectre, so intentional fuzzing has been introduced e.g. in Firefox.
- Usage (see [MDN](#)):
  - `performance.now()` → timestamp up to microseconds
  - `performance.mark('my_start')`
  - `performance.measure('name', 'my_start' [, 'my_end'])`
  - `performance.getEntries/...ByName/...ByType("measure")`
  - `performance.clearMarks();`
  - `performance.clearMeasures();`

# Web Workers

(iOS 5.0+, Android 4.4+, IE 10)

See e.g.: [http://w3schools.com/html/html5\\_webworkers.asp](http://w3schools.com/html/html5_webworkers.asp)  
<http://www.html5rocks.com/en/tutorials/workers/basics/>  
[https://developer.mozilla.org/en-US/docs/Web/Guide/Performance/Using\\_web\\_workers](https://developer.mozilla.org/en-US/docs/Web/Guide/Performance/Using_web_workers)

Specification: <http://www.w3.org/TR/workers/>

# What are web workers?

- A way how to bring Threads into JavaScript
- It is a (separate) JavaScript file that runs in background, independently of other scripts.
- Shouldn't affect the performance of page (i.e. should not cause “The script is not responding...” message).
- Use messaging (events) to communicate between main application and web worker
  - The worker posts messages to main thread with `postmessage`, which are processed by `onmessage` event on worker object and vice versa.
- We would really benefit from them, were they available on all mobile platforms, but for now it is not possible
- Workers are terminated by calling `terminate()` on worker object.



# Web workers example

```
var w;
function startWorker() {
 if(typeof(Worker)!="undefined") {
 if(typeof(w)=="undefined"){
 w=new Worker("counter.js");
 }
 w.onmessage = function(event) {
 document.getElementById
 ("result").value=event.data;
 };
 }
}

function stopWorker() {
 w.terminate();
}
```

## counter.js

```
var i=0;
function increase() {
 i=i+1;
 postMessage(i);
 setTimeout(increase, 500);
}
increase();
```



# Web workers remarks

- Thread safety is considered quite important in the design of web workers, but in very rare situations, race condition may still occur.
- If we want to terminate a worker from itself, we do this by calling: `self.close()`;
- Workers may spawn another workers
- Navigator object is accessible, but objects from the main thread are not shared with web worker (incl. DOM)
- To load external scripts from web worker, we may use the global `importScripts()` function call.
- To detect runtime errors in worker thread, we may register the `onerror` event on worker object.

# HTML5 Geolocation

See e.g.: [http://www.w3schools.com/html/html5\\_draganddrop.asp](http://www.w3schools.com/html/html5_draganddrop.asp)  
<http://diveintohtml5.info/geolocation.html>  
<http://html5demos.com/geo>  
[https://developer.mozilla.org/en-US/docs/WebAPI/Using\\_geolocation](https://developer.mozilla.org/en-US/docs/WebAPI/Using_geolocation)

Specification: <http://www.w3.org/TR/geolocation-API/>

# Geolocation on a mobile platform

- How?
  - from WiFi, GSM, GPS, DNS, GeoIP, ...
  - Oneshot query or periodic update events
- Why?
  - to permit location-aware applications
  - to offer maps, proximity search, ...
- Is it safe?
  - The user must permit page access to location information
  - Browser may contact some service (e.g. Google API) and send potentially sensitive data (cell IDs, visible WiFi AP MAC addresses, ...)
    - Service may require payment, e.g. data connection
- What is the result?
  - Gives the application provided location information (if permitted by the user and processed by the server) through programming API.

# Basic HTML5 Geolocation use

- New BOM object: navigator.**geolocation** with 3 methods:
  - **.getCurrentPosition**(posCallback [,errorCB][, options]);
    - One-shot position is provided to callback
  - watchId = **.watchPosition**(posCallback [,errorCB][, options]);
    - Registers callback to receive repeated updates
  - **.clearWatch**(watchId);
    - Stops receiving repeated updates
- The supplied callback functions have following interfaces:
  - posCallback(Position **position**);
    - **Position** object – contains readonly attributes **coords** (Coordinates) and **timestamp** (DOMTimeStamp)
  - errorCB(PositionError **positionError**);
    - **PositionError** object – contains **code** – error code which is PERMISSION\_DENIED (1), POSITION\_UNAVAILABLE (2) or TIMEOUT(3) and **message** describing the error.

# Geolocation options

Options object contains 3 possible settings:

- **enableHighAccuracy** – if set to true, tries to use the best positioning method available, which may take longer and consume more power. For example use GPS instead of network-based location. Default value is false.
- **timeout** – how long (in [ms]) should we wait for result before the positioning request fails (and if supplied, errorCallback is called, with code set to PositionError.TIMEOUT)
  - Default value is Infinity – we wait until the position is ready
- **maximumAge** – how old (in [ms]) may be the cached location are we willing to accept. If set to 0 (default value) we request a new position each time, i.e. no location caching is done.

# Geolocation Coordinates

Coordinates object uses WGS-84 datum and reference ellipsoid and contains following readonly attributes:

- **latitude, longitude** – latitude (N/S position) and longitude (W/E) in degrees (DD.ddddd). Must be implemented
- **accuracy** – position accuracy level in meters. Must be impl.
- **altitude** – height in meters against WGS-84 ellipsoid/**null**
- **altitudeAccuracy** – accuracy of altitude in meters or **null**
  - Both accuracy values are for 95% confidence interval
- **heading** – the direction of travel of the device and in **degrees**,  $0^\circ \leq \text{heading} < 360^\circ$ , clockwise from *true north*.  
Not available → **NaN**
  - Heading is calculated, not the actual device orientation
- **speed** – magnitude of the horizontal component of the device's current velocity in meters/second [m/s] or **null**
- Only latitude, longitude and accuracy must be provided.



# Basic Geolocation example

```
<!DOCTYPE html>
<html> <body onload="getLocation()">
<input type="text" id="ltext" style="width: 95%; border: none;" />
<script>
var x=document.getElementById("ltext");
function getLocation() {
 if (navigator.geolocation !== undefined) {
 navigator.geolocation.getCurrentPosition(showPosition, failedPosition);
 }
}

function showPosition(pos) {
 x.value="" +pos.coords.latitude + " / " + pos.coords.longitude +
 " @ " + pos.coords.altitude +"m (+/- " + pos.coords.accuracy + "m"); }
function failedPosition(err) { x.value = err.message + ' (code: ' + err.code + ')'; }
</script>
</body></html>
```



# Geolocation final remarks

- No additional data, such as number of satellites in GPS or raw NMEA sentences from GPS are available
- We don't access device orientation sensor and compass with this API.
- The request for network-based location requires network access, i.e. we have to be on-line
- The user may decide not to provide the actual location (`PositionError.PERMISSION_DENIED`)
  - The permission for the site may be stored forever, be valid only during the session, for loaded page or it may have to be granted each time the position is requested by calling geolocation methods (`getCurrentPosition`, `watchPosition`).
- The browser may fail to obtain data through external API due to limits, API changes, etc.

# HTML5 Multimedia

**(No single video format is usable  
on all platforms/browsers)**

See e.g.: <http://www.html5rocks.com/en/features/multimedia>  
<http://diveintohtml5.info/video.html>  
<http://html5multimedia.com/>  
[http://www.w3schools.com/html/html5\\_video.asp](http://www.w3schools.com/html/html5_video.asp)  
Specification: <http://www.w3.org/TR/html5/embedded-content-0.html>

# HTML5 Multimedia

In HTML5, you can embed audio or video using native HTML tags **audio** and **video**, and if the browser supports the tags, it will give users controls to play the file.

- No plugins needed
- Better performance
- Native, accessible controls
- The **audio** element is used for embedding an audio player inside a page for a particular audio file.
  - `<audio src="music.ogg" controls="true" preload="true"></audio>`
- The **video** element embeds a video player for a particular video file.
  - `<video src="movie.ogv" controls width="390"></video>`

# Multimedia Codecs

- **Audio** – the audio track is compressed, stored, and decoded according to a codec. The most relevant audio codecs are:
  - **MP3**: Widespread, but patent-encumbered.
  - **AAC (Advanced Audio Coding)**: Patent-encumbered. Used in Apple products.
  - **Ogg Vorbis**: Free, open-source, patent-free
- **Video file**, like an ".avi" file, is really a **container** for multiple related files that describe a video, like **video track**, one or more **audio tracks** with synchronization markers and **metadata** (title, album art, etc). The most popular video containers are:
  - **MPEG4**: .mp4, .m4v
  - **WebM**: .webm
  - **Ogg**: .ogv

# Video Codec and Container Support

- There are **no official codecs** for browsers to support
- For current top **mobile** platforms, MP4 (H.264+MP3/AAC)

	IE/Edge	FireFox	Chrome	Opera	iOS	Android
HTML video element	9.0+	3.5+	3.0+	10.5+	1.0+	2.0+
<b>Theora + Vorbis</b> Video format + cont.	No/17+	3.5+	5.0+, no Android	10.5+ no A.	No	No
<b>H.264 + MP4</b> Video format + cont.	9.0+	35+ (Android – only SW decoding)	5.0+	25+	3.0+	2.0+ (4.4+ fully)
<b>H.265 video format</b>	No	No	No	No	11+	No
<b>WebM (VP8, VP9 + Vorbis container)</b>	Edge 79+	28	25	62+ on Android	No, only VP8 in WebRTC	2.3+
<b>AV1</b>	Edge 79+	28/67+ no AV1 on Android	70+	62+ on Android	No	5.0+

# Audio Codec Support

Separate audio support may differ from audio together with video in its container. As audio sound file, WAV/MP3 are good choices

	IE/Edge	FireFox	Chrome	Opera	iOS	Android
HTML audio element	9.0+	3.5+	3.0+	10.5+	1.0+	2.0+
<b>OGG vorbis</b>	No/17+	3.5+	5.0+	11.5+	No	2.3+
<b>MP3</b>	9.0+	22+	5.0+	15+	4.0+	2.0+
<b>AAC</b>	9.0+	only in MP4!	12+	62+ on Android	4.0+	3.0+
<b>FLAC</b>	No/16+	51+ (not on KaiOS)	56+	42/12.1+	11+	5.0+
<b>WAV</b> (typically lossless PCM)	No/12+	3.5+	8+	11.5+	3.2+	2.3+
<b>Opus</b> (used mainly in WebRTC transfers)	No/14+	15+	33+	only on desktop	11+ only CAF file	No

# Multiple Media and Fallback Options

- HTML5 allows to specify multiple sources for the video and audio elements; browsers can use whichever works for them.

- Example

```
<video height="200" controls="">
 <source src="video.webm" type="video/webm"/>
 <source src="video.mp4" type="video/mp4"/>
 <source src="video.ogv" type="video/ogg"/>
</video>
```

- Some browsers that don't support these new elements. In that case, you can provide a fallback option via a browser plug-in (works mainly on desktop), like Java or Flash (or you can include any other fallback content in video/audio tag).

- ```
<video src="video.ogv" controls>  
  <object data="flvplayer.swf" type="application/x-shockwave-flash">  
    <param value="flvplayer.swf" name="movie"/>  
  </object>  
</video>
```

API Attributes

- Many of these attributes can be set via JS, and trigger events on change:
 - duration
 - currentTime
 - playbackRate
 - autoplay
 - networkState
 - paused
 - loop
 - preload
 - ended
 - controls
 - buffered
 - width
 - muted
 - readyState
 - height
 - src/currentSrc
 - volume
 - poster
- Example

```
var video = document.getElementsByTagName("video")[0];
alert(video.currentTime);
video.playbackRate = video.playbackRate * 2;
```
- <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/video#Attributes>

API Events

- **loadstart**
- progress
- suspend
- abort
- error
- emptied
- stalled
- loadedmetadata
- loadeddata
- canplay
- canplaythrough
- **playing**
- **waiting**
- seeking
- seeked
- ended
- durationchange
- timeupdate
- **play**
- **pause**
- ratechange
- volumechange

• Example

```
<div id="time">0</div>
```

```
<script>
```

```
  var video = document.getElementsByTagName("video")[0];
```

```
  video.addEventListener('timeupdate', function() {
```

```
    document.getElementById("time").innerHTML = video.currentTime;
```

```
  }, false);
```

```
</script>
```

• <http://www.whatwg.org/specs/web-apps/current-work/multipage/the-video-element.html#mediaevents>

API Functions

- Basic functions

- **load()**
- **play()**
- **pause()**
- **canPlayType(type)** - is supplied MIME type supported?
 - "maybe"/"probably"/"" (means no support)
- **addTextTrack()**

- Example

- ```
<button id="play" title="play" onclick="pv()">play</button>
<script>
function pv() {
 var video = document.getElementsByTagName("video")[0];
 video.play();
}
</script>
```

# Media Capture

(+ Image Capture API and Media Stream Recording – both only Chrome)

(Android 5.0+, iOS 11+)

See e.g.: [Media Devices](#), [Media Stream Recording API](#) (+WebRTC)  
Specification: [Media Cap. and Str.](#), [WebRTC](#) and its web site: <https://webrtc.org/>  
Tutorials: [HTML5Rocks](#), [Dictaphone on MDN](#), [Google Chrome](#) (no iOS support)  
Polyfill to compensate for browser differences: [Adapter.js](#)

# Media Stream in Browsers

- If available, we can access `navigator.mediaDevices` object
  - `.enumerateDevices()` will list available inputs & their kinds
    - Interesting are `audioinput` and `videoinput` kinds
    - We can use their `ids` as `deviceId` in constraints
  - `.getUserMedia(constrain_obj)` will try to provide you with requested inputs based on your constraints
    - Constraint object contains audio and/or video properties
      - either as e.g. `video : true`
      - or an object with suggested/exact settings:
        - `video: { width: { min: 1280 }, height: { min: 720 } }`
        - `video: { width: { exact: 640 }, height: { exact: 480 } }`
        - `audio: { deviceId: { exact: audioSelect.value } }`
      - More settings: `frameRate`, `facingMode` (e.g. `'user'`), `resizeMode`, `aspectRatio` for video; `sampleRate`, `sampleSize`, `echoCancellation`, `autoGainControl`, `noiseSuppression`, `latency`, `channelCount` for audio
      - On failure, `OverconstrainedError` can be thrown
    - Promise from `getUserMedia` `then` provides the actual stream
  - `.getDisplayMedia()` for screen capture → not implemented on mobile devices

# Media Stream Processing – photo

- The media stream can be bound to audio/video element through their **srcObject** property and also the image captured and drawn to the Canvas.

```
video = document.createElement('video'); c=document.createElement('canvas');
img = document.createElement('img'); // Everything off-screen (or use selectors)
var stream = await navigator.mediaDevices.getUserMedia({video: true});
video.srcObject = stream; video.autoplay=true /*Needed for Chrome*/;
function captureFrame() { //canvas and img variables were prepared earlier
 c.width = video.videoWidth; c.height = video.videoHeight;
 c.getContext("2d").drawImage(video, 0, 0);
 img.src = c.toDataURL("image/jpeg");
}
setTimeout(captureFrame, 5000); //Auto timer, or we could use btn.onclick
setTimeout(function() {stream.getTracks()[0].stop()}, 7500); //Stop capture
```

- On Chrome (and Opera on Android), we can also grab frames  
const track = stream.getVideoTracks()[0];  
imageCapture = new ImageCapture(track);  
imgB = await imageCapture.grabFrame(); // c.getContext("2d").drawImage(imgB, 0, 0);  
//OR: img.src = URL.createObjectURL(await imageCapture.takePhoto())
- Alternatively, it can be streamed with WebRTC (to a server)

# Media Stream Capture – audio/video

For now it only works locally on Android, and desktop Chromium-based browsers (Chrome, Mozilla, Edge 79+), Opera and in Safari Technology Preview on Desktop, **not on** iOS, so we will be brief here:

- For photo capture with `.takePhoto()`, you can adjust its resolution as it will stop video playback and capture a single still photo
  - `imageCapture.getPhotoCapabilities()`, e.g. min/max/step values for `imageHeight`, `imageWidth`, `zoom`, `exposure`, `focus`, ...
  - `imageCapture.getPhotoSettings()`, `fillLightMode` (auto/on/off), `imageHeight`, `imageWidth`, `redEyeReduction`
- For video capture, we can use `captureStream(fps)` on canvas or record chunks of video we will process later (frames), see [MDN](#), [G](#)

```
var options = { mimeType: "video/webm; codecs=vp8" };
mediaRecorder = new MediaRecorder(stream, options);
mediaRecorder.ondataavailable = handleDataAvailable;
mediaRecorder.start(); // .pause(), .resume(), .stop()
function handleDataAvailable(event) { ... event.data ... } //chunk processing
```

  - `isTypeSupported` can be called to check data format
- For audio capture, the process is similar, see e.g. [MDN](#)

# HTML5 Media Capture

(where it can't be replaced by WebRTC/MSR API)

**(iOS 6.0+, Android 3.0+, FF)**

See e.g.: <http://www.html5rocks.com/en/tutorials/getusermedia/intro/>  
<http://www.francesco.iovine.name/w3c/mediacapture/>  
<http://mobilehtml5.org/ts/?id=23>

Specification: <http://www.w3.org/TR/html-media-capture/>

# Media capture basics

- We can capture still images, audio and video
  - Extends the `<input type="file">` by specifying capture mode
  - We can already specify the required content type by setting `accept` attribute:
    - `accept='image/*'`, `accept='video/*'`, `accept='audio/*'`
  - Further we add to the input:
    - `capture` property (or capture attribute set to one of the values bellow) according to new (old specification)
      - graceful fallback should is possible: basic capture application is offered as first action, but the platform may still offer us to select the item from other external applications (even file managers),
    - Older variant: `capture=camera`, `capture=camcoder`, `capture=microphone` (works directly on Android 3.0+, it should work on iOS 6.0+, NOT on desktop browsers)
- This approach is not usable for video/audio streaming



# Media capture examples



- Image

- `<input type="file" accept="image/*" capture>`
- `<input type="file" accept="image/*" capture="camera">`
- `<input type="file" accept="image/*;capture=camera">`

- Audio

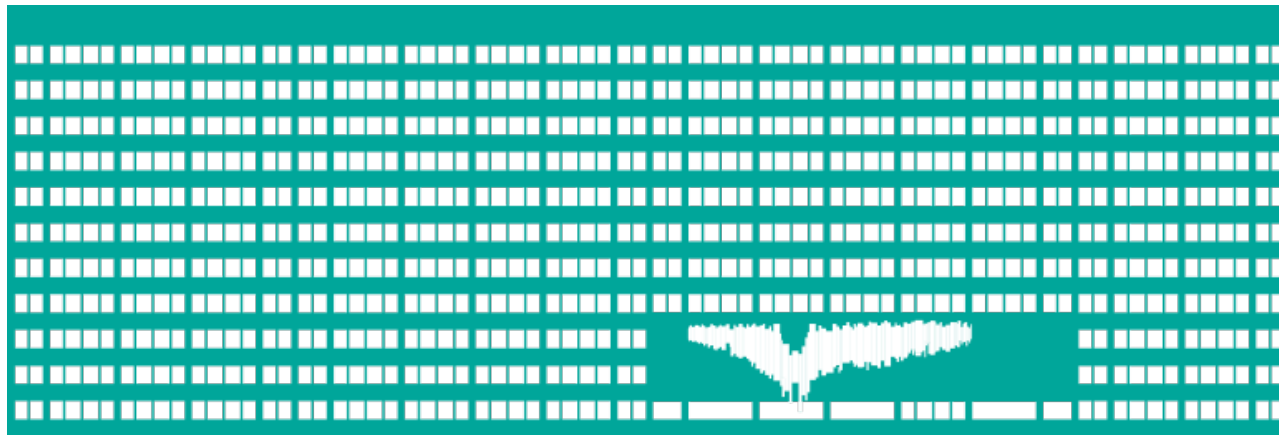
- `<input type="file" accept="audio/*" capture>`
- `<input type="file" accept="audio/*" capture="microphone">`
- `<input type="file" accept="audio/*;capture=microphone">`

- Video

- `<input type="file" accept="video/*" capture>`
- `<input type="file" accept="video/*" capture="camcoder">`
- `<input type="file" accept="video/*;capture=camcoder">`

*Note: On Android, if the browser is paused because the opened external (capture) application requires more memory and resumed on return, the page is reloaded and the file with media is not selected.*

# Orientation Apache Cordova



**TAMZ 1**  
**Lecture 8**



# HTML5 Orientation

(Android 3.0+, iOS 4.2+, KaiOS,  
has issues, considered experimental)



See e.g.:

<http://sandbox.juurlink.org/accelerometer/>

<http://ariya.github.io/device/compass/>

<http://goo.gl/uIUFYO>

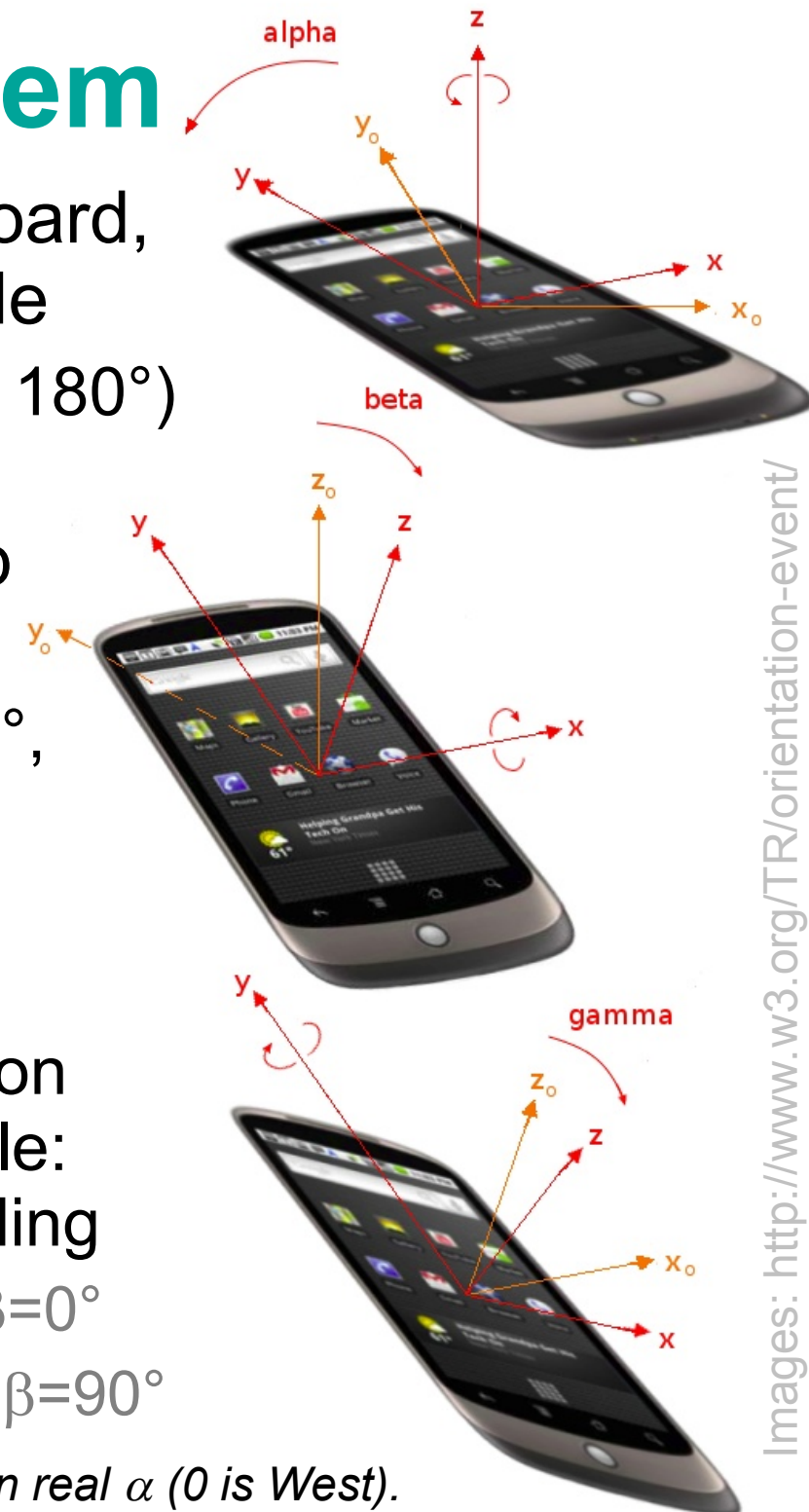
<http://www.peterfrieese.de/how-to-use-the-gyroscope-of-your-iphone-in-a-mobile-web-app/>

Specification:

<http://www.w3.org/TR/orientation-event/>

# Device coordinate system

- **x** – in the plane of screen or keyboard, positive towards the right hand side
  - **beta** ( $\beta$ ) rotation around **x**  $[-180^\circ, 180^\circ]$
- **y** – in the plane of the screen or keyboard, positive towards the top of them
  - **gamma** ( $\gamma$ ) rotation around **y**  $[-90^\circ, 90^\circ]$
- **z** – perpendicular to the screen or keyboard, positive out of them
  - **alpha** ( $\alpha$ ) counterclockwise rotation around **z**  $[0^\circ, 360^\circ]$ , absolute angle: mostly opposite of compass heading
    - horizontal position:  $360 - \alpha$  for  $\beta = 0^\circ$
    - vertical position:  $360 - (\alpha + \gamma)$  for  $\beta = 90^\circ$



*Note: Alpha angle in default Android browser is  $90^\circ$  less than real  $\alpha$  (0 is West).*

# Obtaining device position

Based on the device support, we have following events:

- **deviceorientation** – event provides current orientation angles **alpha**, **beta**, **gamma** ( $\alpha$ ,  $\beta$ ,  $\gamma$ ), or null if unknown, **absolute** indication (true/false) & method **initDeviceOrientationEvent**
  - All angles use the right-hand convention (thumb – positive axis, remaining fingers – rotation)
  - Init method args: **type**, **bubbles**, **cancellable**,  $\alpha$ ,  $\beta$ ,  $\gamma$ , **absolute**
  - **deviceorientationabsolute** for Chrome to have absolute **true**
- **devicemotion** – provides 3 separate information mentioned later, data time **interval** (if set) and **initAccelerometerEvent**
  - **acceleration** – the **x**, **y** & **z** acceleration of the device relative to the Earth frame (from gyroscope if available, without the impact of gravity) with axes defined in previous slide [0,0,0]
  - **accelerationIncludingGravity** – as above, but with gravity included (both in [m s<sup>-2</sup>]), for devices which lack gyroscope
  - **rotationRate** – device rotation rate in [deg s<sup>-1</sup>] in angles **alpha**, **beta**, **gamma**
  - Init method args: **type**, **bubbles**, **canc.**, **acc**, **accG**, **rotR**, **interval**
- **compassneeds Calibration** event is almost unsupported

# Coordinate transformations

- To transform the orientation angles column vector [**alpha**, **beta**, **gamma**] ( $[\alpha, \beta, \gamma]$ ) to a vector in WGS-84 XYZ earth frame  $[R_x, R_y, R_z]$  vector, we use following transformation matrix:

$$R = \begin{bmatrix} -\cos(\alpha) \sin(\gamma) - \sin(\alpha) \sin(\beta) \cos(\gamma) \\ -\sin(\alpha) \sin(\gamma) + \cos(\alpha) \sin(\beta) \cos(\gamma) \\ -\cos(\beta) \cos(\gamma) \end{bmatrix}$$

- To calculate compass heading (absolute angles are required for correct result), we can calculate the angle by

$$\theta = \tan^{-1} \left( \frac{R_x}{R_y} \right) = \tan^{-1} \left( \frac{-\cos(\alpha) \sin(\gamma) - \sin(\alpha) \sin(\beta) \cos(\gamma)}{-\sin(\alpha) \sin(\gamma) + \cos(\alpha) \sin(\beta) \cos(\gamma)} \right)$$

# Fixing compass heading

- Different compass rotation direction (some do not observe right-hand-rule) and zero angle for different devices:
- A good overview, shown in following table may be found at <https://github.com/ajfisher/deviceapi-normaliser>:

|                                | Zero point | Right Hand Rule | Range    |
|--------------------------------|------------|-----------------|----------|
| API Reference                  | North (0)  | Y               | [0, 360] |
| iOS Chrome/Safari              | East (90)  | Y               | [0, 360] |
| Blackberry                     | South(180) | N               | [0, 360] |
| Chrome/Opera<br>Android ICS/JB | North (0)  | Y               | [0, 360] |
| Stock browser<br>Android ICS   | West (270) | Y               | [0, 360] |
| Firefox Android<br>ICS/JB      | North (0)  | N               | [0, 360] |

- The page also contains normalizer script to fix behavior across the devices (but it is quite old)

# Additional HTML5 APIs

(Experimental, support typically on Android 5.0+ mostly in Google Chrome)

Documentation: [Vibration API](#), [USB](#), [Bluetooth](#), ...



# Not much coverage and usage yet...

- These APIs are mostly experimental and not supported outside of the desktop and Android Chrome (&Edge 79+)
- Some may have to be enabled in `chrome://flags`
- Solve issues with missing functionality:
  - Allow access to USB devices connected to phone (OTG)
    - USB API
  - Bluetooth communication via WebBluetooth
  - Web NFC (only Chrome on Android)
- Make usual functionality available:
  - Battery Status API (will be removed due to fingerprinting issues)
  - Vibration API (general Android 4.4+ support, no iOS support)
  - Sensor API
    - Gyroscope, LinearAccelerationSensor, Magnetometer
    - AmbientLightSensor
    - Absolute/RelativeOrientationSensor

# Some examples

- Vibration API – most support (but still only Android)
  - `navigator.vibrate(time_ms);`
  - Can be extended like dashed line in Canvas with a pattern:
    - `navigator.vibrate([vibrate_ms, wait_ms, vibrate2_ms]);`
  - Cancellation during vibration `navigator.vibrate(0);`
- Sensor API – gyroscope

```
let gyroscope = new Gyroscope({frequency: 60});
gyroscope.addEventListener('reading', e => {
 console.log("Angular velocity [X,Y,Z]: [" + gyroscope.x
 + ", " + gyroscope.y + ", " + gyroscope.z + "]);
});
gyroscope.start();
setTimeout(() => gyroscope.stop(), 5000);
```
- Battery – Battery API
  - `bat = await navigator.getBattery();`  
`console.log("Battery API lvl: " + (bat.level*100) + "%")`

# HTML5 Widgets

**(General specification, which is followed by Apache Cordova)**

Specifications: <http://www.w3.org/TR/widgets/>  
<http://www.w3.org/TR/widgets-access/>

# Widgets / Packaged Web Apps

- Name comes from **Window gadget**
- Defined packaging, media type, extension requirements
  - ZIP archive (+permitted variant), application/widget, .wgt
- Reserved file names, package structure definition
  - Reserved: **config.xml**, **icon**.{png|gif|jpg|ico|svg}, **locales**
    - Default start files: **index**.{html|htm}, **index**.{svg|xhtml|xht}
  - The files with reserved filenames located in root directory are the widget **configuration** document, default **icon**, *start page* and **locale** folder (default icon and start page may be located in root of locale folder, custom defined in the config.xml instead)
- Localization (internationalization) may be done with folder structure or xml:lang attribute.
  - Standard country codes are used, e.g. /locales/cs **/locales/cs-cz** or **xml:lang="en-us"**
- Configuration file contains basic settings, also origin:
  - **<access origin="http://example.org" subdomains="true"/>**
  - **<access origin="\*" />**

# Widget configuration file example

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<widget xmlns="http://www.w3.org/ns/widgets"
id="http://example.org/exampleWidget" version="0.1" height="200"
width="200" viewmodes="fullscreen">
```

```
 <name short="Example 0.1">The example Widget!</name>
```

```
 <name short="Ukázka 0.1" xml:lang="cs">Ukázkový Widget!</name>
```

```
 <feature name="http://example.com/camera" required = "true">
```

```
 <param name="autofocus" value="true"/>
```

```
 </feature>
```

```
 <preference name="apikey" value="deadbeef0075417e" readonly="true"/>
```

```
 <description>Adapted example from documentation</description>
```

```
 <description xml:lang="cs">Jen ukázka konfigurace</description>
```

```
 <author href="http://foo-bar.example.org/" email="foo-bar@example.org">
```

```
 Foo Bar Corp
```

```
 </author>
```

```
 <icon src="icons/example.png"/>
```

```
 <icon src="icons/boo.png"/>
```

```
 <content src="myWidget.html"/>
```

```
 <license>Example license (based on MIT License) ...</license>
```

```
</widget>
```

# Widget configuration file items

- Only the **widget** element is compulsory
  - All widget attributes (id, version, ...) are optional
  - Viewmodes: fullscreen, windowed, floating, maximized, minimized
  - Change of default locale ("" ) by e.g. **defaultlocale="en-us"**
- You can specify 0-\* children elements
  - name, description, license only once per language
  - author, icon, feature, preference may appear multiple times
- License tag may contain href attribute directing it to the full text of the license and the license description between tags
- Icon and content tags are used to specify custom icons and start page instead of the default files.
  - `<content src="lbg-maps.swf" type="application/x-shockwave-flash" />`
  - `<content src="olddocument.htm" encoding="iso-8859-2" />`
  - `<icon src="xxx.png" width="48" height="48"/>`
- Widget will not work without required features
- Span element may be used in config (changes xml:lang, dir)  
`<span xml:lang="en" dir="ltr">English text</span>`

# Native applications with Apache Cordova (PhoneGap)

(Android, iOS, WP, ...)

See e.g.: <https://cordova.apache.org/docs/en/latest/>  
<https://cordova.apache.org/docs/en/latest/guide/support/index.html>  
<http://netbeans.org/kb/docs/webclient/cordova-gettingstarted.html>  
Web site: <https://cordova.apache.org/> <https://cordova.apache.org/plugins/>

# Apache Cordova basics



APACHE  
CORDOVA™

- Web-oriented (HTML5+JS+CSS3)
- But it's used for building embedded applications (with installation packages instead of mobile web)
- Large scope of platforms (Android, iOS, WP, ~~Blackberry, Tizen, Ubuntu, Firefox OS, ...~~)
- Common JavaScript API over different platforms, frameworks programming languages & IDEs
- Uses features from HTML5
- But you have to prepare the basic project for deployment to individual platforms
  - Instead of building locally, it is possible to use cloud-based build (may require publically-available sources on Github)



# How to use Apache Cordova

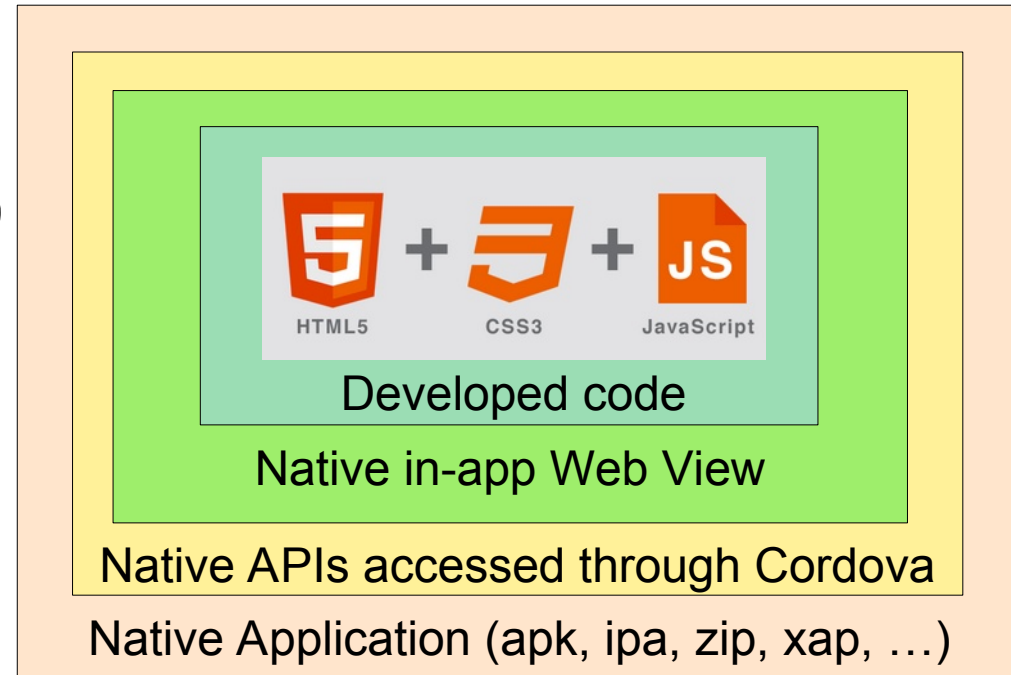
- You add Cordova libraries into your project:
  - the common platform-independent core
  - the platform-dependent API adaptation code
- You use the native IDE for given platform
  - Some IDEs, like NetBeans, offer support for seamless use of Apache Cordova
- You can sell the application using platform-dependent store (App Store, Google Play, Windows Marketplace, ...)
- It is possible to use (3<sup>rd</sup>-party) plugins
  - Plugin may be available only for some platforms, e.g. Android & iOS
  - Examples: Calendar plugin, SMS plugin, ...

# Libraries for individual platforms

List for Cordova 10.x (2020-08) supported platforms:

- Android
- iOS (develop on Mac OS)
- Windows 8.1, 10,
- MacOS
- Windows Phone 8.1
- Electron framework
- Removed in 8.x:
  - Blackberry 10, Ubuntu with QT, Windows Phone 8
- Removed in 6.x:
  - Firefox OS, Amazon Fire OS, Tizen
- Supported in past: ~~WebOS, Bada~~

Development (with limitations) on: Windows, MacOS, Linux



# Apache Cordova disadvantages

- Applications are slower than native ones
  - especially graphics-intensive applications such as games (unless we write a lot of native code)
- Not fully integrated into the platform
  - may lack the platform-specific look&feel (but you may develop different CSS-based skins for different platforms)
  - lack of pre-built UI widgets, transitions, standard controls – but you can use something else (e.g. Onsen UI 2) for this task
- For Android development you may use a plugin, for debugging `chrome://inspect` or `adb logcat` for console
  - pure-HTML parts may be debugged as usual in browser simulation (browser platform) for core plugins

# Apache Cordova project workflows

- Native platform (the original one)
  - We have common platform-independent sources in HTML, JS & CSS, but the project is created for specific platform
  - Needs plugman for each platform
  - Hard to make multi-platform projects
  - Easier to change the lower-level platform details
  - Suitable, when the CordovaWebView is a small part in a larger native application
- Web project (available since Cordova 3.0)
  - A set of wrapper scripts, CLI (esp. cordova command)
  - Multi-platform, easy to add/remove platforms
    - As many mobile operating systems as possible
  - As little platform-specific development work as possible

# Installing & using Apache Cordova

- Installation requires Node.js framework
  - Node.js Installation files are at <http://nodejs.org/download/> or in your distribution (external) repositories (Linux)
- Cordova installation uses Node.js package manager:
  - `npm install -g cordova` #(or: `sudo npm install -g cordova`)
  - Use `update` instead of `install` to upgrade the installation
- We can also install plugman – Plugin Manager
  - `npm install -g plugman` #(or: `sudo npm install -g plugman`)
- Cordova is integrated in Netbeans, but you can also use Cordova CLI (first creates project dir, use the rest in it):
  - `cordova create <project_PATH> [ID [NAME [CONFIG]]]`
    - e.g. `cordova create example cz.vsb.mor03.Ex Example`
  - `cordova platform [{add|remove|update} <PLATFORM> | ls]`
    - e.g. `cordova platform add android`
  - `cordova plugin [{add|remove} <PATH|URI> | ls | search kw]`
    - e.g. `cordova plugin add org.apache.cordova.device`
  - `cordova {build|run|emulate|serve} [PLATFORM]`

# Apache Cordova config.xml

- The configuration file is placed in the main web project directory and/or in a subdirectory based on given platform
- Uses Widget specification mentioned earlier
  - extra namespace xmlns:cdv="<http://cordova.apache.org/ns/1.0>" for widget tag
- The supplied configuration preference may also
  - add fullscreen setting:  
`<preference name="fullscreen" value="true"/>`
  - enforce screen orientation:  
`<preference name="Orientation" value="landscape" />`
    - Values: **default** (both orientations), landscape, portrait
  - disable rubber-band scrolling bounce:  
`<preference name="webviewbounce" value="true"/>`
  - be set to prevent overscrolling on iOS & Android:  
`<preference name="DisallowOverscroll" value="true"/>`
- Some platform-specific preferences may be also included
- Since Cordova 7.x **package.json** is generated with plugin/pl. list, in future may fully replace config.xml (until 9.x it was synchronized from config.xml)

# Executing native code

Apache Cordova offers a system of Native Plugins

- There is a set of basic (core) plugins which should be available for all/most platforms.
- You can create a wrapper JavaScript class to provide missing/additional API.
- The native calls are executed by `cordova.exec()`
  - e.g. `cordova.exec(function(winParam) {}, function(error) {}, "service", "action", ["firstArg", "secondArg", 42, false]);`
- Plugin repository has a `plugin.xml` file in top directory, which describes the plugin
  - platforms, their source files and configuration, ...
- The native code differs platform-to-platform, e.g. Android
  - extends `CordovaPlugin`
  - overrides method: `public boolean execute(String action, JSONArray args, CallbackContext cCont) throws JSONException`
- You can publish your plugins for others via plugman
  - You have to create an account first.

# Apache Cordova core + plugins

- Battery Status
- Camera
- Connection (was Network Information)
- *Console (integrated)*
- Contacts (works so far, deprecated in 8.x)
- Device
- Device Motion (works, deprecated in 8.x)
- Device Orientation (works, deprecated in 8.x)
- *Events (integrated)*
- *File (System)*
- *Geolocation*
- Globalization
- In-App Browser
- Media
- *Notification*
- Splashscreen
- Status Bar
- *Storage (integrated)*
- Vibration



# Core plugins (A-F)

- *Battery status* – support for battery state
- *Camera* – get a picture from Camera
- *Connection (was Network Information)* – access to the device's cellular and WiFi connection information.
- *Contacts* – access contacts in device database (search and adding)
- *Device* – provides information about device HW, SW, UUID, manufacturer, ...
- *Device Motion (Accelerometer)* – returns x,y,z acceleration, being replaced by **devicemotion**
- *Device Orientation (Compass)* – current compass heading or watching the heading, **being replaced**
- *File (System)* – provides W3C File API to read, write and navigate file system hierarchies.
- *File Transfer* – old way of file download and upload

# Core plugins (G-Z)

- *Geolocation* – access to the device's location provider (GPS sensor).
- *Globalization* – information & operations specific to the user's locale and timezone.
- *InAppBrowser* – web-browser after the use of `window.open`
- *Media* – recording and playback of audio files
- *Media Capture* – capture audio, video or image with external application
- *Notification (has been Dialogs)* – visual and audible (sound) device notifications.
- *Splashscreen* – show&hide the application's splash screen.
- *Status Bar* – controls visibility and look&feel of status bar.
- *Vibration* – haptic (tactile) feedback.

# Built-in features, other plugins

Built-in features:

- *Console* – logging to device's console (log), changes the way how console.\* work
- *Storage* – local storage and SQL database W3C "Web SQL Database", Indexed DB & Web Store API Specification
- *Lifecycle Events* – HW buttons and changes in state
  - deviceready, pause, resume
  - backbutton, menubutton, searchbutton, startcallbutton, endcallbutton
  - volumedownbutton, volumeupbutton

Additional plugins can be found at

<https://cordova.apache.org/plugins/>

# Support matrix (5.4.0)

	amazon-fireos	android	blackberry10	Firefox OS	ios	Ubuntu	wp8 (Windows Phone 8)	windows (8.0, 8.1, 10, Phone 8.1)	tizen
<b>cordova CLI</b>	✓ Mac, Windows, Linux	✓ Mac, Windows, Linux	✓ Mac, Windows	✓ Mac, Windows, Linux	✓ Mac	✓ Ubuntu	✓ Windows	✓	✗
<b>Embedded WebView</b>	✓ (see details)	✓ (see details)	✗	✗	✓ (see details)	✓	✗	✗	✗
<b>Plug-In Interface</b>	✓ (see details)	✓ (see details)	✓ (see details)	✗	✓ (see details)	✓	✓ (see details)	✓	✗
	Platform APIs								
<b>Accelerometer</b>	✓	✓	✓	✓	✓	✓	✓	✓	✓
<b>BatteryStatus</b>	✓	✓	✓	✓	✓	✗	✓	✓ * Windows Phone 8.1 only	✓
<b>Camera</b>	✓	✓	✓	✓	✓	✓	✓	✓	✓
<b>Capture</b>	✓	✓	✓	✗	✓	✓	✓	✓	✗
<b>Compass</b>	✓	✓	✓	✗	✓ (3GS+)	✓	✓	✓	✓
<b>Connection</b>	✓	✓	✓	✗	✓	✓	✓	✓	✓
<b>Contacts</b>	✓	✓	✓	✓	✓	✓	✓	partially	✗
<b>Device</b>	✓	✓	✓	✓	✓	✓	✓	✓	✓
<b>Events</b>	✓	✓	✓	✗	✓	✓	✓	✓	✓
<b>File</b>	✓	✓	✓	✗	✓	✓	✓	✓	✗
<b>File Transfer</b>	✓	✓	✓ * Do not support onprogress nor abort	✗	✓	✗	✓ * Do not support onprogress nor abort	✓ * Do not support onprogress nor abort	✗
<b>Geolocation</b>	✓	✓	✓	✓	✓	✓	✓	✓	✓
<b>Globalization</b>	✓	✓	✓	✗	✓	✓	✓	✓	✗
<b>InAppBrowser</b>	✓	✓	✓	✗	✓	✓	✓	uses iframe	✗
<b>Media</b>	✓	✓	✓	✗	✓	✓	✓	✓	✓
<b>Notification</b>	✓	✓	✓	✗	✓	✓	✓	✓	✓
<b>Splashscreen</b>	✓	✓	✓	✗	✓	✓	✓	✓	✗
<b>Status Bar</b>	✗	✓	✗	✗	✓	✗	✓	✓ Windows Phone 8.1 only	✗
<b>Storage</b>	✓	✓	✓	✗	✓	✓	✓ localStorage & indexedDB	✓ localStorage & indexedDB	✓
<b>Vibration</b>	✓	✓	✓	✓	✓	✗	✓	✓ * Windows Phone 8.1 only	✗

New in 5.x →

# Support matrix (9.0+)

Platform:	Android	iOS	OS X	Windows 8.1, Phone 8.1, 10	Electron
CLI shorthand:	android	ios	osx	windows	electron
	Cordova CLI Development Platform				
Mac	✓	✓	✓	✗	✓
Windows	✓	✗	✗	✓	✓
Linux	✓	✗	✗	✗	✓
	Core Plugin APIs				
BatteryStatus	✓	✓	✗	✓ Windows Phone 8.1 only	Tests Pending
Camera	✓	✓	✗	✓	✓
Capture	✓	✓	✗	✓	Tests Pending
Connection	✓	✓	✗	✓	Tests Pending
Device	✓	✓	✓	✓	Tests Pending
Events	✓	✓	✗	✓	Tests Pending
File	✓	✓	✓	✓	Tests Pending
Geolocation	✓	✓	✗	✓	Tests Pending
Globalization	✓	✓	✗	✓	Tests Pending
InAppBrowser	✓	✓	✗	uses iframe	Tests Pending
Media	✓	✓		✓	Tests Pending
<u>Notification</u>	✓	✓	✗	✓	Tests Pending
Splashscreen	✓	✓	✗	✓	Tests Pending
Status Bar	✓	✓	✗	✓ WP 8.1 only	Tests Pending
Storage	✓	✓	✗	✓ localStorage & indexedDB	Tests Pending
Vibration	✓	✓	✗	✓ WP 8.1 only	✗
	Platform Features				
Plugin Interface	✓ (see details)	✓ (see details)	✓	✓	Tests Pending
Embedded WebView	✓ (see details)	✓ (see details)	✓	✗	Tests Pending

# Information plugins

- Battery Status Events

(<http://www.w3.org/TR/2011/WD-battery-status-20110915/>)

- Defines 3 events on window: `batterystatus`, `batterycritical`, `batterylow`. The event for each event object contains:

- `level` (0-100) and `isPlugged` (true/false) – is charging?

- Connection (Network information)

(<http://www.w3.org/TR/2011/WD-netinfo-api-20110607/>)

- Adds navigator.connection object with network `type`  
`connection.type: Connection.UNKNOWN, .ETHERNET, .WIFI, .CELL_LL_2G, .CELL_3G, .CELL_4G, .CELL, .NONE`

- Device – adds a custom `device` object

- available after `deviceready` event
  - properties: `device.cordova` (Cordova version), `.model` (e.g. 9800, iPhone 5,1, ...), `.platform` (e.g. Android), `.uuid` (in most cases NOT IMEI), `.version` (of device OS, e.g. 4.3, 3.2)

- Dialogs – adds navigator.notification object (native dialogs)

- `notification.alert`, `notification.confirm`, `notification.prompt`
  - `navigator.notification.beep(times)` – sound feedback

# Camera plugin

Camera API – get still image via `navigator.camera.getPicture` (`cameraSuccess`, `cameraError`, [ `cameraOptions` ]) as a string

- Options passed in in `cameraOptions` object:
  - `quality` (0-100, 100 – best, e.g. JPEG quality)
  - `destinationType` (0: base-64 string, 1: file://, 2: native URI)
  - `sourceType` (0: photo library, 1: camera, 2: saved photo)
  - `allowEdit` (simple image editing, e.g. cropping if true)
  - `encodingType` (0: JPEG, 1: PNG)
  - `targetWidth/targetHeight` (scale to output, keep aspect r.)
  - `mediaType` (for sources from photo library and saved photos, 0:picture, 1: video, 2: all media)
  - `correctOrientation` (auto-rotate image according to device position when captured?)
  - `saveToPhotoAlbum` (store captured image),
  - `cameraDirection` (0: back, 1: front in iOS)
- Success: Based on destination type, the resulting `imageData` is a string with file URL (file:// or native like capture://) or base-64 string with image data of encoding Type
- Error: error object with `message` property is returned

# Media Capture plugin

- Media Capture API – adds **capture** object to record media through external file(s), stored on file system
  - Properties: arrays of **ConfigurationData**: **.supportedAudioModes**, **.supportedImageModes**, **.supportedVideoModes**
  - Methods: **capture.captureAudio/.captureImage/.captureVideo** (**successCB**, **errorCB** [, options])
  - Success: we get a **MediaFile** instance, which provides:
    - **name** (without path), **fullPath** (incl. name), **type** (mime type), **lastModifiedDate**, **size** (in bytes) and **getFormatData()**
  - Error: we get error object with a pre-defined **code**:
    - **CaptureError.CAPTURE\_INTERNAL\_ERR**
    - **CaptureError.CAPTURE\_APPLICATION\_BUSY**
    - **CaptureError.CAPTURE\_INVALID\_ARGUMENT**
    - **CaptureError.CAPTURE\_NO\_MEDIA\_FILES**
    - **CaptureError.CAPTURE\_NOT\_SUPPORTED**
  - Options: **limit** – the number of captured items of given type, audio+video: **duration** – maximum length of a clip in [s]
- Media plugin – uses W3C media & media capture described in last lecture. Does not adhere fully for capture, will be reworked



# Some frequently used utility plugins

- Vibration – adds `navigator.notification.vibrate([time_ms])`
- Status bar – changes status bar behavior by `StatusBar` o.
  - Methods: `overlaysWebView`, `backgroundColorByHexString`
  - `show()`, `hide()`, `statusTap` event to detect tapping, `isVisible`
- SplashScreen – adds `splashscreen` object
  - Methods: `show()`, `hide()`
  - `config.xml`
    - `<preference name="splashscreen" value="file_name" />`
    - `AutoHideSplashScreen` preference (true/false)
- InAppBrowser – a browser window created by `window.open()`, which is part of Cordova application, but does not have access to Apache Cordova
  - e.g. `ref=window.open('http://apache.org', '_blank', 'location=yes');`
- File – implements HTML5 FileSystem API inside of the application (<http://www.w3.org/TR/file-system-api/>)

# Accelerometer & Compass plugins

In both cases, the API copies geolocation API approach:

- Acceleration – adds navigator.**accelerometer**
  - Methods: **getCurrentAcceleration**(accSuccess, accError), **id = watchAcceleration**(accSuccess, accError[, **opts**]), **clearWatch**(id)
  - Success: we get acceleration object with **x**, **y**, **z** including the effect of gravity (e.g. 0, 0, 9.81) and **timestamp** field
  - Options: we can specify freq. in [ms], e.g. { **frequency: 3000** }
- Compass – compass heading through navigator.**compass**
  - Methods: **getCurrentHeading**(compassSuccess, compError), **id=watchHeading**(compassSuccess, compassError[, **opts**]), **clearWatch**(id)
  - Success: we get heading object with **magneticHeading**, **trueHeading** (geographic to the North Pole, negative - can't be determined), **headingAccuracy** (deviation in [°] between reported and the true heading) and **timestamp** fields
  - Error: we get error object with **code** being
    - **CompassError.COMPASS\_INTERNAL\_ERR**
    - **CompassError.COMPASS\_NOT\_SUPPORTED**
  - Options: we can specify frequency (see above) and filter (iOS)

# Contacts Plugin – classes

Following classes are defined in contacts plugin:

- **Contact** – contains instances of **ContactName** – **name**, **ContactField[]**, **ContactOrganization[]** – organizations, **ContactAddress[]** – addresses, and other properties:
  - **id** (globally unique), **displayName** (end user-friendly name), **nickname** (casual name), **birthday**, **note**
    - **ContactField[]** based: **phoneNumbers**, **emails**, **ims** (IM addresses), **photos**, **categories**, **urls**
- **ContactField** – contains **type**, **value** & **preferred** (true/false)
- **ContactName** – **formatted**, **familyName**, **givenName** (Jim), **middleName**, **honorificPrefix** (Dr.), **honorificSuffix** (Esq.)
- **ContactAddress** – **pref** (true/false), **type** (e.g. home, work), **formatted**, **streetAddress**, **locality** (e.g. Ostrava), **region** (e.g. Alabama), **postalCode**, **country**
- **ContactOrganization** – **pref**, **type** (e.g. home), **name**, **department**, **title** (of contact in organization)
- **ContactFindOptions** – **filter** and **multiple** (returns all finds)
- **ContactError** – **code** value is one of the pre-set error codes

# Contacts Plugin – methods

- A new contacts property is added to navigator object (navigator.contacts)
  - It has two methods – **contacts.create** & **contacts.find**
    - `var contact = navigator.contacts.create(Contact);`
      - A new local contact object, not yet saved on device
    - `navigator.contacts.find(contactFields, contactSuccess, contactError, contactFindOptions)` – asynchronous search:
      - `var options = new ContactFindOptions();`  
`options.filter = "John"; options.multiple = true;`  
`var fields = ["name", "displayName"];`  
`navigator.contacts.find(fields, onSuccess, onError, options);`
  - **Contact** object has following methods:
    - **clone** – returns a new, copied, Contact object (deep copy, id is set to null)
    - **remove** – removes the contact from the device contacts database, error callback if the contact is not present
    - **save** – inserts a new contact or updates an existing contact in device database

# Interesting 3<sup>rd</sup> party plugins for Apache Cordova 3.x

- SMS plugin – <https://github.com/aharris88/phonegap-sms-plugin>
  - Allows to send SMS directly without using sms:// URL
- Calendar Plugin (Android & iOS only)  
<https://github.com/EddyVerbruggen/Calendar-PhoneGap-Plugin>
- Google Maps Plugin (Android & iOS with GM SDK use only)  
<https://github.com/wf9a5m75/phonegap-googlemaps-plugin>
- Barcode Scanner (Android & iOS only)  
<http://plugins.cordova.io/#/package/com.mirasense.scanditsdk.plugin>
  - WP8 alternative  
<http://plugins.cordova.io/#/package/org.bloxlabs.barcodescanner>
  - Full JS alternative (no Cordova required, slower)  
<https://github.com/LazarSoft/jsqrcode>

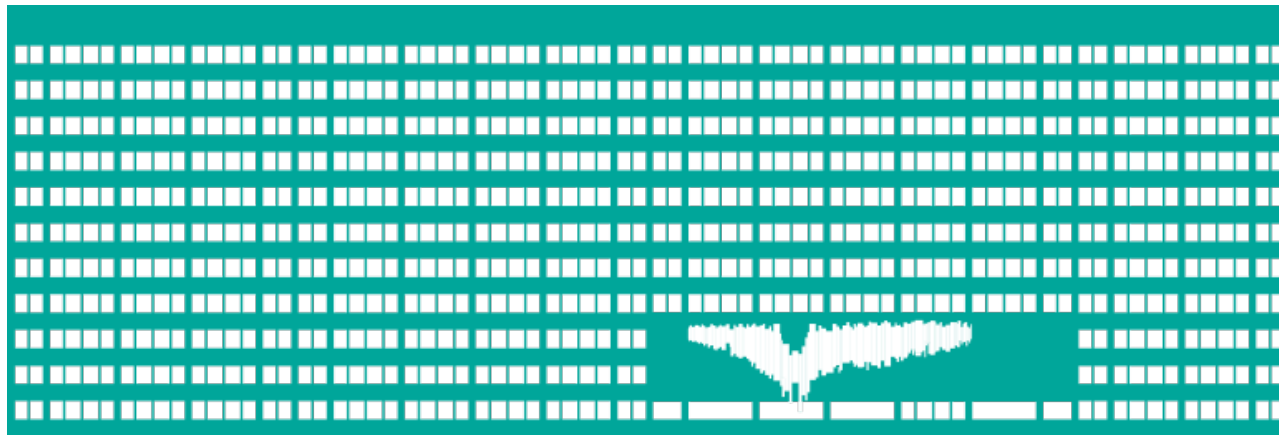
# Interesting 3<sup>rd</sup> party communication plugins for Apache Cordova 3.x

- Bluetooth SPP plugin – <https://github.com/don/BluetoothSerial>
  - Allows to communicate through Bluetooth with standard BT devices emulating serial port through serial port profile
- Bluetooth Low Energy (BLE) Plugins
  - Android 4.3+, iOS 7+ - <https://github.com/don/cordova-plugin-ble-central>
  - Mainly for HRM, not fully tested. Android 4.3+, iOS 7+, W. Phone 8.1- <https://github.com/randdusing/BluetoothLE>
- NFC Plugin - <https://github.com/chariotsolutions/phonegap-nfc>
  - Support for basic NFC tags and communication on Android, Windows Phone and Blackberry together with NDEF messages (encrypted tags are not supported)

# Interesting 3<sup>rd</sup> party communication plugins for Apache Cordova 3.x

- Socket-based communication (TCP)
  - Sockets for Cordova – [www.blocshop.cz/blog/?p=6](http://www.blocshop.cz/blog/?p=6)
    - socket-based communication for Android/iOS/WP8
    - <https://github.com/blocshop/sockets-for-cordova>
  - Tlantic cdv-socket-plugin Plugins
    - Android/iOS/WP8
    - <https://github.com/Tlantic/cdv-socket-plugin>
- Push notifications (platform-specific)
  - Cordova Push Notifications Plugin
    - Android/Fire OS, iOS, WP8, Windows8, BlackBerry 10
    - <https://github.com/zckrs/cordova-plugin-push-notification>
- Background execution plugins
  - cordova-plugin-background-app – Android, iOS
  - com-badrit-backgroundjs – Android, iOS
  - ...

# Storage alternatives (SQL, IDB, VFS)



**TAMZ 1**  
**Lecture 9**



# Extended storage of user data

- Legacy mobile platforms provided only basic record-oriented data storage (similar to Web Storage)
- Current platforms offer several ways how to store data:
  - Preferences (key/value pairs) – closest to the legacy record-oriented storage & Web Storage
  - Database – structured data in (private) database
    - Relational: Most current platforms use SQL & SQLite DB
    - Object database: stores objects directly
  - XML file – may be also internally used for preferences
  - Internal/Isolated storage – application-specific private storage space, typically inaccessible by other means
  - External storage – typically in external memory (SD card, emulated SD card), public storage space, accessible by user and other applications
  - Cloud storage – data/files stored in network, possible offline copies

# Extended storage of data in HTML5

Several specifications, many are not part of core HTML5

- Preferences (key/value pairs)
  - Web DOM Storage
  - Widget preferences in config.xml
- Database – structured data in (private) database
  - Web SQL Database – unmaintained, work on it stopped
    - Main reason: everybody was using SQLite database
    - Supported on many platforms
  - Indexed Database API (only Android 4.4+ & WP/MSIE 10)
    - New specification, almost no mobile support
    - Polyfill for Web SQL exists (not the other way around)
- Internal/Isolated storage
  - File API: Directories and System – not supported on many platforms directly in browsers, supported in Apache Cordova (File/File System plugin for main platforms)
- External storage
  - File API (read-only), File API: Writer (limited support even in Apache Cordova – only Android and iOS )

# Web SQL Database

**(Unmaintained, Android, iOS < 13)**

See e.g.: [http://www.tutorialspoint.com/html5/html5\\_web\\_sql.htm](http://www.tutorialspoint.com/html5/html5_web_sql.htm)  
<http://www.html5rocks.com/en/tutorials/webdatabase/todo/>  
<http://html5doctor.com/introducing-web-sql-databases/>  
<http://html5demos.com/database-rollback>

Specification: <http://www.w3.org/TR/webdatabase/>

# Web SQL Database basics

- To allow incremental updates between different database schemas, each database has a version (double)
- Databases observe same-origin policy (and ~5MB “limit”)
  - Web pages on the same host have access to all databases for the given origin → risk of information leakage/spoofing
- We have to open the database first
  - There is a database with the same name from same origin but with different version → `INVALID_STATE_ERR`
  - The page is prevented to open DB → `SECURITY_ERR`
- Databases use standard SQL syntax
  - The `?` character in SQL query is a placeholder for query parameter on the same position (to prevent SQL injection)
  - Standard (core) SQL statements
    - DDL: `CREATE TABLE`, `DROP TABLE`, `ALTER TABLE`
    - DML: `INSERT`, `DELETE`, `UPDATE`, `SELECT`, ...
- It is possible to use WebSQL database in a web worker
  - Synchronous DB access is also possible

# Web SQL JavaScript interface

- Window and WorkerUtils objects provide a method to open & access the database:
  - Database `openDatabase(name, version, displayName, estimatedSize [, creationCallback]);`
    - name, version – database name & version (revision)
    - displayName – description of the database
    - estimatedSize – suggest an initial quota to the use, allocate the resources, let the user know beforehand
    - creationCallback – if the database does not exist yet...  
e.g. `db=openDatabase('mydb', '1.0', 'my db example', 4*1024*1024);`
  - DatabaseSync `openDatabaseSync(...)` – synchronous DB access in web workers
- The Database/DatabaseSync object (`db`) have read-only attribute `version` (see above) and following methods:
  - `changeVersion(oldV, newV [, CB [,errorCB [,successCB]]]);`
  - `transaction(CB [,errorCB [,successCB]]);` – R/W transaction
  - `readTransaction(CB [,errorCB [,successCB]]);` – R/O trans.

# Web SQL transactions

- Database transaction – *unit of work treated coherent and reliable way independent of other transactions. It must either complete entirety or have no effect whatsoever.*
  - The callback (CB) is provided the SQLTransaction object (**t**) on which we are going to execute the queries
  - The error callback (**errorCB**) is provided error object (with code & message) to provide basic error feedback.
  - Success callback does not get any arguments
- The SQLTransaction passed to the callback has method
  - **t.executeSql(sqlStatement [, arguments] [, CB] [, errorCB]);**
    - **sqlStatement** – the actual SQL query
    - **arguments** – array for replacing each ? in SQL statement with the argument on same position (in SQL ↔ in array)
    - CB is the callback for query result – it is given the transaction object and SQL **resultSet**, containing:
      - **insertId** – ID of (last) inserted row, error otherwise
      - **rowsAffected** – number of rows that were changed
      - **rows** – list of result rows with **length** and **item(idx)** method

# Web SQL examples

- Create two tables in a single transaction

```
db.transaction(function (tx) {
 tx.executeSql('CREATE TABLE IF NOT EXISTS foo (id unique, text)');
 tx.executeSql('CREATE TABLE IF NOT EXISTS bar(rec INTEGER
PRIMARY KEY ASC)');
});
```

- Insert data with parametrized query

```
tx.executeSql('INSERT INTO foo (id, text) VALUES (?, ?)', [id, val]);
```

- Insert new record or replace existing record

```
tx.executeSql('INSERT OR REPLACE INTO foo VALUES(1,"rab");');
```

- Create an index on existing table for 1 or more attributes

```
tx.executeSql('CREATE INDEX IF NOT EXISTS index1 ON foo (text);');
```

- Number of records matching the query

```
tx.executeSql('SELECT count(*) AS lines FROM foo;', [], processCount);
```

- Simple select

```
tx.executeSql('SELECT * FROM foo', [], function (tx, results) {
 var len = results.rows.length, i;
 for (i = 0; i < len; i++) { $("#r").append(results.rows.item(i).text + '
');
 }
});
```

# Indexed Database API

## (Android 4.4, WP)

See e.g.: <http://code.tutsplus.com/tutorials/working-with-indexeddb--net-34673>  
[https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB\\_API/Using\\_IndexedDB](https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API/Using_IndexedDB)  
<http://www.codeproject.com/Articles/325135/Getting-Started-with-IndexedDB>  
<http://www.html5rocks.com/en/tutorials/indexeddb/todo/>

Polyfill: <http://nparashuram.com/IndexedDBShim/>

Specification: <http://www.w3.org/TR/IndexedDB/>



# Indexed Database API basics

- To allow incremental updates between different database schemas, each database has a version (64 bit), default: 1
- Databases observe same-origin policy (and ~5MB “limit”)
  - Web pages on the same host have access to all databases for the given origin → risk of information leakage/spoofing
- We can check for API presence in window object:  
if("indexedDB" in window) {...}
- We have to open the database first
  - We register event listeners to process
    - Update requests, Error states, successful queries
- Databases is object database storing key-value pairs
  - it is built on a transactional database model, less powerful than WebSQL, does not use SQL language (object stores)
  - we may define additional indexes
- It is possible to use Indexed database in a web worker
  - Synchronous DB access is defined but not implemented

# IndexedDB basic concepts

- **IDBKeyRange** may be used when selecting data, providing **lower** and **upper** bounds and if the interval is open – **lowerOpen**, **upperOpen** (all are read-only)
  - The range is provided by **IDBKeyRange**.**only**(val), **lowerBound**(lower [, open]), **upperBound**(upper [, open]), and **bound**(lower, upper [, lowerOpen, upperOpen])
- **Cursors** iterate over records – the directions are "next", "nextunique", "prev", "prevunique"
- Requests are either "pending" or "done"
- The keys in database may be auto-generated & returned from put request.
- Following **options** may be set for the object store:
  - **autoIncrement** – key is automatically generated as next id
  - **unique** – unique keys, insertion with the same key → error
  - **multiEntry**; **keyPath** – what is going to serve as the key
- Comparison: Arrays > Strings > Dates > Numbers

# IndexedDB – factory & transaction

- Window and WorkerUtils objects provide indexedDB obj.:
  - Methods: `r=indexedDB.open(name [, version])` – open IDB store, `.deleteDatabase(name)`, `.cmp(a,b)` –  $a < b$ : -1, 0, +1
- Once the IDB is opened, we may use the result (`r`) to assign the event handlers `onsuccess`, `onerror`, `onblocked` – we have other open db connections, `onupgradeneeded`)
  - Properties: `result`, `error`, `source`, `transaction` & `readyState`
  - **Upgrade needed** – database does not exist or has old ver.
    - The only place where we can create a new object store by `event.target.result.createObjectStore(name, options)`
    - The event has extra properties: `oldVersion` & `newVersion`
- The `transaction` (IDBTransaction) has following features:
  - Attributes: `mode` (readonly, readwrite & versionchange), `db` reference to database, `error`
  - Event handlers: `onabort`, `oncomplete` & `onerror`
  - Methods: `abort()`, `objectStore(name)` → `IDBObjectStore`

# IndexedDB database

- IDBDatabase has following read-only attributes:
  - **name**, **version**, **objectStoreNames** – list of object stores
- We can register following event handlers: **onabort**, **onversionchange** & **onerror**
- Besides the read-only attributes and events the database has a set of methods:
  - **createObjectStore**(**name** [, options]) → IDBObjectStore
  - **deleteObjectStore**(**name**);
  - **transaction**(**storeNames** [, **mode**]) → IDBTransaction – provide a list of store names and create a transaction with given mode (default is "readonly") to work with the database
    - **Modes**: "readonly", "readwrite", "versionchange"
  - **close**() - close object store

# IndexedDB object store

- IDBObjectStore has following read-only attributes:
  - `name`, `keyPath`, `indexNames`, `transaction`, `autoIncrement`
- Besides the read-only attributes, object store has a set of methods to provide the database functions
  - Methods returning IDBResult (`range` may be used instead of `key` where applicable):
    - `add(value [, key])` – adds a new record into database
    - `put(value [, key])` – updates/adds a record
    - `delete(key)` – remove entries, `clear()` – removes all entries
    - `count([key])` – number of records in object store
    - `get(key)` – retrieves data from object store
    - `openCursor([range] [, direction])` – creates `cursor` to iterate
    - read-only transaction → all changes cause `ReadOnlyError`
  - Methods working with indexes:
    - `idx = createIndex(name, keyPath[, parameters])`
      - `parameters` – object may set `unique` or `multiEntry` true
    - `idx = index(name), deleteIndex(name)`

# IndexedDB indexes and cursors

- The index (IDBIndex) has following features:
  - Attributes: `name`, `keyPath`, `objectStore`, `unique`, `multiEntry`
  - Methods (all returning IDBRequest):
    - `openCursor([range] [,direction])`
    - `openKeyCursor([range] [,direction])` – organized by index
    - `count([key])`, `get(key)`
    - `getKey(key)` – gets the key from referenced entry
- The cursor (IDBCursor) has following features:
  - Attributes: `source`, `direction`, `key`, `primaryKey`,
  - Methods returning IDBRequest: `delete()`, `update(value)`
  - Other methods:
    - `advance(count)` – move count records in given direction
    - `continue([key])` – move to the next position in given direction (which matches `key`)

# IndexedDB example

```
if("indexedDB" in window) { idbSupported = true; }
var db;
if(idbSupported) {
 var r = indexedDB.open("example",1);

 r.onupgradeneeded = function(e) {
 var myDB = e.target.result;
 if(!myDB.objectStoreNames.contains("students")) {
 var os = myDB.createObjectStore("students", {keypath: "login"});
 os.createIndex("name", "name" /*, { unique: false }*/);
 os.createIndex("email", "email", { unique: true });
 }
 }
 r.onsuccess = function(e) {
 db = e.target.result;
 var tr = db.transaction(["students"],"readwrite")
 var students = tr.objectStore("students");
 students.add({name: "Jan Novák", email: 'xxx9999@vsb.cz', login:
'xxx999', gender: 'M'});
 }
}
```

# Indexed DB Version 2

- Only on iOS 10+ and Android 5.0+
  - Allows to change store name during upgrade
  - Adds `onclose` handler on database (`db`)
  - Allows indexing on keys which contain binary data
  - Adds `IDBTransaction.objectStoreNames` array to access all object store names used in the transaction
  - Adds “missing” `getKey(query)` to `IDBObjectStore` as well
  - Also adds `IDBObjectStore.openKeyCursor(range, direction)`
  - Adds new methods `getAll/getAllKeys(range, count)` to `IDBObjectStore` and `IDBIndex`
  - `IDBCursor.continuePrimaryKey(key, primaryKey)` can be used to resume iteration of a cursor closed in previous iteration
  - `IDBKeyRange.includes(key)` can be used to check if the key is inside of the specified range



# File API

**(Cordova; basic R/O support:  
Android 3.0+, iOS 6.0+, WP8)**

See e.g.: <http://www.html5rocks.com/en/tutorials/file/dndfiles/>  
<http://www.html5rocks.com/en/tutorials/file/filesystem/>  
<https://developer.mozilla.org/en-US/docs/Web/API/DirectoryReader>  
<http://goo.gl/CYu3I> <http://goo.gl/uP9FY>

Specifications: <http://www.w3.org/TR/FileAPI/>  
<http://www.w3.org/TR/file-system-api/>  
<http://www.w3.org/TR/file-writer-api/>

# Basic File System Support

- For a long time, we may upload files to server with `<input type="file">` (and we may use Media Capture API)
  - Access to files selected in browser is granted to JavaScript through File API (which also provides blob: URL)
- Accessing (sandboxed) file system or isolated storage in JavaScript programmatically (not possible in current mobile browsers, possible in Apache Cordova)
  - File API: Directories and System (File System API)
- Writing files back to the device directly (same limitations)
  - File API: Writer
  - We can generate files and download them from server
  - Is there a way to generate a file by JavaScript and prompt the user to download it?
- Apache Cordova: we can specify if we want to use internal storage/library or filesystem access (default) by `<preference name="AndroidPersistentFileLocation" value="Internal" />`  
`<preference name="iosPersistentFileLocation" value="Library" />`

# File API basics

- The File input (<input type="file">) has a **FileList** instance **files** with **length** and **item(index)** which returns given **File**
  - `var file = document.forms['u1']['fileChooser'].files[0];`  
`var file = document.forms['u1']['fileChooser'].files.item(0);`
- The API works with **Blob** objects representing binary data
  - Blob represents immutable RAW data, snapshot state
  - Properties: **size**, **type** (MIME type)
  - Methods: **slice**([start [,end]] [,contentType]) – clone, **close**()
  - Constructor: `new Blob(blobparts, options);`  
examples from specification:
    - `var buffer = new ArrayBuffer(1024);`  
`var shorts = new Uint16Array(buffer, 512, 128);`  
`var bytes = new Uint8Array(buffer, shorts.byteOffset + shorts.byteLength);`  
`var b = new Blob(["foobazetcetc" + "birdiebirdieboo"], {type: "text/plain;charset=UTF-8"});`
    - `var c = new Blob([b, shorts]);`
    - `var d = new Blob([buffer, b, c, bytes]);`

# File API: File & FileReader

- The **File** is a **Blob** with following extensions
  - Properties: **name** (without path), **lastModifiedDate**,
  - Constructor: `new File(Blob fileBits, fileName);`
- The File is read with **FileReader** which is as follows:
  - Methods: Removed: **readAsBinaryString(blob)** (some examples!)
    - **readAsArrayBuffer(blob)** – only Android, iOS
    - **readAsDataURL(blob)** – returns **data:** URL (rfc2397)
    - **readAsText(blob [, encoding])** – read text data with specified (e.g. "utf-8") or default encoding (from MIME type, system)
    - **abort()** – abort reading of file data
  - Properties: **readyState**: **EMPTY** (0), **LOADING** (1), **DONE** (2)
    - **result**: data based on used method
    - **error**: returned error code (NotFoundError, SecurityError, NotReadableError)
  - Event handlers: **onloadstart** (loadstart), **onprogress** (progress – changes in download), **onabort** (abort), **onerror** (error), **onload** (file/blob loading completed), **onloadend** (loadend – the request has completed either way)

# File API: File System (Cordova)

- The window object contains two new methods:
  - `requestFileSystem(type, req_size, successCB, errorCallback);`
    - `type` is either `TEMPORARY`(0) or `PERSISTENT`(1)
    - `req_size` specifies how many bytes are needed (ask user?)
    - the `FileSystem` (or error) object is passed to callback
  - `resolveLocalFileSystemURL(url, successCB, errorCallback);`
    - The successCB is provided File System `Entry`
- The `FileSystem` object provides `name` of FS and `root` entry
- The `Entry` provides file/directory information & manipulation
  - Properties: `isFile`, `isDirectory`, `name` (no path), `fullPath`, `filesystem` (filesystem `type`)
  - Query methods: `toURL()`, `getMetadata(successCB, errorCallback);`
    - `successCB` is provided with `MetaData` object containing `modificationTime` (Date) and `size`
  - `getParent(successCB, errorCallback)` – successCB gets `Entry`
  - Manipulation: `remove(successCB [, errorCallback])`  
`moveTo(parent, newName [,successCB, errorCallback])`  
`copyTo(parent, newName [,successCB, errorCallback])`

# File API: FS – Files & Directories

- The **DirectoryEntry** is a special **Entry** for directory traversal
  - Query methods: **createReader()** – returns a **DirectoryReader** with **readEntries(successCB, errorCallback)** method
    - **readEntries** passes an array of all Entries in given directory to **successCB** for processing
  - Directory Manipulation: **getFile/getDirectory(path[, options][, successCB][, errorCallback])** – provides Entry in **successCB**, either the file/directory is found or it may be created based on options.
    - Options may contain **create** (true → create new file) and **exclusive** (file/dir is to be created & already exists → error)
    - **removeRecursively(successCB[, errorCallback])** – removes the whole subtree of this directory
- The **FileEntry** is extension of **Entry** for R/W file access
  - Makes other parts of File API accessible through FS
  - File Writer API: **createWriter(successCB [, errorCallback])**
    - The **FileWriter** instance is passed to **successCB** callback
  - File API: **file(successCB [, errorCallback])** – and instance of **File** is passed to **successCB**

# File API: File System errors

Following errors may appear when working with file system

- `EncodingError` – malformed path or URL
- `InvalidModificationError` – file system collision, invalid file operation
- `InvalidStateError` – cached state is invalid – entry was modified in storage by other means
- `NotFoundError` – missing file/directory
- `NotReadableErr` – file or directory could not be read.
- `NoModificationAllowedError` – the file/directory can't be modified due to the underlying FS state
- `PathExistsError` – creating duplicate file/directory
- `QuotaExceededError` – the requested storage quota limit would be required
- `SecurityError` – the access to API was denied for some reason (unsafe file, too many calls, ...)
- `TypeMismatchError` – working with directory like it was a file and vice versa.



# File API: File Writer

- We have to generate the **Blob** to be saved first
  - by using Blob constructor (current standard approach)
  - with deprecated BlobBuilder (append & getBlob methods)
- API defines **FileSaver** interface with some basic features:
  - Method: **abort()**
  - Attributes: **readyState** – **INIT** (0), **WRITING** (1), **DONE** (2),  
**error** – AbortError, InvalidStateError, NotFoundError,  
NoModificationAllowedError, NotReadableError,  
QuotaExceededError, SecurityError, SyntaxError
  - Events handlers: **onwritestart** (writestart), **onprogress**  
(progress), **onwrite** (write), **onabort** (abort), **onerror** (error),  
**onwriteend** (writeend) – see similar read events
- The **FileWriter** implements **FileSaver** interface and adds following necessary features:
  - Methods: **write**(Blob data), **seek**(offset) – negative offset  
references position from the end of the file, **truncate**(size)
  - Attributes: **position** (current file offset), **length** (current file length)



# File API – final remarks

- We may use synchronous variant of the the FS part of API in web workers, instead of callbacks (if implemented)
  - since WW are not present everywhere and File API does not work in default web browsers on mobile platforms we did not address this issue
- Will be probably replaced in future with File System Access API
- Some platforms do not support
  - FileReader (mainly desktop browsers)
  - FileWriter & FileReader.readAsArrayBuffer (most platforms in built-in browser, support in Apache Cordova is limited to iOS and Android)
  - Cordova Specifics:
    - `dirent.removeRecursively(...)` may fail on BB 10 if the directory was created by different version of application or if USB storage is connected to PC
    - Encoding parameter in `readAsText()` is ignored on iOS

# Writing to a file in JavaScript

It is also possible to create a downloaded file in memory and let the browser store it (e.g. done by Mega), with some limitations (available RAM/5, ~500 MiB)

- You may use [FileSaver.js](#) which chooses the best way
- By creating an `<a>` element and Blob object with data, setting the Blob to *href*, name to *download* and simulating click on the link and then removing the link from DOM:

```
var file = new Blob([data], {type: type});
var a = document.createElement("a"), url = URL.createObjectURL(file);
a.href = url; a.download = filename;
document.body.appendChild(a); a.click();
setTimeout(function() {document.body.removeChild(a);window.URL.revokeObjectURL(url);},0);
```

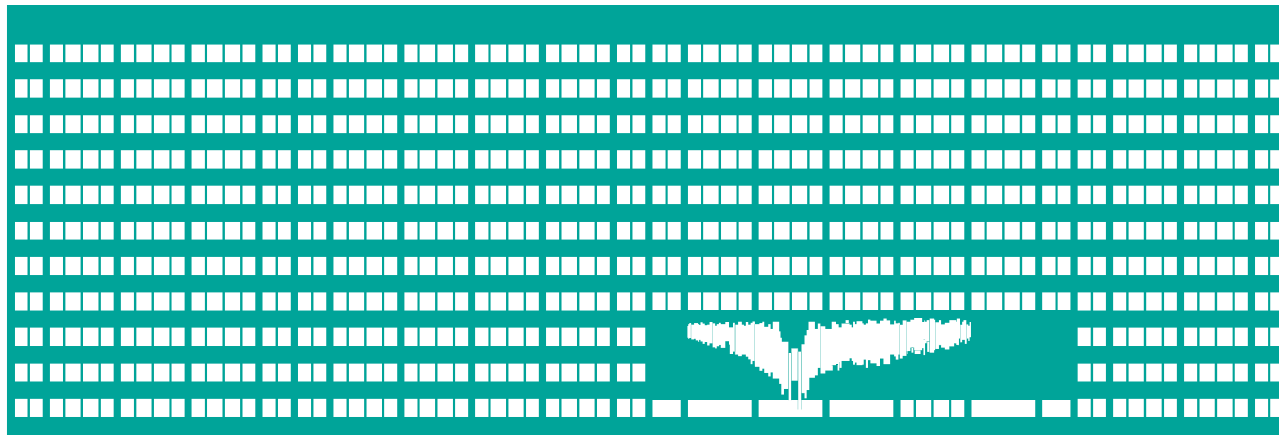
- Using CORS AJAXv2
- By using Blob conversion to `data : URL` by FileReader and a popup

# File System Access API

- New approach to accessing file systems in newest iOS (15.2+) and **Chromium-based** browsers, **not yet fully implemented**
- Uses ECMAScript 6+ Promises, does not require input type="file", but secure origins (HTTPS) are required
  - Files: showOpenFilePicker(pickerOpts)/showSaveFilePicker()
  - Dirs: showDirectoryPicker(), **OPFS**: navigator.storage.getDirectory()
  - options include extensions, mime types, multiple selections
  - read/write/close/flush/truncate/getSize should be available
- Examples:
  - ```
const [fileHandle] = await window.showOpenFilePicker();  
const fileData = await fileHandle.getFile();
```
 - ```
const dirH = await window.showDirectoryPicker();
const subD = dirH.getDirectoryHandle("sub", {create:true})
```
  - ```
const outH = await window.showSaveFilePicker();  
const writableStream = await outH.createWritable();  
await writableStream.write(imgBlob);  
await writableStream.close();
```

XML, Web Services

Data Exchange Formats



TAMZ 1
Lecture 10

Formats to exchange data

- Several data formats for information exchange with mobile devices have already been mentioned
 - HTML
 - JSON
 - XML (without details)
 - CSV (comma-separated values)
- Some additional application specific format should be considered
 - Contacts
 - vCard
 - MECARD (esp. in combination with QR codes)
 - Calendar entries
 - iCalendar (not to be confused with Apple iCal)
 - ATOM feed (XML)

Processing XML data

How does XML document look?

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<gpx xmlns="http://www.topografix.com/GPX/1/1"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
creator="Example-builder" version="1.1"  
xsi:schemaLocation="http://www.topografix.com/GPX/1/1  
http://www.topografix.com/GPX/1/1/gpx.xsd">
```

```
<metadata />
```

```
<wpt lat="50.0000" lon="17.1717">
```

```
  <name>Sample point</name>
```

```
  <cmt>Just a sample &amp; nothing more</cmt>
```

```
  <desc>An example of XML file to use for GPS app</desc>
```

```
  <sym>Dot</sym>
```

```
  <type>Reference Point</type>
```

```
</wpt>
```

```
</gpx>
```

XML Format Definition

There exist two basic types of data format definition for XML:

- **DTD** (older, deprecated, considered obsolete)

Basic specification of XML elements and their attributes and possible contents

```
<!DOCTYPE GPX [  
  <!ELEMENT GPX (WPT*,RTE*,TRK*,EXTENSIONS?)>  
  ...  
  <!ATTLIST WPT lat CDATA #REQUIRED>  
  ...  

```

- **XML Schema**

XML representation of another XML format structure, defining attribute types, annotations, number of occurrences and many more details ...

XML Schema Example

```
<?xml version="1.0" encoding="UTF-8"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns="http://www.topografix.com/GPX/1/1"
targetNamespace="http://www.topografix.com/GPX/1/1"
elementFormDefault="qualified">
  <xsd:element name="gpx" type="gpxType" />
  <xsd:complexType name="gpxType">
    <xsd:sequence>
      <xsd:element name="wpt" type="wptType" minOccurs="0"
maxOccurs="unbounded"/>
      <xsd:attribute name="version" type="xsd:string"
use="required" fixed="1.1">
...
    </xsd:sequence>
  </xsd:complexType>
</xsd:schema>
```

XML Processing

- From a developer's perspective, Java/JavaScript makes application portable among platforms and XML makes data portable among applications.
- For traditional applications, several native XML parsers are available from different software vendors. These XML parsers provide a rich set of features for dealing with XML data within enterprise applications. But these parsers are too big for or not implemented in JavaScript
 - However, they may be usable on other mobile platforms...

There are several ways how to parse XML in your JavaME application:

- Write your own parser
- Use one of custom parser libraries
- Use built-in (DOM) XML parser

Event- and Tree-based XML Parsers

Two types of interfaces are available for parsing XML documents:

- **Event-based**

Reports parsing events directly to the application through callback methods. It provides a serial-access mechanism for accessing XML documents. Applications that use a parser's event-based interface need to implement the interface's event handlers to receive parsing events.

- Typically found in native frameworks on mobile platforms

- **Tree-based**

Reads an entire XML document into an internal tree structure in memory. Each node of the tree represents a piece of data from the original document. It allows an application to navigate and manipulate the parsed data quickly and easily.

- Parser in mobile JavaScript, DOM approach

XML Parsers

There are three models of XML parsers.

- **Push (SAX)**

The parser generates a transient flow of events. Each caught event is handled immediately and the relevant information is passed on with the event. A push parser reads through an entire XML document. As it encounters various parts of the document, it notifies a listener object.

- **Model**

First, the parser creates a tree-like data structure, that models the XML source document; then the application walks through the tree. A model parser reads an entire document and creates a representation of the document in memory.

- **Pull**

A pull parser reads a small amount of a document at once. The application drives the parser through the document by repeatedly requesting the next piece.

Some typical XML parsers

- XML DOM parser in JavaScript – new DOMParser(), .NET (WP), ...
- Libraries for programming languages/platforms:

Name	Language	Type
Simple API for XML (SAX)	Java (Android), .NET	push
NSXMLParser (SAX-based)	Objective-C (iOS)	push
Streaming API for XML (StAX)	Java	pull
XMLPull API	Java (Android)	pull
XMLReader	.NET, PHP	pull
TinyXML	C (e.g. iOS)	dom

- On some platforms, we may also use direct data binding
 - XML document works as the business object
 - Examples:
 - Java Architecture for XML Binding (JAXB)
 - XML Serialization in .NET

XML data processing showcase: Android platform

See e.g.: <http://www.ibm.com/developerworks/opensource/library/x-android/>

SAX: Push Parser

- The **javax.xml.parsers.SAXParserFactory** enables applications to configure and obtain a SAX-based parser:
 - **SAXParserFactory factory = SAXParserFactory.newInstance();**
//factory.setValidating(true);
// ^ Check that document is correct, default: false
SAXParser parser = factory.newSAXParser();
- The **SAXParser.parse()** method starts the parse process on given input data.
 - **InputStream input = ...**
CountHandler handler = new CountHandler();
parser.parse(input, handler);
- We can create a parser directly instead of using the SAX parser factory by simply calling
 - **android.util.Xml.parse(input, Xml.Encoding.UTF_8, handler);**
 - **android.util.Xml.parse(reader_or_string, handler);**

SAX: Event Handler

The **org.xml.sax.helpers.DefaultHandler** is a default implementation of a class receiving and parsing events.

- All methods are present with an empty implementation

```
public class CountHandler extends DefaultHandler {  
    protected int count;                //Counter  
    private String wanted = "wpt";      //Counted tag name  
  
    public void startDocument() throws SAXException { count = 0; }  
  
    public void startElement(String nameSpace, String localName,  
                             String qualifiedName, Attributes attributes)  
                             throws SAXException {  
        if (qualifiedName.equals(wanted)) { count++; }  
    }  
}
```


DOM Parser (w3c document builder)

The **javax.xml.parsers.DocumentBuilderFactory** enables applications to configure and obtain a DOM parser

```
int count=0;           //Counter
String wanted = "wpt"; //Counted tag name
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document document = builder.parse(input);

Element root = document.getDocumentElement();
NodeList children = root.getChildNodes();
int chcnt = children.getLength()
for (int i = 0; i < chcnt; i++) { //Works here, since every <wpt> is directly in <gpx>
    Element child = children.item(i);
    String name = child.getLocalName();
    if (name.equals(wanted)) { count++; }
}
```

XmlPull API: Pull Parser

The **org.xmlpull.v1.XmlPullParserFactory** enables applications to configure and obtain a pull parser

- We can also use **android.util.Xml.newPullParser()** directly

```
int count=0;           //Counter
String wanted = "wpt"; //Counted tag name
XmlPullParserFactory fac = XmlPullParserFactory.newInstance();
XmlPullParser parser     = fac.newPullParser();
//Instead of green lines: XmlPullParser parser=android.util.Xml.newPullParser()
parser.setInput(xmlStream,"UTF-8");
while (true) {
    int t = parser.getEventType();
    if (t== XmlPullParser.START_TAG && parser.getName().equals(wanted)){
//parser.getAttributeCount(); parser.getAttributeName(i), .getAttributeValue(i)
        count++;
    }
    if (t == XmlPullParser.END_DOCUMENT) { break; }
    parser.next(); // parser.nextTag() for tags, etc.
}
```

XmlPull API: XML Serializer

The **org.xmlpull.v1.XmlPullParserFactory** enables applications to configure and obtain a serializer to store an XML document

- We can also use **android.util.Xml.newSerializer()** directly

```
XmlPullParserFactory factory = XmlPullParserFactory.newInstance();
XmlSerializer serializer    = factory.newSerializer();
//Or directly: XmlSerializer serializer = android.util.Xml.newSerializer();
String namespace            = "vsb";          //Tags namespace
OutputStream output         = ...
```

```
serializer.setOutput(output, "UTF-8"); // OutputStream & Encoding / Writer
serializer.startDocument("utf-8", Boolean.TRUE); // TRUE: Standalone doc.
serializer.startTag(namespace, "course").text("TAMZ 1")
                                                .endTag(namespace, "course");
serializer.endDocument();
```

Output:

```
<?xml version="1.0" encoding="UTF-8"?>
<vsb:course>TAMZ 1</vsb:course>
```

XML usage example – Android UI

UI Layout may be defined in XML instead of the code:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >
    <TextView android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a TextView" />
    <Button android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a Button" />
</LinearLayout>
```

Contacts exchange formats

vCard format

- Originally proposed as Versitcard in 1995, several versions of the standard. Most used versions:
 - 2.1 – widest support, export often to this format
 - 3.0 – defined in RFC 2425 and RFC 2426, slight changes
- MIME type is text/vcard (Android directly shows the vCard text in browser, unless the Content-Disposition is attachment)
- File extension is .vcf, several vCards may be stored in one file
- Records are in the form KEY:VALUE
 - Both keys and values may contain several fields, separated by ; (we can supply several phone numbers, ...)
 - Some records are compulsory (BEGIN, END, VERSION, N until version 3.0, FN since version 3.0)
- Several formats have been derived from vCard as well
 - xCard – XML representation of vCard (RFC 6351)
 - jCard – JSON representation
 - hCard – embedding vCard into HTML using CSS

vCard 2.1 example

BEGIN:VCARD

VERSION:2.1

N:Moravec;Pavel;;Ing., Ph.D.

FN:Pavel Moravec

ORG:VSB-TU Ostrava

TITLE:Assistant professor

NOTE:Know your lecturer!

TEL;WORK:+420597325896

TEL;CELL:+420608*****

ADR;WORK:;;;EA 409, FEECS;VSB-TU Ostrava;17. listopadu 15, Ostrava - Poruba

URL;WORK:<http://homel.vsb.cz/~mor03/>

EMAIL;PREF:pavel.moravec@vsb.cz

REV: 20140421T231132+0200

END:VCARD

vCard 3.0 example

BEGIN:VCARD

VERSION:3.0

N:Moravec;Pavel;;Ing., Ph.D.

FN:Pavel Moravec

ORG:VSB-TU Ostrava

TITLE:Assistant professor

NOTE:Know your lecturer!

PHOTO;VALUE=URL;TYPE=JPEG:http://wiki.cs.vsb.cz/images/5/5d/Moravec-head-thumbnail.jpg

TEL;TYPE=WORK:+420597325896

TEL;TYPE=CELL:+420608*****

ADR;TYPE=WORK:;;EA 409, FEECS;VSB-TU Ostrava;17. listopadu 15, Ostrava - Poruba

URL;TYPE=WORK:http://homel.vsb.cz/~mor03/

EMAIL;TYPE=WORK:pavel.moravec@vsb.cz

REV: 20140421T231132+0200

END:VCARD

MECARD

- vCards may be included in QR codes both directly and indirectly (URL leading to the vCard) Disadvantages:
 - The QR code may be hard to read or even impossible to generate in direct approach – too many characters have to be stored in it
 - Internet connectivity is required for indirect approach
- QR readers support the MECARD format, which is simpler and requires less characters (but supports more phone numbers, emails, ..., but not their types like vCard)
 - The file starts with MECARD: prefix, followed by individual records (N – name, if comma-separated, the field order is lastname,firstname, titles; TEL; TEL-AV – video phone; EMAIL; NOTE; BDAY; ADR; URL; NICKNAME)

MECARD example

MECARD:

N:Ing. Pavel Moravec Ph.D.;

TEL:+420596995896;

EMAIL:pavel.moravec@vsb.cz;

ADR:EA409, FEECS, VSB-TU Ostrava, 708 00 Ostrava-Poruba;



Calendar events exchange formats

iCalendar format

- Proposed by Apple, supported in many applications
 - based on older vCalendar 1.0 format (.vcs extension), which is a reason why VCALENDAR is still used in syntax
- MIME type is text/calendar (see Android vCard problem)
- File extension is .ics or .ifb (availability information).
- Current iCalendar standard is defined in RFC 5545
- Records are in the form KEY:VALUE
 - Both keys and values may contain several fields, separated by ; (we can supply several phone numbers, ...)
 - Some records are required (BEGIN, END, VERSION, ...)
- The calendar contains basic information and one or more of following records:
 - VEVENT – calendar event (may be repeated)
 - VTODO – TODO list entry, VJOURNAL – journal entry
 - VFREEBUSY – request for free/busy time
 - VTIMEZONE, VALARM (even inside of other records)

vCalendar example

BEGIN:VCALENDAR

VERSION:1.0

TZ:+01:00

BEGIN:VEVENT

COMPLETED:20140518T101500Z

DTSTART:20140211T094500Z

CLASS:PUBLIC

DTEND:20140211T111500Z

LOCATION:PORC3

SUMMARY;CHARSET=UTF-8;ENCODING=QUOTED-

PRINTABLE:Tvorba aplikac=C3=AD pro mobiln=C3=AD

za=C5=99=C3=ADzen=C3=AD l - p=C5=99edn=

. =09=0D=0A

RRULE:W1 TU 20140518T084500Z

AALARM:20140211T093000Z;;;

END:VEVENT

REV: 20140421T231132+0200

END:VCALENDAR

iCalendar VEVENT ex.

BEGIN:VCALENDAR

PRODID:-//BUSINESS-CALENDAR//APPGENIX-SOFTWARE//

VERSION:2.0

CALSCALE:GREGORIAN

METHOD:PUBLISH

X-WR-TIMEZONE:Europe/Prague

BEGIN:VEVENT

SUMMARY:TAMZI – en

UID:6:38@google.com

ORGANIZER;CN=pavel.moravec@vsb.cz:mailto:pavel.moravec@vsb.cz

DTSTART;TZID=Europe/Prague:20140422T141500

DTEND;TZID=Europe/Prague:20140422T154500

DTSTAMP:20140422T064437Z

CREATED:20140221T091422Z

LAST-MODIFIED:20140422T064437Z

LOCATION:JA401

RRULE:FREQ=WEEKLY;INTERVAL=1;BYDAY=TU;UNTIL=20140518T121500Z

STATUS:TENTATIVE

BEGIN:VALARM

DESCRIPTION:This is an event reminder

ACTION:DISPLAY

TRIGGER:-P0DT0H15M0S

END:VALARM

END:VEVENT

END:VCALENDAR

iCalendar VTODO example

BEGIN:VCALENDAR

PRODID:-//Google Inc//Google Tasks//EN

VERSION:2.0

BEGIN:VTODO

UID:XXXXXXXXXXXXXXXXXXXXXXXXXXXXwMjgy@google.com

DTSTAMP:20140421T121001Z

DUE;VALUE=DATE:20140422

SUMMARY:Prepare presentation for TAMZ lecture

STATUS:NEEDS-ACTION

CATEGORIES:WORK

END:VTODO

END:VCALENDAR

iCalendar VJOURNAL example

BEGIN:VCALENDAR

PRODID:-//ABC Corporation//NONSGML My Product//EN

VERSION:2.0

BEGIN:VJOURNAL

UID:19970901T130000Z-123405@example.com

DTSTAMP:19970901T130000Z

DTSTART;VALUE=DATE:19970317

SUMMARY:Staff meeting minutes

**DESCRIPTION:1. Staff meeting: Participants include Joe\,
Lisa\, and Bob. Aurora project plans were reviewed.
2. Telephone Conference: ABC Corp. sales representative
called to discuss new printer. Promised to get us a demo by
Friday.\n3. Henry Miller (Handsoff Insurance): Car was
totaled by tree.**

END:VJOURNAL

END:VCALENDAR

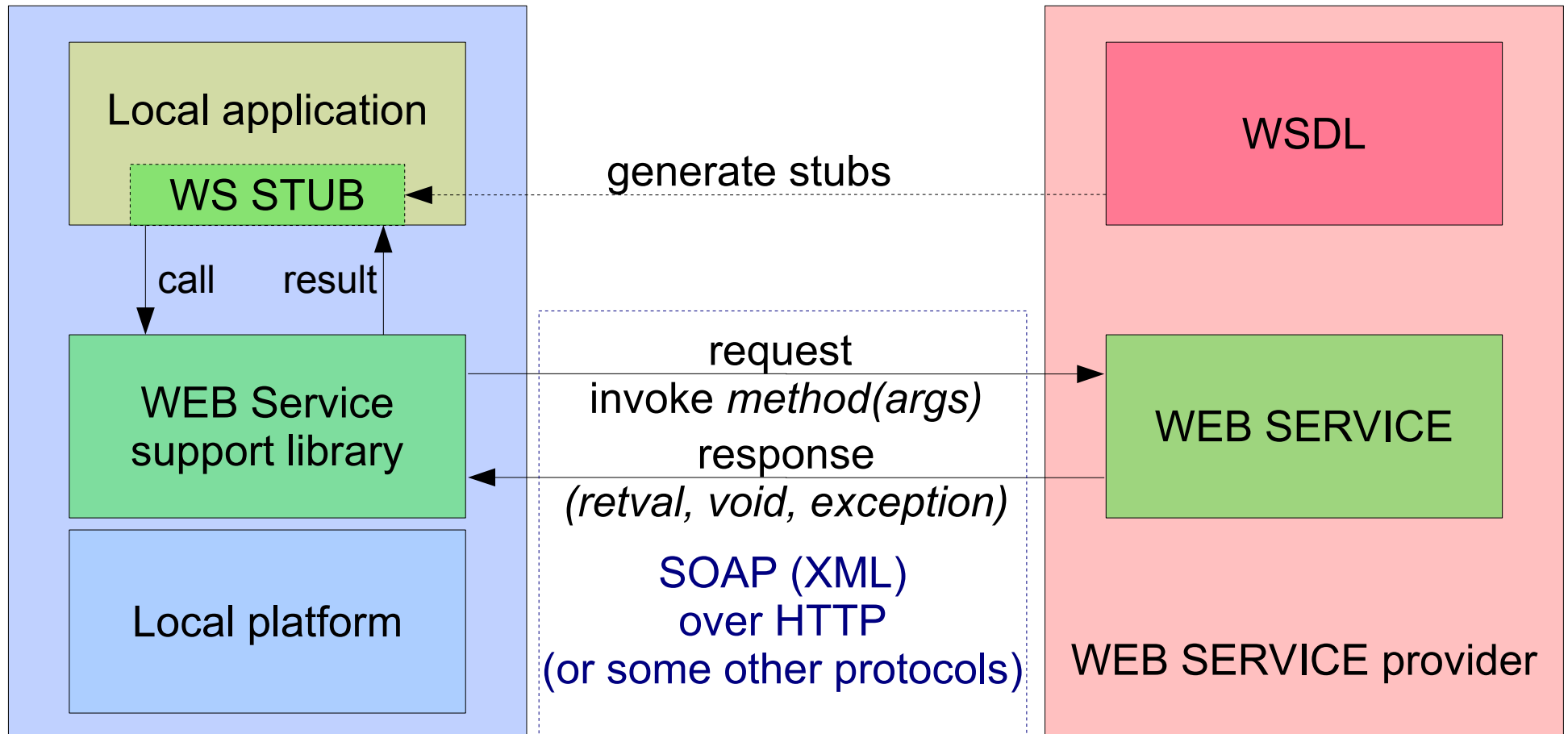
Web Services

Web Services Summary

With Web services, we are able to invoke a method on remote server and obtain its results. Following core specifications define web services:

- WSDL – Web Services Definition Language (web service description in XML format, stored on server and used for local stub generation), W3C recommendation
 - XML 1.0 – used by both SOAP and WSDL
 - XML Schema
- SOAP – Simple Object Access Protocol – data encoding using XML and transport encoding via HTTP, accessing web services, W3C recommendation
- UDDI (Universal Description, Discovery, and Integration) specification for remote service discovery, described in WSDL, SOAP communication. Some platforms do not use.
- RDF – Resource Description Framework, W3C recommendation, uses XML.

Basic Web Service Communication



Data type mapping example – Java

The mapping of SOAP data types to Java is following:

- Basic types remain the same:
xsd:boolean $\Leftrightarrow$ *boolean* or *Boolean*, *xsd:byte* $\Leftrightarrow$ *byte* or *Byte*.
xsd:short $\Leftrightarrow$ *short* or *Short*, *xsd:int* $\Leftrightarrow$ *int* or *Integer*.
xsd:long $\Leftrightarrow$ *long* or *Long*, *xsd:string* $\Leftrightarrow$ *String*
- Floating point numbers (may not be available $\rightarrow$ String):
xsd:float $\Leftrightarrow$ *float* or *Float*
xsd:double $\Leftrightarrow$ *double* or *Double*
- Some types are undefined, they represent byte arrays:
xsd:base64Binary $\Rightarrow$ *byte[]*, *xsd:hexBinary* $\Rightarrow$ *byte[]*.
- Special types are mapped as follows:
xsd:complexType $\Rightarrow$ Sequence of primitive & class types.
xsd:QName $\Leftrightarrow$ *javax.xml.namespace.QName*.
Arrays $\Leftrightarrow$ *array*

WSDL Excerpt

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" ...
xmlns:http="http://schemas.xmlsoap.org/wsdl/http/"
targetNamespace="http://tempuri.org/"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/">
  <wsdl:types>
    <s:schema elementFormDefault="qualified" targetNamespace="http://tempuri.org/">
      <s:element name="FahrenheitToCelsius">
        <s:complexType><s:sequence>
          <s:element minOccurs="0" maxOccurs="1" name="Fahrenheit" type="s:string"/>
        </s:sequence></s:complexType>
      </s:element>
      <s:element name="FahrenheitToCelsiusResponse">
        <s:complexType><s:sequence>
          <s:element minOccurs="0" maxOccurs="1" name="FahrenheitToCelsiusResult"
            type="s:string"/>
        </s:sequence></s:complexType>
      </s:element>
    </s:schema>
  </wsdl:types>... </wsdl:definitions>
```

Source: <http://www.w3schools.com/webservices/tempconvert.asmx?WSDL>

Generated classes – Java example

- Following classes are generated from WSDL (with **wscompile**):
 - FahrenheitToCelsius.java – complex type from previous slide
 - FahrenheitToCelsiusResponse.java – complex type from previous slide
 - CelsiusToFahrenheit.java – backwards conversion
 - CelsiusToFahrenheitResponse.java – backwards conversion
 - *TempConvertSoap.java* – interface defining the WS methods
 - TempConvertSoap_Stub.java – WS stub implementing the interface
- Following two methods are provided by the *TempConvert* web service:
 - `public String fahrenheitToCelsius(String fahrenheit)
throws java.rmi.RemoteException;`
 - `public String celsiusToFahrenheit(java.lang.String celsius)
throws java.rmi.RemoteException;`

Client code – Java example

```
import java.rmi.RemoteException;
import TemperatureConverter.*;
import javax.xml.rpc.Stub;
class TemperatureConverter {
    //String wsURL="http://www.w3schools.com/webservices/tempconvert.asmx?WSDL";
    TempConvertSoap_Stub wsConv = new TempConvertSoap_Stub();
    public TemperatureConverter() {
        //wsConv._setProperty(Stub.ENDPOINT_ADDRESS_PROPERTY, wsURL);
        wsConv._setProperty(Stub.SESSION_MAINTAIN_PROPERTY,
                           new Boolean(true));
    }
    public void printCelsius(float fahrenheit) {
        try {
            System.out.println(wsConv.FahrenheitToCelsius(Float.toString(fahrenheit)));
        } catch (RemoteException ex) {
            System.out.println("Unable to contact web service, reason: " + ex.toString());
        }
    }
}
```

Client Communication with WS

```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:soap=
    "http://schemas.xmlsoap.org/soap/envelope/" xmlns:tns="http://tempuri.org/">
  <soap:Body>
    <tns:FahrenheitToCelsius>
      <tns:Fahrenheit>80.5</tns:Fahrenheit>
    </tns:FahrenheitToCelsius>
  </soap:Body>
</soap:Envelope>
```

Request

```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <soap:Body>
    <FahrenheitToCelsiusResponse xmlns="http://tempuri.org/">
      <FahrenheitToCelsiusResult>26.94444444444444</FahrenheitToCelsiusResult>
    </FahrenheitToCelsiusResponse>
  </soap:Body>
</soap:Envelope>
```

Response

Web Services in JavaScript

- We can write our own custom JavaScript code to send a request to a web service, with following HTTP headers:
 - Content-Type: application/soap+xml
 - SOAPAction: <Endpoint>/<Action>
SOAPAction: <http://www.w3schools.com/webservices/FahrenheitToCelsius>
- Response Content-Type will be text/xml
- We can also use existing libraries:
 - Apache CXF – <http://cxf.apache.org/>
 - wsdl2js – tool to generate JS client and stubs
<http://sourceforge.net/projects/wsdl2js/>
 - JavaScript SOAP client
 - <http://www.guru4.net/articoli/javascript-soap-client/en/>
- Currently, we use **JSON instead** of WS, but it may lack the (WSDL/RDF) description framework. Some solutions:
 - WADL – Web Application Description Language
 - RSDL – RESTful Service Description Language
 - RAML – RESTful API Modelling Language – <http://raml.org>

JavaScript Example – method

```
var url="http://www.w3schools.com/webservices/tempconvert.asmx"
function FahrenheitToCelsius(/*string*/ Fahrenheit) {
    soapMess ="<?xml version=\"1.0\" encoding=\"utf-8\"?>"
    +"<soap:Envelope xmlns:xsi=\"http://www.w3.org/2001/XMLSchema-instance\"
    xmlns:xsd=\"http://www.w3.org/2001/XMLSchema\"
    xmlns:soap=\"http://schemas.xmlsoap.org/soap/envelope/\">" + "<soap:Body>"
    +"<FahrenheitToCelsius xmlns=\"http://www.w3schools.com/webservices/\">"
    soapMess += "<Fahrenheit>" + Fahrenheit + "</Fahrenheit>"
    soapMess += "</FahrenheitToCelsius>"
    soapMess += "</soap:Body></soap:Envelope>"
    xmlHttpRequest = getHttpRequest();
    xmlHttpRequest.onreadystatechange = FahrenheitToCelsius_callback;
    xmlHttpRequest.open("POST",this.url,true);
    xmlHttpRequest.setRequestHeader("SOAPAction",
        "http://www.w3schools.com/webservices/FahrenheitToCelsius");
    xmlHttpRequest.setRequestHeader("Content-Type","text/xml; charset=utf-8");
    xmlHttpRequest.send(soapMess);
}
```

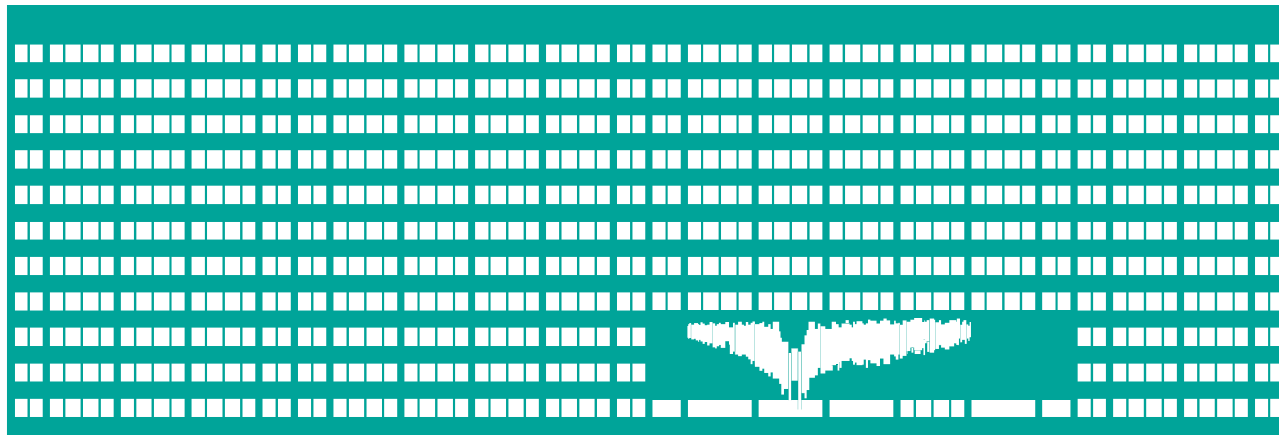
JavaScript Example – result

```
function FahrenheitToCelsius_callback(){
    var rawData;
    if (xmlHttpRequest.readyState == 4) {
        if(xmlHttpRequest.status == 200) {
            rawData = xmlHttpRequest.responseXML;
            var resultNode = rawData.documentElement.firstChild.firstChild.firstChild;
            var resultValue = resultNode.firstChild.nodeValue;
            FahrenheitToCelsius_handler(resultValue);
        }else{
            alert("web service response error:" + xmlHttpRequest.status + ","
                + xmlHttpRequest.statusText);
        }
    }
}

function FahrenheitToCelsius_handler(/*string*/ resultValue) {
    // Write your own code here...
}
```

Minification and Obfuscation

Out of the scope of HTML5:
Bluetooth, Threads



TAMZ 1
Lecture 11

Verifying the remotely loaded code

(this is NOT application signing)

Subresource Integrity in HTML5

We can use Subresource Integrity for loading of resources from other origins when we want to ensure their legitimacy

- Each `<link>` and `<script>` element may have `integrity` attribute.
 - Integrity attribute contains hash identifier, then dash (-) and base-64 encoded data of the digest
 - Examples of generating SRI data:
 - `cat x.js | openssl dgst -sha512 -binary | openssl base64 -A`
 - `sha512sum -b -a 512 x.js | cut -f1 | xxd -r -p | base64`
 - Both will be prefixed with `sha512-`
 - Or generate at: <https://www.srihash.org/>
 - CDNs provide the hash, e.g. <https://cdnjs.com/libraries/onsen>
 - Click on copy Script tag and you will get it including SRI (but you must trust the CDN first)
- If the hash does not match, the resource is not loaded
 - Do not use on URLs like `library/latest/library.js` where hashes change

Obfuscation and Minimization

(Why and how)

Obfuscation

JavaScript code is directly available; several tools exist which disassemble Android applications, Java bytecode, ...
→ it is suitable to make the code less readable.

- The obfuscator produces compatible (byte)code. It has three basic functions:
 - **Shrinking** – removes unnecessary code and classes,
 - **Optimalization** – makes the code smaller/more effective
 - **Obfuscation** – tries to make the code less readable (this may also reduce the code size, since methods/fields/classes can be renamed to a,b,c, etc.)

Netbeans/Eclipse use opensource *Proguard* obfuscator for Java. The “command line” options (set in project) change the aggressiveness of obfuscation (from no obfuscation to a very optimized/obfuscated). Other obfuscators are also available for different platforms(some free, some paid).

Proguard – configuration example

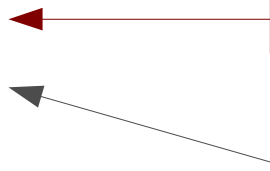
Basic settings to shrink, optimize, obfuscate, and preverify all public midlets in `in.jar`:

```
java -jar proguard.jar @proguard.cfg
```

Contents of `proguard.cfg`:

```
-injars      in.jar
-outjars     out.jar
-libraryjars library.jar
-overloadaggressively
-repackageclasses "
-defaultpackage "
-allowaccessmodification
-printseeds
-keep public class * extends Main_platform_class_name {
    public *;
}
```

different methods can get same names



change class packages
(new name for defaultpackage)

Minification

- Unlike obfuscation, minification is easily reversible through beautifier (pretty-printer, indenter)
- The main idea is to remove all unnecessary characters (whitespaces, newlines, ...) and comments
- The reason is to reduce the amount of the code to be transferred
 - We may also compress (gzip) the resulting file to further reduce the amount of transferred data
- Some tools provide both the minification and obfuscation
- You may use source maps to make debugging minified code feasible in browsers which are able to use them
 - Minified code than contains comment with older `//@ sourceMappingURL="..."` or `//# sourceMappingURL=`
 - The map file contains JSON object with mapping information, which browsers such as Chrome may use.

Minification example – original code

```
$('#get').on('click', getLocation);
```

```
function getLocation() { /* Start location search */  
  if (navigator.geolocation !== 'undefined') {  
    navigator.geolocation.getCurrentPosition(showPosition, failedPosition,  
      {maximumAge: 30000});  
  }  
}
```

```
function showPosition(pos) { //current position information callback  
  d = (new Date(pos.timestamp)).toISOString();  
  $("#ltext").val(d + ': ' + pos.coords.latitude +  
    " / " + pos.coords.longitude + " (+/- " + pos.coords.accuracy + "m)");  
  if (pos.coords.altitude !== null ) {  
    $("#ltext").append(" @ " + pos.coords.altitude + ' m (+/- ' +  
pos.coords.altitudeAccuracy + 'm)');  
  }  
}
```

```
function failedPosition(err) { x.value = err.message +  
  ' (code: ' + err.code + ')'; }
```

Minification tools (ignore the newlines)

- **uglifyjs:**

```
function getLocation(){navigator.geolocation!==undefined&&
navigator.geolocation.getCurrentPosition(showPosition,failedPosition,
{maximumAge:3e4})}function showPosition(e){d=(new
Date(e.timestamp)).toISOString(),$("#ltext").val(d+":
"+e.coords.latitude+" / "+e.coords.longitude+" (+/-
"+e.coords.accuracy+"m")),e.coords.altitude!==null&&$("#ltext").append("
@ "+e.coords.altitude+" m (+/-"+e.coords.altitudeAccuracy+"m))}function
failedPosition(e){x.value=e.message+" (code: "+e.code+"")}$
("#get").on("click",getLocation);
```
- **minify:**

```
function getLocation(){"undefined"!==navigator.geolocation&&
navigator.geolocation.getCurrentPosition(showPosition,failedPosition,
{maximumAge:3e4})}function showPosition(o){d=new
Date(o.timestamp).toISOString(),$("#ltext").val(d+": "+o.coords.latitude+" /
"+o.coords.longitude+" (+/- "+o.coords.accuracy+"m")),null!
==o.coords.altitude&&$("#ltext").append(" @ "+o.coords.altitude+" m
(+/-"+o.coords.altitudeAccuracy+"m))}function failedPosition(o)
{x.value=o.message+" (code: "+o.code+"")}$
("#get").on("click",getLocation);
```
- **Google Closure compiler, YUI Compressor, JSMin, ...**

Obfuscation extreme examples

- **jsobfuscate.com:**

```
eval(function(p,a,c,k,e,d){e=function(c){return c.toString(36)};if(!".replace(/^\//,String)){while(c--){d[e(c)]=k[c]||e(c)}k=[function(e){return d[e]};e=function(){return'\w+'};c=1};while(c--){if(k[c]){p=p.replace(new RegExp('\b'+e(c)+'\b','g'),k[c])}}return p}('$(\#f\').n(\k\,c);2 c(){9(4.6!==(j\') {4.6.i(7,b,{h:e})}}2 7(0){d=(g l(0.t)).w();$("#5").z(d+\': \' +0.1.y+" / "+0.1.o+" (+/- "+0.1.v+"m)");9(0.1.a!==(u){$("#5").p(" @ "+0.1.a+\ ' m (+/-\' +0.1.q+\ 'm)\')}2 b(3) {x.r=3.s+\ ' (8: \' +3.8+\ ')\'}',36,36,'pos|coords|function|err|navigator|ltext| geolocation|showPosition|code|if|altitude|failedPosition|getLocation||30000|get| new|maximumAge|getCurrentPosition|undefined|click|Date||on|longitude|append| altitudeAccuracy|value|message|timestamp|null|accuracy|toISOString||latitude| val'.split('|'),0,{}))
```

To deobfuscate quite well, use: <http://jsbeautifier.org/>

- **javascriptobfuscator.com:**

```
var _0x72b5=["\x63\x6C\x69\x63\x6B","\x6F\x6E", "\x23\x67\x65\x74","\x67\x65\x6F\x6C\x6F\x63\x61\x74\x69\x6F\x6E","\x75\x6E\x64\x65\x66\x69\x6E\x65\x64","\x67\x65\x74\x43\x75\x72\x72\x65\x6E\x74\x50\x6F\x73\x69\x74\x69\x6F\x6E","\x74\x6F\x49\x53\x4F\x53\x74\x72\x69\x6E\x67","\x74\x69\x6D\x65\x73\x74\x61\x6D\x70","\x3A\x20","\x6C\x61\x74\x69\x74\x75\x64\x65","\x63\x6F\x6F\x72\x64\x73","\x20\x2F\x20","\x6C\x6F\x6E\x67\x69\x74\x75\x64\x65","\x20\x28\x2B\x2F\x2D\x20","\x61\x63\x63\x75\x72\x61\x63\x79","\x6D\x29","\x76\x61\x6C","\x23\x6C\x74\x65\x78\x74","\x61\x6C\x74\x69\x74\x75\x64\x65","\x20\x40\x20","\x20\x6D\x20\x28\x2B\x2F\x2D","\x61\x6C\x74\x69\x74\x75\x64\x65\x41\x63\x63\x75\x72\x61\x63\x79","\x61\x70\x70\x65\x6E\x64","\x76\x61\x6C\x75\x65","\x6D\x65\x73\x73\x61\x67\x65","\x20\x28\x63\x6F\x64\x65\x3A\x20","\x63\x6F\x64\x65","\x29"];$_0x72b5[2])[_0x72b5[1]](_0x72b5[0],getLocation);function getLocation(){if(navigator[_0x72b5[3]]!=_0x72b5[4]){navigator[_0x72b5[3]][_0x72b5[5]](showPosition,failedPosition,{maximumAge:30000});} ;};function showPosition(_0xfe8cx3){d=( new Date(_0xfe8cx3[_0x72b5[7]]))[_0x72b5[6]]();$_0x72b5[17]][_0x72b5[16]](d+_0x72b5[8]+_0xfe8cx3[_0x72b5[10]][_0x72b5[9]]+_0x72b5[11]+_0xfe8cx3[_0x72b5[10]][_0x72b5[12]]+_0x72b5[13]+_0xfe8cx3[_0x72b5[10]][_0x72b5[14]]+_0x72b5[15]);if(_0xfe8cx3[_0x72b5[10]][_0x72b5[18]]!=null){$_0x72b5[17]][_0x72b5[22]](_0x72b5[19]+_0xfe8cx3[_0x72b5[10]][_0x72b5[18]]+_0x72b5[20]+_0xfe8cx3[_0x72b5[10]][_0x72b5[21]]+_0x72b5[15]);} ;};function failedPosition(_0xfe8cx5){x[_0x72b5[23]]=_0xfe8cx5[_0x72b5[24]]+_0x72b5[25]+_0xfe8cx5[_0x72b5[26]]+_0x72b5[27];};
```

Some things are missing in HTML5

- No direct communication with local peripheral devices
 - Serial ports
 - USB
 - Bluetooth
- Multi-threaded approach is not possible
 - Web workers are not available everywhere
 - Missing support for full-featured threads (like pthreads, Java threads), partly to avoid most multi-threading issues
 - May not fully utilize the possibilities of current multi-core CPUs
- Limited use of web services
 - we may write a wrapper or use a library to communicate in XML with XML-RPC server or a web service but it's easier to use JSON or custom XML format...
- No execution of native code unless we use Cordova

Bluetooth basics

Bluetooth

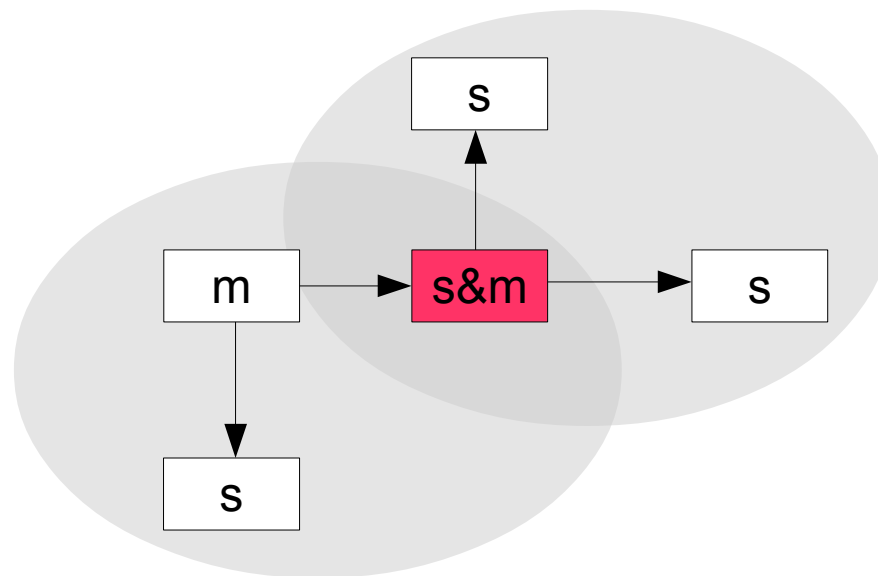


The Bluetooth wireless connectivity technology was originally envisioned in 1994 by the Swedish phone equipment maker Ericsson as a way for mobile devices to communicate with each other at short ranges, up to **10 meters** (100 meters). In 1998, Ericsson, IBM, Intel, Nokia, and Toshiba formed the Bluetooth Special Interest Group consortium to develop a royalty-free, open specification for short-range wireless connectivity.

Since then, more than 2000 companies have joined the Bluetooth SIG, including virtually all manufacturers of phone, computer, and PDA equipment.

Bluetooth Network Topology

Bluetooth-enabled devices are organized in groups called **piconets**. A piconet consists of a **master** and up to seven active **slaves**. A master and a single slave use point-to-point communication; if there are multiple slaves, point-to-multipoint communication is used. A master unit is the device that **initiates** the communication. A device in one piconet can communicate to another device in another piconet, forming a **scatternet**.



Bluetooth classification

- Low power consumption, short range, low-cost chips
- Classes (higher-class devices support lower classes):
 - Class 1 – up to ~ 100m, 100 mW
 - Class 2 – up to ~ 10m, 2.5 mW
 - Class 3 – up to ~ 1m, 1 mW
- Bluetooth main versions:
 - 1.2 – 1 Mbps, Basic rate
 - 2.0 & 2.1 - optional 2/3 Mbps EDR + small improvements, secure simple pairing in 2.1
 - 3.0 + HS – BT link used for establishment of co-located 802.11 link with speeds up to ~24 Mbps. AMP
 - 4.0 – adds Bluetooth low energy to 3.0
 - ...

Bluetooth protocol stack

- Mandatory protocols:
 - **LMP** (*Link Management Protocol*) – control of radio link between two devices. Implemented on controller.
 - **L2CAP** (*Logical Link Control & Adaptation Protocol*) – packet service multiplexing multiple connections of higher-level protocols, segmentation, 48 B → 64 kB per packet, default MTU 672 B. Additional Streaming & Enhanced ReTransmission Modes
 - **SDP** (*Service Discovery Protocol*) – services & their parameters – detection of supported profiles
- **HCI** (*Host/Controller Interface*) – communication with host stack (e.g. USB, UART). Optional for single-chip solutions.
- **RFCOMM** (*Stream transfer*) – virtual serial stream
- **OBEX** – sitting on top of RFCOMM and L2CAP. OBEX, the **Object Exchange protocol**, can be used to send and receive complete objects (e.g. files, images etc).

Bluetooth Profiles

Bluetooth profiles are intended to ensure **interoperability** among different devices and applications. A profile defines the **roles** and **capabilities** for specific types of applications.

- Generic Access Profile (GAP)
 - Defines connection procedures, device discovery, and link management. At a minimum all Bluetooth devices must support this profile.
- Service Discovery Application and Profile (SDAP)
 - Defines the features and procedures for an application to discover services registered in other devices.
- Serial Port Profile (SPP)
 - Defines the requirements for Bluetooth devices that need to set up connections that emulate serial cables.
- ...

Discovering a Remote Device

- Each Bluetooth device has a specific code (**CoD – class of device**), which consists of **service class** of each device (e.g. Telephony, networking, ...) and **device class** consisting of
 - **major** part (computer, audio, ...)
 - **minor** part (PDA for computer, headset for audio), which identifies the type of the device.

This code identifies, what is the function and role of given device.

- Each Bluetooth device has also an unique **6-byte MAC** address, through which it can be addressed.
- Some devices may be discovered, while others may not
 - **Generally discoverable devices** (GIAC) can be discovered at all times (always visible)
 - **Limited discoverable devices** (LIAC) can be discovered for short time periods (e.g. 30 seconds after a button on BT handsfree is pressed)
 - **Non-discoverable devices** (NOT_DISCOVERABLE) are „invisible“ (but we may still connect using MAC address).

Offering a Bluetooth Service

- The steps involved in registering a service are defined in the **Service Discovery Protocol** (SDP), which is part of the Bluetooth Specification, and are as follows:
 - create a service record
 - add the service record to the **Service Discovery Database**
 - set security measures associated with client connections
 - accept client connections
- Each Bluetooth service offered by a host is represented by a **Service Record** in the **Service Discovery Database** (SDDB). To connect to a service a client obtains a Service Record from the server and uses the information therein to connect to the service.

Well-known BT Service UUIDs

SDP	0x0001	16-bit
RFCOMM	0x0003	16-bit
OBEX	0x0008	16-bit
HTTP	0x000C	16-bit
L2CAP	0x0100	16-bit
BNEP	0x000F	16-bit
Serial Port	0x1101	16-bit
ServiceDiscoveryServerServiceClassID	0x1000	16-bit
BrowseGroupDescriptorServiceClassID	0x1001	16-bit
PublicBrowseGroup	0x1002	16-bit
OBEX Object Push Profile	0x1105	16-bit
OBEX File Transfer Profile	0x1106	16-bit
Personal Area Networking User	0x1115	16-bit
Network Access Point	0x1116	16-bit
Group Network	0x1117	16-bit

Base UUID Value

(used in promoting 16-bit and 32-bit UUIDs to 128-bit UUIDs)

0x00000000000001000800000805F9B34FB

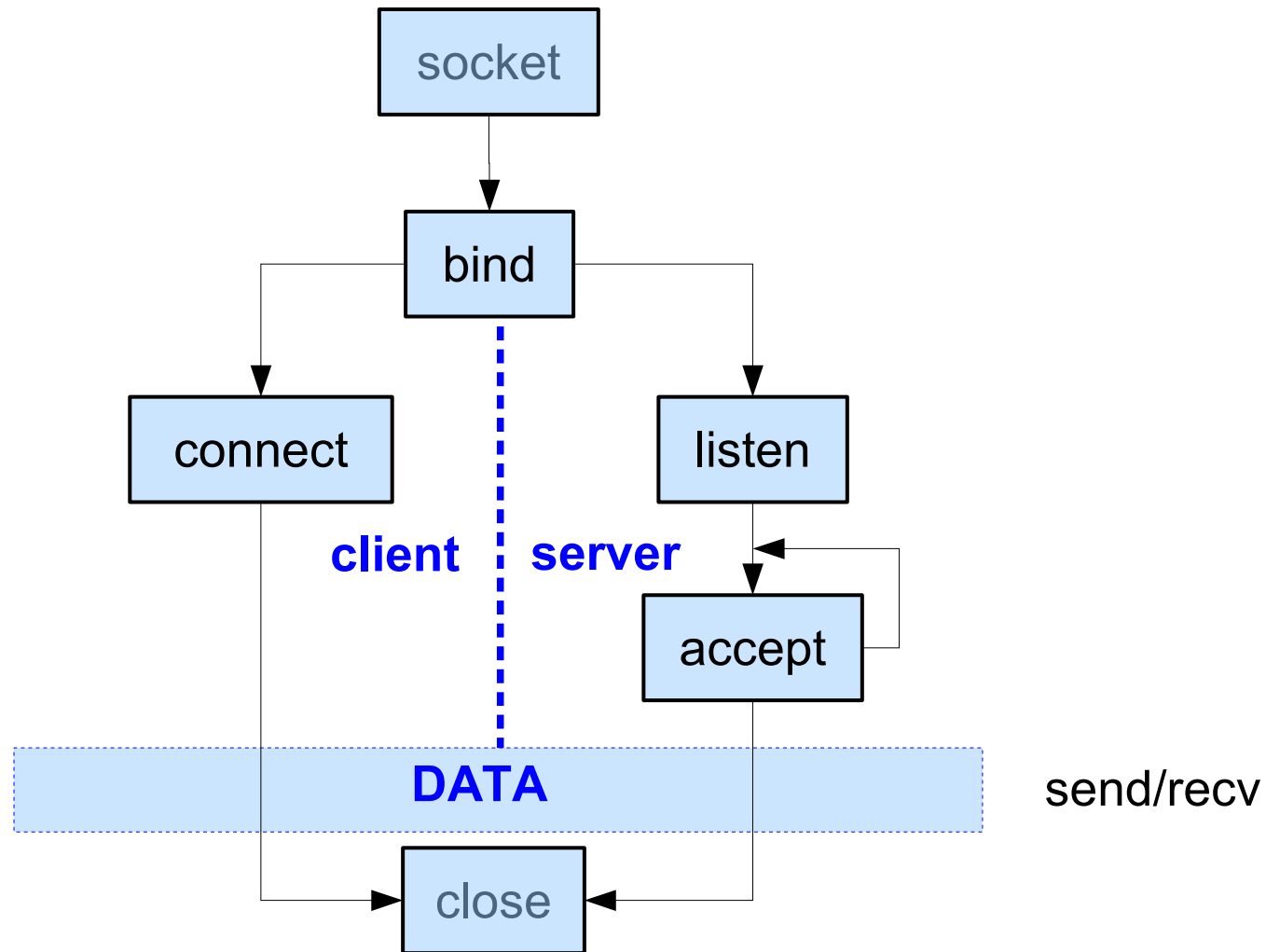
Socket-based communication

C/C++ BSD Sockets in POSIX

POSIX functions allow to access network connection in the same way as regular files:

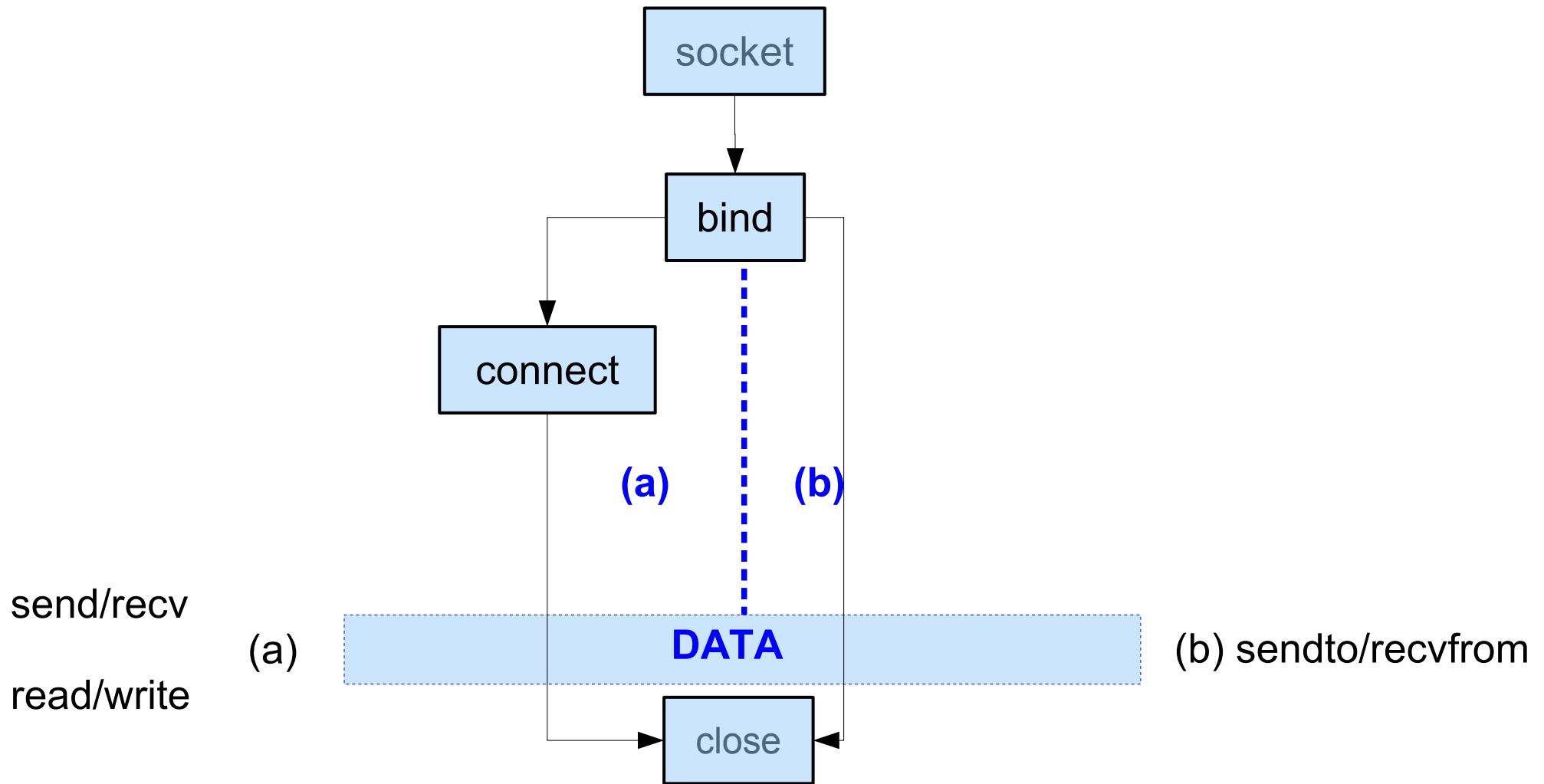
- There are special functions for opening the connections, the connection is known in POSIX as socket and may be used for different types of communication – inter-process communication on the same machine, different network protocols, etc.
- Socket- and protocol-specific parameters are set and retrieved by calling special functions
- Data transfer is done by specific functions or in regular way (read, write)
- Modern programming languages copy this approach in Object-oriented way to a degree (methods have similar or even the same names as POSIX functions).

Stream Communication (TCP)



In modern programming languages, we may usually just supply all needed info in constructor, skipping socket, bind, connect | listen

Datagram Communication (UDP)



Most modern platforms have a class representing datagram socket or “connection” and a class representing packet with sent/received data

Threads

Threads

There is often need to turn a program into separate, independently running subtasks. Each of these independent subtasks is called a **thread**, and is programmed as if each thread runs by itself and has the CPU to itself. The thread model is **a programming convenience** to simplify juggling several operations at the same time within a single program. There is several reasons why to do this.

- There can be one thread controlling and responding to a GUI, while another thread carries out the tasks or computations requested, while a third thread does file I/O, all for the same program.
- Some programs are easier to write if they are split into threads. The classic example is the server part of a client/server. When a request comes in from a client, it is very convenient if the server can spawn a new thread to process that one request.

Showcase: Threads on Android

- Android uses standard Java-based threads in basic API
 - Simplified pthread (POSIX threads) library is used in Android NDK (native development in C)
- Thread are best used for tasks which would block some part of program (e.g. GUI – application not responding) and represent a single, monolithic task.
- We may also use the AsyncTask to execute tasks asynchronously
 - Enables proper and easy use of the UI thread
 - Allows to perform background operations and publishes results on the UI thread without having to manipulate threads or handlers.
 - Defined by a computation that runs in a background thread and whose result is published on the UI thread
- On following slides, we will address the basic use of Java threads (more information will be shown in TAMZ 2)

Class Thread

The simplest way to create a thread is to inherit from class `java.lang.Thread`. The most important method for `Thread` is `run()`, which is the code that will be executed **simultaneously** with the other threads in a program.

```
public class CountdownThread extends Thread {  
    private static final int MAX_COUNT = 10;  
    private static int threads = 0;  
    private int thread = threads++;  
    public void run() {  
        for (int i = MAX_COUNT; i > 0; i--)  
            System.out.println("thread #" + thread + ": " + i);  
    }  
}
```

```
Thread thread = new CountdownThread();  
thread.start();
```

Interface Runnable

Sometimes it is impossible or inconvenient to inherit from `Thread` class. The `java.lang.Runnable` interface, declaring the `run()` method, can be implemented in these cases.

```
public class CountdownThread implements Runnable {  
    private static final int MAX_COUNT = 10;  
    private static int threads = 0;  
    private int thread = threads++;  
    public void run() {  
        for (int i = MAX_COUNT; i > 0; i--)  
            System.out.println("thread #" +  
                               thread + ": " + i);  
    }  
}
```

```
Thread thread = new Thread(new CountdownThread());  
thread.start();
```


The Life Cycle of a Thread

A thread goes through several states during its life cycle:

- **Creating**

When a thread is created, it is merely an empty object. No system resources are allocated. The thread in this can be only started.

- **Starting**

The `start()` method creates the system resources necessary to run the thread, schedules the thread to run, and calls the thread's `run()` method.

- **Making a Thread Not Runnable**

A thread becomes Not Runnable when one of these events occurs: its `sleep()` or `wait()` method is invoked.

- **Stopping**

A program does not stop a thread directly. Rather, a thread arranges for its own death by leaving its `run()` method.

Synchronization

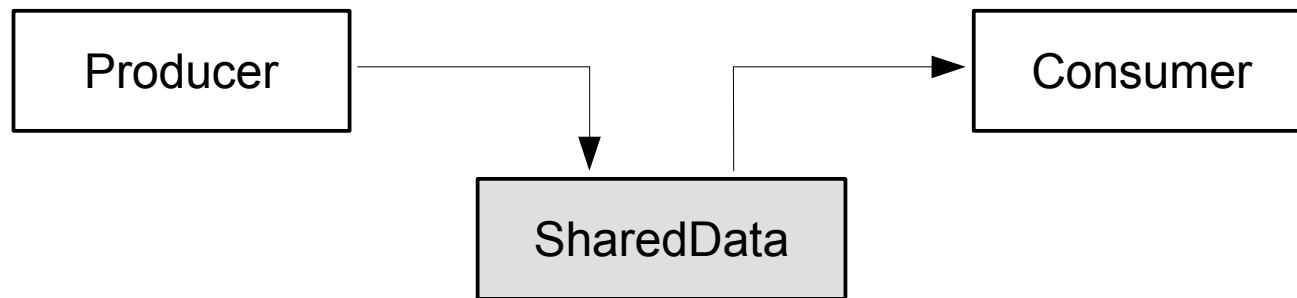
There are many interesting situations where separate, concurrently running threads do share data and must consider the state and activities of other threads.

Java provides a synchronization mechanism based on **monitors**. Each object in Java has a **lock** and a **monitor** to manage the lock. A block marked as **synchronized** forces any thread wishing to enter the block to acquire corresponding lock first. If another thread already holds the lock, the acquiring thread will **block** until the lock will be released. The lock is released when the thread leaves the block (newest versions of Java SE offer more complex ways to handle locks and deprecate this mechanism)

```
public class Game {  
    private Player black, white, previous;  
    public synchronized void turn(Player player) {  
        if (player == previous)  
            throw new IllegalArgumentException(...);  
        ...  
    }  
}
```

Producer-Consumer Problem

The producer-consumer problem is a classic synchronization problem. The producer and consumer processes share a common data. The producer executes a loop in which it puts new items into the data and the consumer executes a loop in which it removes items from the data.



```
public interface SharedData {  
    public int getSize();  
    public void put(int[] data);  
    public int[] get();  
}
```

Producer

The producer executes a loop in which it puts new items into the data store.

```
public class Producer extends Thread {  
    private SharedData data;  
  
    public Producer(SharedData data) { this.data = data; }  
    public void run() {  
        for (int i = 0; i < 10; i++) {  
            int[] array = new int[data.getSize()];  
            System.out.print("put");  
            for (int j = 0; j < array.length; j++) {  
                array[j] = i;  
                System.out.print(" " + array[j]);  
            }  
            System.out.println();  
            data.put(array);  
        }  
    }  
}
```

Consumer

The consumer executes a loop in which it removes items from the data store.

```
public class Consumer extends Thread {  
    private SharedData data;  
  
    public Consumer(SharedData data) { this.data = data; }  
  
    public void run() {  
        for (int i = 0; i < 10; i++) {  
            int[] array = data.get();  
            System.out.print("got");  
            for (int j = 0; j < array.length; j++)  
                System.out.print(" " + array[j]);  
            System.out.println();  
        }  
    }  
}
```

...

Naïve Shared Data

Traditional implementation of the `SharedData` interface seems to be all right, but ...

```
public class NaiveSharedData implements SharedData {  
    private int[] data;  
    public int getSize() { return data.length; }  
  
    public NaiveSharedData(int count) { data = new int [count]; }  
    public void put(int[] data) {  
        for (int i = 0; i < data.length; i++)  
            this.data[i] = data[i];  
    }  
  
    public int[] get() {  
        int[] data = new int[this.data.length];  
        for (int i = 0; i < data.length; i++)  
            data[i] = this.data[i];  
        return data;  
    }  
}
```

Putting It All Together

The producer-consumer problem can be assembled by a few lines of code.

```
SharedData data = new NaiveSharedData(3);  
Producer producer = new Producer(data);  
Consumer consumer = new Consumer(data);  
producer.start();  
consumer.start();
```

Output produced by the program is incorrect, because it contains inconsistent sequence, e.g. **0 0 1**.

```
...  
put 0 0 0  
put 1 1 1  
got 0 0 1  
got 1 1 1  
put 2 2 2  
...
```

Acquiring a Lock

The code segments within a program that access the same object from separate, concurrent threads are called **critical sections**. A critical is identified with the **synchronized** keyword. A lock is associated with every object that has synchronized code.

```
public class BetterSharedData extends NaiveSharedData {  
    public synchronized void put(int[] data) {  
        super.put(data);  
    }  
    public synchronized int[] get() {  
        return super.get();  
    }  
}
```

...

```
put 2 2 2  
got 2 2 2  
got 2 2 2
```


Releasing a Lock (1)

A thread **releases** an object's lock when it enters into `wait()` method of the object. The `wait()` method causes the thread to wait until another thread invokes the `notify()` or the `notifyAll()` method **for the object**.

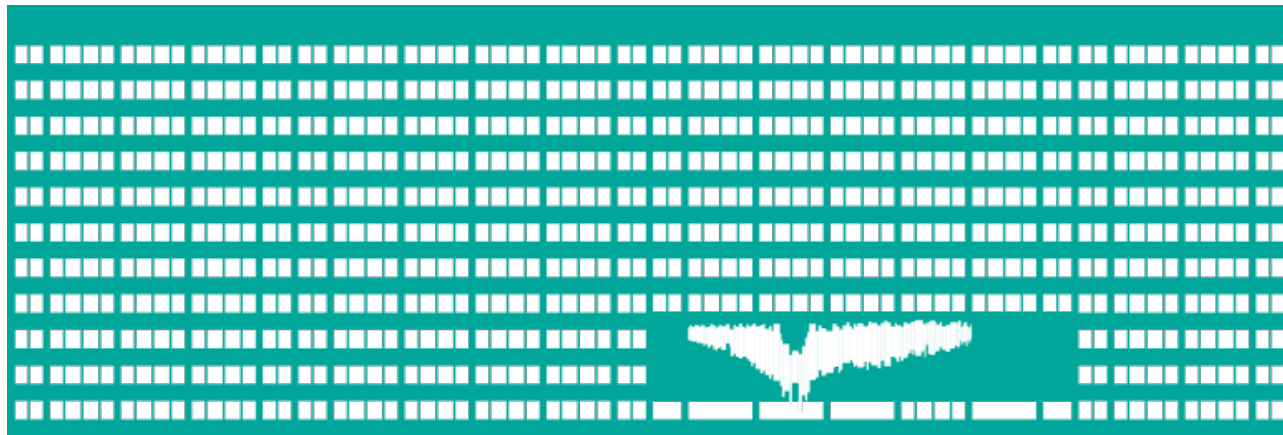
```
public class PerfectSharedData extends BetterSharedData {  
    private boolean read = true;  
    public synchronized void put(int[] data) {  
        try {  
            while (!read)  
                wait();  
            super.put(data);  
            read = false;  
            notifyAll();  
        }  
        catch (InterruptedException e) {...}  
    }  
}
```

Releasing a Lock (2)

The `InterruptedException` is thrown by the `wait()` method if another thread interrupts the current thread.

```
public synchronized int[] get() {  
    int data[] = null;  
    try {  
        while (read)  
            wait();  
        data = super.get();  
        read = true;  
        notifyAll();  
    }  
    catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
    return data;  
}
```

Templating Signing Good Practices Automated Build Tools



TAMZ 1
Lecture 12 – 13

Administrative FAQ

Student Presentations – Remarks

- Time for presentation is approximately 10 minutes (up to 20 min), followed by questions from instructor and fellow students. Topic must be chosen (and confirmed)
- 7-15 pt (minimal qualitative requirements must be met)
- Outline:
 - What is the prepared subject area about?
 - What are the differences against HTML5, OnsenUI, Cordova, PWA, ... (weak vs. strong points of this area)?
 - What are other interesting aspects of the subject area?
 - A short example (code sample on slides or demonstration)
 - Availability (are there any devices which implement this API, ...)
 - Q&A

Student Projects – Remarks

- The project should be handed-in in during labs in the last week of the semester and final version sent before the examination period starts
 - 10-30 pt are awarded (to award at least 10pts, minimal qualitative requirements must be met)
- The project must be able to run on at least on one actual device (phone, tablet) besides the emulator/browser
- The student must explain the function of developed application and understand the source code (which will be verified by instructor by asking random questions concerning the source code during the lab).
- After successful defense of the project, the project has to be sent to instructor (the method will be specified by the instructor)

Exams – Remarks

- The exam (55 pts) has three basic sections:
 - Theoretical section – single choice and fill-in questions covering the theory provided on lectures (and labs)
 - 4 questions → ~20pt
 - Question concerning the understanding of mobile application development → you are to develop this application, what are you going to use, is Apache Cordova required, ... → 10pt
 - Practical questions – practical knowledge from labs (and lectures) – jQM selectors and actions (code efficiency), what does this code do, fill-in the blanks in example code, write a part of a application (0-10 pt, based on how much the code matches the expected result) → 4 questions → ~25pt
- At least 20 pts from exam questions (51 total) to pass

Page Templates

(Underscore/lodash, mustache/handlebars)

Why Should we use Templating?

- Instead of placing custom code into your page, you may generate the content of forms/lists on client side in a way similar to server-side scripting
- The frameworks also offer methods similar to what jQuery offers and special constructs
- The constructs may be used in templates
 - Strings in JavaScript code
 - `<script>` tags in HTML code
- Easy to generate lists and input forms (presetting the values to stored values)
- Usually combined with some MVC framework
- We can, of course, use EcmaScript 6+ literal strings

Underscore/lodash at a Glance

- Uses `<% %>` sequences to enclose the JavaScript statements
- We may directly print a value by using `<%= %>`
- Offers a `_` object (like `$` for jQuery) with a set of advanced function for
 - Array processing (sorting, filtering, merging, ...)
 - Collections (mapping, reducing, conversions, shuffling, **forEach** processing, adjusting keys, searching, ...)
 - Comparison and `isXYZ` checking, conversions
 - Custom math functions, String processing, Object manipulation
 - Sequences
 - Custom and utility functions
 - ...

Underscore/lodash List Template

```
<script id="RuleItems" type="text/template">
  <% _.each(items.toJSON(), function( rule, id ) { %>
    <li data-filtertext="<%= rule.name %>, <%= rule.id %>">
      <a href="#edit?rule" data-ref="<%= id %>">
        <%= rule.name %></a>
      <% if (router.gV.collection.where({ruleId:
        rule.id}).length === 0) { %>
        <a data-ref="<%= id %>" class="ui-icon-delete"></a>
      <% } %>
    </li>
  <% }); %>
</script>
```

...

```
code = _.template( $("script#RuleItems").html(),
                    { "items": this.collection } );
```

Underscore/lodash form template

```
<script id="RuleEdit" type="text/template">
```

```
  <fieldset class="edit-container" >
```

```
    <label>Rules list</label><input type="text" name="name"  
required="required" value="<%= entry.name %>">
```

```
    <label>Commands</label>
```

```
    <textarea name="commands" placeholder="Commands set">
```

```
      <%= entry.commands %></textarea>
```

```
  </fieldset>
```

```
  <% if (mode === 'edit') { %>
```

```
    <div class="ui-grid-a ui-responsive">
```

```
      <div class="ui-block-a"><input type="submit" value="Change"  
class="itemChanged" data-iconpos="right" data-icon="check"></div>
```

```
      <div class="ui-block-b"><input type="submit" value="Insert a copy"  
class="itemAdded" data-iconpos="right" data-icon="plus"></div>
```

```
    </div>
```

```
  <% } else { %>
```

```
    <input type="submit" value="Insert" class="itemAdded" data-  
iconpos="right" data-icon="plus">
```

```
  <% } %>
```

```
</script>
```

Mustache/Handlebars

- Logic-less templates for many programming languages
- Uses templates enclosed in `{{` and `}}`
 - `{{template}}` for escaped HTML (`<abc/>` → `<abc/>`)
 - `{{{template}}}` for unescaped HTML (`<abc/>` → `<abc/>`)
 - `{{& template}}` to unescape HTML (`<abc/>` → `<abc/>`)
- Sections are possible (iteration over arrays)
 - Enclosed in `{{#section}}` and `{{/section}}`
 - Not only arrays, but a function may be used as well
 - Non-false values are rendered as a single item (0-1)
- Rendering something instead of section if section is empty – enclosed by `{{^section}}` and `{{/section}}`
- Comments are inside a block enclosed with `{{!` and `}}`
- Partial may be inserted in templates by `{{> partial}}`
- *Hashes* (data) are used with templates

Mustache/Handlebars List Template

```
{{#groups}}
  {{#text}}
    <li><strong>{{name}}</strong></li>
  {{/text}}
  {{^text}}
    <li>{{{name}}}</li>
  {{/text}}
{{/groups}}
{{#nonempty}}
  <p>The item is not empty.</p>
{{/nonempty}}
{{! just a comment}}
```

```
<li><strong>first</strong></li>
<li>second <b>best</b></li>
<li>third <i>in a row</i></li>
```

```
{
  "groups": [
    {
      "name": "first",
      "text": true,
      "id": 1
    },
    {
      "name": "second <b>best</b>",
      "id": 2
    },
    {
      "name": "third <i>in a row</i>",
      "id": 3
    }
  ],
  "nonempty": false
}
```

Backbone.js

Model-View-Controller Framework

Backbone.js/Spine.js

- Minimal, popular, generic MVC framework used for writing heavy HTML5 web & mobile applications
 - provides an object model with inheritance
- REST (REpresentational State Transfer) Sync
 - Collection Operations (POST = Create, GET = Read)
 - Model Operations (PUT = Update, DELETE = Delete)
 - The data will be immediately synced to server through AJAX
- Can be used with jQuery for manipulating DOM
- The framework may use custom routing between pages

Backbone.js – Components

- **Routers** – custom navigation between document fragments (hashes, pages), also possible with page (fragment) with History API. Replace jQuery navigation
- **Models** – represent single objects of given type, providing interactive data, representing the record (e.g. data from a form) and logic – conversions, validations, computed properties, access control.
- **Collections** – ordered sets of models with the possibility of adding/removing models and fetching data from server.
- **Views** – controlling the presentation of data, typically applying templates and rendering the output based on current action (list all, edit, ...)
- ***Backbone.sync*** – binding the local data to RESTful API (using JSON by default)

Application Signing

(Netbeans, Eclipse – automatically,
command line in other cases)

Permission Model – Reasons

- As mentioned in the first lecture, the mobile application typically runs inside of the sandbox
- Some API function are considered potentially dangerous, because they can affect resources outside of the sandbox and could be potentially used by malicious software (trojans, malware, ...) to:
 - Obtain personal information (e.g. retrieve the contacts, ...)
 - Spy the user (open stored files, capture photographs, record audio, ...)
 - Bring unwanted/unexpected costs (Internet, SMS, paid services, ...)
 - Make connections to other devices without user interaction (Bluetooth, IrDA, sockets, ...)
 - Change stored user files (data, images, ...)
 - ...

Permission Models

- On some platforms, we may install both signed and unsigned (untrusted applications)
 - Unsigned applications typically require more user interaction when permitting actions
 - Some APIs may be unavailable for unsigned applications
 - Sometimes the signing is not part of a platform, or is done by other means (e.g. HTTPS in mobile web)
- Working with application permissions differs:
 - Complex permission model, some permissions may be fine-tuned by the user in application properties to reduce number of questions (platform defines minimal required security level) → Java ME
 - The permissions are shown to the user during installation and update (esp. if they change) and once accepted, the user may not limit them → Android

Signing Remarks

- The certificate which is used to verify signing key should identify the developer and prove their identity.
- Signed applications may refuse to install a new version if the signing key does not match.
- Multiple applications signed with the same key, may share data folders with each other.
- What happens if the signing keys expires?
 - We are typically not able to install an application with expired or not yet valid key (clock change may help)
 - What if the key expires during the use of the application?
- Manufacturer (and sometimes the Operator/Provider) may have special keys, which enable full access to the device:
 - They may provide applications (e.g. for Backup/Restore) which normal developer is not able to create.
 - OS updates are often signed by such keys to prevent unauthorized changes on system partition (e.g. Android)

Signing Java-based applications

- Netbeans/Eclipse (and other IDEs) usually provide basic key management and an easy way to sign projects.
- If we need to work without them from command line, we may use following tools:
 - Certificate import :
keytool -import -alias *MyKey* -file *certificate_file*
 - Create test certificate (valid for one year):
keytool -genkey -alias *MyKey* -keyalg *RSA* -validity 365
keytool -export -alias *MyKey* -file *mycertificate.crt*
 - Sign JAR/APK:
jarsigner -keystore *MyKS* -storepass *KSPWD* *app_file* *MyKey*
 - e.g. **jarsigner** -verbose -sigalg *SHA1withRSA* -digestalg *SHA1* -keystore *my-release-key.keystore* *my_application.apk* *key_alias_name*
 - Verify the signature:
jarsigner -verify -verbose -certs *signed_app_file*

Programming Strategies (Good Practices)

General Recommendations

- Use the best-suited programming language for given task
 - JavaScript vs. Java vs. native C/C++ code on Android
- Move computation to the server where reasonable
 - Avoid running computationally intensive tasks on small device. Instead, let a server computer run them.
 - But this requires good and stable connectivity, we may consider an off-line fallback
- Separate the model
 - We need to adapt user interface to different platforms and device types. Commonly, we separate the logic of the application from the code that controls the presentation.
 - Separating models and presentations takes some effort, but makes it easier to adapt or rewrite user interface code (instead of the whole application)
- Avoid restricted APIs when possible (+watch instead of get)
 - You may consider to use alternative libraries for functions which require too much confirmations

Build Smaller Applications

A smaller application takes up less memory on the device and requires less time to install/download.

- Speed up startup
 - A significant part of the startup time is spent loading the application. Consider using application cache and CDN-hosted scripts if you do not package them with application
- Consider using jQuery, jQM, ... download builders to include only features you will be using in your code
- Use minification (+obfuscation for local names)
- Consider compression (gzip), HTML simplification, ...
- Also, exchange less data (use e.g. AJAX with JSON)

Optimize Loops

Avoid repeated invoking of a method if it is obvious that the result is the same. Cache that value. However, be sure that the value may not be changed by other means (e.g. in other thread)

Vector items = ...

```
int size = items.size();  
for (int i = 0; i < size; i++) {  
    ...  
}
```

String command = ...

```
int length = command.length();  
for (int i = 0; i < length; i++) {  
    ...  
}
```

Native Platform Good Practices

Use Less Memory at Run Time

The runtime memory capacity of a small computing device can be quite limited.

- **Use scalar types** – each object must be allocated from the runtime memory heap. The object's constructor runs as part of that allocation process. Therefore, each object impacts an application's performance as well as the amount of memory that it requires. To reduce the number of objects that are allocated, consider using **scalar types** – the non-object types such as **int** and **boolean** in place of objects whenever possible.
- **Help the garbage collector** – be sure to help the garbage collector do its work by setting object references to **null** whenever you are finished with them. Clearing out object references make it easier for the garbage collector to find and reclaim unreferenced objects.

Release Resources Early

Whether writing for small devices or for desktop systems, it makes sense to release resources **as soon as possible**.

- **Do not hang onto them longer than necessary**

This frees the resource for use by another application, but it also enables the system to free any memory associated with that resource.

- **Do not depend on finalizers**

Finalizers might never run, and as we will see later, they are not even supported by some Java interpreters.

Always provide methods for **explicitly** freeing the resources, and document their use.

Avoid Unnecessary Exceptions

Java and other modern programming languages provide built-in support for exception handling, which is extremely convenient.

- However, exceptions are sometimes overused. In general, you want to reserve exceptions for **unusual** or **unexpected** (exceptional) situations.
- Errors that are expected to occur in the normal course of running an application should be handled through **other means**. By avoiding exceptions, you can reduce the size of the class files and also the number of objects to be allocated (because e.g. in Java, each exception throws a new exception object).

Use Lazy Instantiation

To reduce overall and peak memory usage is to only allocate objects as they are needed. The technique is usually referred to as **lazy instantiation**.

```
class LazyInitialization {  
    private Vector items;  
  
    public Vector getItems() {  
        if (items == null)  
            items = new Vector();  
        return items;  
    }  
}
```

Reuse Objects

A technique that can pay good dividends is to reuse objects instead of continually allocating and abandoning (and thus deallocating) them.

```
class ReuseObject {  
    private Vector items;  
  
    public void initialize() {  
        if (items == null)  
            items = new Vector();  
    }  
  
    public void deinitialize() {  
        items.removeAllElements();  
    }  
}
```


Use Local Variables

It is generally slower to access class members than to access local variables. It also makes sense when dealing with arrays. Each time an array element is accessed, the Java interpreter performs a bounds check in order to ensure that the array index is valid.

```
char[] buffer = ...  
for (int i = 0; i < buffer.length; i++) {  
    char ch = buffer[i];  
    if (ch == '\r' || ch == '\n')  
        ...  
    else if (ch >= '0' && ch <= '9')  
        ...  
}
```

Avoid String Concatenation

Java and some other programming languages make it easy to build strings by concatenation. However, in many cases String concatenation involves several steps, e.g. in Java creating a new `StringBuffer` object, calling its `append()` method, and then calling its `toString()` method in order to obtain the final string. For such platforms, the **better** solution is to do most of the work yourself.

```
String concatStrings(String[] strings) {  
    StringBuffer result = new StringBuffer();  
    for (int i = 0; i < strings.length; i++)  
        result.append(strings[i]);  
  
    return result.toString();  
}
```

Use Buffered I/O

Don't read bytes one at a time from a stream, and don't write them out one at a time. Although the stream classes provide methods that read and write a single byte, you should avoid them if at all possible. It will almost always be more efficient to read or write a **whole array** full of data.

```
int read, total = 0;
byte buffer[] = new byte[1024];
do {
    read = input.read(buffer);
    total += read;
}
while (read == buffer.length);
```

Use Threads, Avoid Excess Synchronization

Threads are an important part of Java and other modern programming languages, and your application should take advantage of them whenever possible.

- The usual rule is that any operation that will take more than a **tenth of a second** to run should run on a separate thread so that it will not block the user interface.
- When using threads, controlled access to shared data have to be ensured. In Java, it we may use the **synchronized** keyword. But there is an overhead associated with handling of a lock and because of 2 lines of code, we should not make the whole method synchronized.

```
public Vector getItems() {  
    if (items == null) {  
        synchronized (this) {  
            if (items == null)  
                items = new Vector();  
        }  
    }  
}
```

...

Building Projects*

(Netbeans, Eclipse, Android, ...)

**Gradle and Maven (and sometimes ANT)
are used nowadays for Java projects
Gulp, Grunt, Bower, ... for JS*

Apache Maven

- Also XML-based
- More limited than ant, but has a different structure and relies on dependencies and tasks
- Problems are conflicts in the versions of the same library

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>cz.vsb</groupId>
  <artifactId>java-build-tools</artifactId>
  <packaging>jar</packaging>
  <version>1.0</version>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.11</version>
    </dependency>
    <dependency>
      <groupId>org.hamcrest</groupId>
      <artifactId>hamcrest-all</artifactId>
      <version>1.3</version>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>2.3.2</version>
      </plugin>
    </plugins>
  </build>
</project>
```

Gradle

- Groovy-based (not XML-based)
- Shorter and clearer scripts
- Custom dependency resolution engine
- Used for builds in Android (studio)

```
apply plugin: 'java'  
apply plugin: 'checkstyle'  
apply plugin: 'findbugs'  
apply plugin: 'pmd'
```

```
version = '1.0'
```

```
repositories {  
    mavenCentral()  
}
```

```
dependencies {  
    testCompile group: 'junit',  
                name: 'junit', version: '4.11'  
    testCompile group: 'org.hamcrest',  
                name: 'hamcrest-all', version: '1.3'  
}
```

JavaScript Bundlers

- Provide output JavaScript file with everything (your code in separate files, external dependencies) put together
- You may specify a config file to parametrize the bundler
- May use preprocessing of styles, e.g. with SASS (syntactically awesome style sheets)
- Most cooperate with node.js/npm
- Multiple bundlers (some support newer ES6+, some not)
 - Webpack
 - Rollup
 - Browserify
 - ESBuild
 - Parcel
 - ...

Webpack

- Uses a *build script*, a *config file*, *loaders* for file transformations and also supports plugins
Normally you use package.json build directive to execute the webpack, e.g.

```
"scripts": {  
  "build": "rm -rf dist && webpack --mode development"  
}
```
- Config file is named *webpack.config.js* (contains JS object *module.exports*), it defines entry point, output file and directory and inside module section the rules array for different file extensions

